Croqtile

Programming Guide

A guide to the Croqtile language

Version 1.0 — November 2026

LancerLab

Preface

Over the last few years the shape of the performance problem has changed. Model families have converged on a small number of designs, yet the number of operators that have to run well keeps multiplying. With the layer fixed, every remaining variation has to live in the kernel: the same mathematics appears with different sparsity, different positional encodings, different quantisation and different dataflow, and each variant wants a kernel of its own. At the same time the range of machines a program must serve keeps widening, and each new generation changes the programming interface enough that a kernel written for the previous one has to be re-derived rather than simply recompiled. Fused operators have become ordinary, because at scale a small efficiency gap is worth real money. The result is that the operator — not the model — is where effort concentrates, and writing one well remains among the most labour-intensive tasks in systems work: tile sizes, pipeline depths, cooperation between parallel regions and staging all interact, and the right combination changes with the shape of the data.

This is why a ladder of higher-level kernel languages has appeared in quick succession, nearly all of them in the last few years. They promise one program that can be reasoned about independently of the machine it runs on, and each rung trades some measure of control for author productivity: a program becomes shorter and easier to write, read and revise, because the language takes over decisions the author used to spell out by hand. The trade has a ceiling, though. The decisions that are easiest to hide are often the ones that separate a merely correct kernel from a fast one, so an abstraction that removes them makes the easy programs easier but caps what an expert can express.

Croqtile, a new-generation language for high-performance heterogeneous computing, was designed around that tension, and its first motivation was human productivity. Rather than raising the level of abstraction by concealing the machine, it compresses the programming abstraction: shapes and movement are first-class constructs, a complete kernel fits in a few dozen lines, and a structural decision changes in one or two statements instead of five scattered places. Symbolic dimensions let one program serve many input sizes. The compiler checks shapes and indices while the program is built and reports a violation in terms a person can act on. An engineer can therefore hold the whole kernel in view, change one idea at a time, and get a precise answer before running anything.

Only then does the second motivation come into play. As language models began to write and revise kernels, the same properties turned out to be what an automated agent needs: a compact program fits in a context window alongside many previous attempts, decoupled primitives make the space of structural variants enumerable instead of demanding coordinated rewrites, and compile-time rejection with structured diagnostics makes every failed attempt informative rather than silent. In that sense Croqtile is agent-ready — and AI-native — as a consequence of being a good language for a person to work in, not as a substitute for that goal.

Croqtile is embedded in C++: in a `.co` file, ordinary C++ prepares shaped buffers and calls a `__co__` function, whose body names the iterations, the tiles, the movement statements and the events that order them. The rest of this guide takes those constructs apart, explains what each one means, and puts them back together in complete programs.

Who this guide is for

This guide is written for developers who write or read Croqtile programs. It asks for C++ at the level of functions, pointers and arrays, and for the idea of a shaped tensor. No previous Croqtile experience is needed, nor any knowledge of a particular machine. It is not a book about compiler construction or about one specific platform. Its subject is the language: program structure, shapes and types, iteration and tile selection, data movement, coordination, C++ interoperability and diagnostics. By the end you will be able to build and run a `.co` file, read a kernel line by line, express tiling, movement and contraction in your own programs, and follow how the compiler reasons about shapes and parallel regions.

How to read this guide

The three chapters approach the same language from different directions. Chapter 1 takes you from an empty directory to a running program: building the compiler, using the command line, compiling and running a first example, working through a complete program, and following a debugging workflow. Chapter 2 is built from short, program-shaped examples that say what each construct means — signatures and shapes, the host API, iteration, tile selection, movement, coordination, branching, contraction, C++ embedding and diagnostics. Chapter 3 organises the language by function: program structure and compilation models, the type system, symbolic shapes, parallelism, tile selection, movement, contraction, coordination, interoperability and the reasoning behind them. Its sections open with the problem each one solves, so the chapter also works as a reference.

If this is your first Croqtile program, begin with the first chapter and work forward. Once you can read existing `.co` files with confidence, the third chapter can be taken in any order: its sections are independent, and each keeps the worked kernels next to the feature it demonstrates. Appendix A collects the syntax in one place, for looking up a spelling or a meaning without re-reading an example. It is worth consulting whenever the exact form of a construct escapes you.

Conventions

Croqtile code is set in a monospaced font. Longer examples appear as numbered listings, collected in the *List of Code Listings*; short fragments appear inline, for example `s32 [4, 8] output;` or `dma.copy src => shared`. Three kinds of callout box carry the running commentary: a **Chapter roadmap** opens each chapter with what it covers and where to go afterwards, a **Key point** marks an idea worth carrying forward, and a **Common pitfall** records a rule that is easy to get wrong. Examples use the `croq::` namespace for host-side helpers and `__co__` for the function that describes parallel work; the two forms appear together in almost every complete program.

Contents

Preface iii

1 Getting Started 1
- 1.1 What is Croqtile . . . 1
- 1.2 System Requirements . . . 2
- 1.3 Building the Compiler from Source . . . 3
- 1.4 Verifying the Installation . . . 3
- 1.5 The croqtile Command Line . . . 4
- 1.6 Development and Test Utilities . . . 7
- 1.7 Your First Program . . . 8
- 1.8 A Complete Program, End to End . . . 9
- 1.9 Inspecting and Debugging a Build . . . 12
- 1.10 Reading a Production .co File . . . 15

2 Language Basics 16
- 2.1 The Two Parts of a Croqtile Program . . . 16
- 2.2 Reading a Kernel Line by Line . . . 17
- 2.3 The Host-Side API: Inputs and Outputs . . . 18
- 2.4 Shapes and mdspan . . . 22
- 2.5 Integers, I-Tuples, and Bounded Values . . . 25
- 2.6 Spanned Data and Storage Specifiers . . . 28
- 2.7 Iteration: parallel-by, with-in, and foreach . . . 32
- 2.8 Tile Selection: chunkat, subspan, and view/from . . . 41
- 2.9 Data Movement: the DMA Statement and Futures . . . 46
- 2.10 Coordination: inthreads, Events, and Async Blocks . . . 53
- 2.11 Branching and Early Return . . . 57
- 2.12 The Four-Step MMA Syntax . . . 57
- 2.13 C++ Embedding, Calls, Macros, and the Preprocessor . . . 58
- 2.14 Printing, Assertions, and Compile-Time Inspection . . . 63
- 2.15 Syntax Quick Reference . . . 64

3 Programming Guide 66
- 3.1 Program Structure and Compilation Models . . . 66
- 3.2 Syntax Design and the Type System . . . 69
- 3.3 Symbolic Shapes, Shape Evaluation, and Value Numbering . . . 74
- 3.4 Parallelism, Space Specifiers, and Masking . . . 81
  - 3.4.1 Worked Example: Tiled Matrix Multiplication . . . 86
  - 3.4.2 Tracing One Output Element . . . 87
- 3.5 Tile Selection and Data Expressions . . . 87
  - 3.5.1 Arbitrary-Offset Windows: view and from . . . 89

3.5.2 Strided Tiles: subspan, step, and at . . . 89
3.5.3 Zero-Padding with zfill . . . 90
3.5.4 Layout Reinterpretation with span_as . . . 91
3.5.5 Tile Selection: Summary and Syntax . . . 91
3.6 Data Movement: Operations, Configuration, and Async Patterns . . . 93
3.6.1 Worked Example: DMA Matmul . . . 97
3.6.2 Multi-Buffering: dma.any, swap, and rotate . . . 99
3.6.3 Worked Example: A Pipelined Kernel . . . 101
3.7 Contraction and Layout Contracts . . . 102
3.7.1 Worked Example: One Cooperative Group . . . 106
3.7.2 Worked Example: Widening the Cooperation Scope . . . 107
3.7.3 Worked Example: Two Cooperative Groups . . . 109
3.7.4 MMA Syntax Summary . . . 110
3.7.5 MMA Design Summary . . . 110
3.8 Coordination, Pipelines, and Persistent Scheduling . . . 110
3.8.1 Canonical Pattern: One Producer and One Consumer . . . 115
3.8.2 Worked Example: A 1P1C Kernel Skeleton . . . 115
3.8.3 Persistent Scheduling . . . 116
3.8.4 Host-Level Concurrency and Stream Binding . . . 117
3.8.5 Worked Example: A Full Event Pipeline . . . 119
3.8.6 Coordination Pattern Summary . . . 121
3.9 C++ Interoperability and the Preprocessor . . . 122
3.9.1 Worked Example: Calling a Device Helper . . . 125
3.9.2 Verbatim C++ Injection . . . 126
3.9.3 Interoperability Summary . . . 126
3.10 Tileflow Patterns and Design Rationales . . . 126
3.10.1 Worked Example: Tiled matmul . . . 128
3.10.2 Worked Example: rgb2gray . . . 129
3.11 Diagnostics and Methodology . . . 130

A Syntax Quick Reference 131
A.1 Declarations and shapes . . . 131
A.2 Iteration . . . 131
A.3 Tile selection and movement . . . 131
A.4 Statement modifiers and contraction . . . 131
A.5 Coordination, interop and diagnostics . . . 132

References 133

List of Code Listings

1.1 Clone and prepare . . . 3
1.2 Build the compiler . . . 3
1.3 Run the test suite . . . 3
1.4 A minimal Croqtile program . . . 4
1.5 Compile and run the check . . . 4
1.6 Compiler Usage . . . 4
1.7 Compiling a first program . . . 4
1.8 Listing the options . . . 5
1.9 Default invocation . . . 6
1.10 Emitting lowered source . . . 6
1.11 Full option list . . . 6
1.12 Everyday development shortcuts . . . 7
1.13 Listing Makefile targets . . . 7
1.14 Preparing a fresh checkout . . . 7
1.15 Building the configured tree . . . 7
1.16 Running the unit tests . . . 8
1.17 All sample operator tests . . . 8
1.18 One sample operator . . . 8
1.19 A complete element-wise addition . . . 8
1.20 Compile and run the example . . . 9
1.21 How .co files compile . . . 9
1.22 A Full Croqtile-target C Code Example . . . 10
1.23 Host Program - the Control Center . . . 11
1.24 Host Program - the Control Center . . . 11
1.25 Host Program - the Control Center . . . 12
1.26 Verbose mode: -v . . . 12
1.27 Runtime checks: -rtc . . . 13
1.28 Debug RTTI and debugger . . . 13
1.29 Debug RTTI and debugger . . . 13
1.30 Guarding debug prints . . . 14
1.31 Debug and production builds . . . 14
2.1 The three regions of one source file . . . 16
2.2 A kernel signature . . . 17
2.3 Allocating the output . . . 17
2.4 The parallel body . . . 17
2.5 Returning the result . . . 18
2.6 Creating the shaped inputs . . . 18
2.7 Calling the kernel . . . 18
2.8 Host Code . . . 19
2.9 Parameters Passing between Croqtile and C Function . . . 19

2.10 Input Parameters . . . 20
2.11 Passing a shaped buffer . . . 20
2.12 The host-side buffer wrapper . . . 20
2.13 A call built from runtime dimensions . . . 20
2.14 Return Type and Type Deduction . . . 21
2.15 Returning a shaped buffer . . . 21
2.16 The host-side result wrapper . . . 21
2.17 span and span(i) . . . 22
2.18 Declaring a shape . . . 23
2.19 A rank mismatch . . . 23
2.20 Omitting the mdspan keyword . . . 23
2.21 The original shape . . . 23
2.22 A derived shape . . . 24
2.23 Rebinding a shape . . . 24
2.24 A kernel with an unknown dimension . . . 24
2.25 The caller supplying that dimension . . . 24
2.26 Step Further: Symbolic Dimension . . . 25
2.27 Scalar Types . . . 25
2.28 I-Tuple Types . . . 26
2.29 Operations over mdspan and i-tuple . . . 26
2.30 Integers for Program Control . . . 26
2.31 Integers for Program Control . . . 27
2.32 Integers for Program Control . . . 27
2.33 Integers for Program Control . . . 27
2.34 Integers for Program Control . . . 27
2.35 Defining I-Tuples . . . 27
2.36 Defining I-Tuples . . . 28
2.37 Operations on I-Tuples . . . 28
2.38 Operations on I-Tuples . . . 28
2.39 Operations on I-Tuples . . . 28
2.40 Fully-Typing . . . 29
2.41 The Storage Qualifier . . . 29
2.42 Defining a Spanned Data . . . 29
2.43 Defining a Spanned Data . . . 30
2.44 Defining a Spanned Data . . . 30
2.45 Storage Specifier . . . 30
2.46 Initialization . . . 31
2.47 Declaring the Parameters and Return Values . . . 31
2.48 Declaring the Parameters and Return Values . . . 31
2.49 1. global . . . 31
2.50 2. shared . . . 32
2.51 3. local . . . 32
2.52 Default Behavior and Usage . . . 32
2.53 Default Behavior and Usage . . . 32
2.54 Example Code: Using Storage Qualifiers . . . 32
2.55 Croqtile s Two-Layer Parallelism Model . . . 33
2.56 : block — parallel blocks . . . 34
2.57 : thread — Threads Within a Block . . . 34
2.58 Nesting Specifiers . . . 34
2.59 Parallel Region: the parallel-by Block . . . 34
2.60 The with-in Block and where Clause . . . 35

2.61 The with-in Block and where Clause 35
2.62 The with-in Block and where Clause 35
2.63 The with-in Block and where Clause 35
2.64 The foreach Block 36
2.65 Basic Syntax: parallel-by 36
2.66 Basic Syntax: parallel-by 36
2.67 Multiple-Level Parallelism 36
2.68 Multiple-Level Parallelism 37
2.69 Sub-Level Parallelism 37
2.70 Loops in Croqtile 38
2.71 Loops in Croqtile 38
2.72 Defining Bounded Variables: 38
2.73 Defining Bounded Variables: 38
2.74 Defining Bounded Variables: 39
2.75 Basic Syntax 39
2.76 Basic Syntax 39
2.77 Basic Syntax 39
2.78 Basic Syntax 39
2.79 Basic Syntax 40
2.80 Syntactic Sugar 40
2.81 Syntactic Sugar 40
2.82 Deriving the Loop From a Bounded Integer 40
2.83 Deriving the Loop From a Bounded Integer 40
2.84 Deriving the Loop From a Bounded Integer 40
2.85 chunkat(...): equal partitioning 41
2.86 subspan(...).at(...): explicit tile extents on a tile grid 41
2.87 view(...).from(...): explicit tile extents with explicit element offsets 42
2.88 Compose: outer inner 42
2.89 Extent: name 43
2.90 Bounded-ituple/integer and chunkat Operation 44
2.91 Ubound Operation 44
2.92 Use chunkat Expression to Tile Data 44
2.93 Use Multiple Bounded Variables 45
2.94 The DMA Statement Family 46
2.95 The DMA Statement Family 46
2.96 First Example: Tiled Addition Through shared storage 47
2.97 Futures, .data, and .span 48
2.98 Futures, .data, and .span 48
2.99 Synchronous and Asynchronous Copies 48
2.100Synchronous and Asynchronous Copies 49
2.101Synchronous and Asynchronous Copies 49
2.102dma.transp 49
2.103dma.pad 49
2.104Tileflow Program: Orchestrating the Data Movement 49
2.105Tileflow Program: Orchestrating the Data Movement 50
2.106Basic Syntax 51
2.107Synchronous and Asynchronous 52
2.108Data Expression 52
2.109Future and Wait 53
2.110Future and Wait 53
2.111Event arrays for multi-stage pipelines 54

2.112 Bootstrap protocol . . . 54
2.113 Parallel Perspective of Tileflow Programs . . . 54
2.114 Implicit Masking . . . 55
2.115 Event Basics . . . 56
2.116 Event Basics . . . 56
2.117 Event Basics . . . 56
2.118 Event Basics . . . 56
2.119 Predicated execution with if . . . 57
2.120 Early return from parallel . . . 57
2.121 Macros inside cpp strings . . . 58
2.122 Macros inside cpp strings . . . 58
2.123 Tile geometry as macros . . . 59
2.124 Compile-time assertions with error . . . 59
2.125 Conditional code paths . . . 59
2.126 Function Calls and Call Croqtile Function . . . 60
2.127 Object-Like Macros . . . 60
2.128 Object-Like Macros . . . 60
2.129 Conditional Compilation . . . 61
2.130 Basic Syntax of Call Statements . . . 61
2.131 Basic Syntax of Call Statements . . . 62
2.132 The Convention: Allowed Argument Types . . . 62
2.133 print and println : compile-time inspection . . . 63
2.134 print and println : compile-time inspection . . . 63
2.135 print and println: runtime device output . . . 63
2.136 print and println: runtime device output . . . 63
2.137 assert: runtime invariant checks . . . 64
3.1 Transpilation and Compilation . . . 67
3.2 Device Program: Parallel Computation . . . 68
3.3 Device Program: Parallel Computation . . . 68
3.4 Heterogeneity: Transpiling to program launch . . . 68
3.5 Embedding Croqtile within C . . . 69
3.6 The Partial Type: mdspan . . . 71
3.7 The Partial Type: mdspan . . . 71
3.8 The Partial Type: mdspan . . . 71
3.9 The Partial Type: mdspan . . . 71
3.10 Values of Bounded Variables . . . 73
3.11 Improved Code Safety . . . 75
3.12 Improved Code Safety . . . 75
3.13 Improved Code Safety . . . 76
3.14 Cost and Limitations . . . 76
3.15 a. Improved Readability and Clarity . . . 77
3.16 b. Type Safety and Compile-Time Checking . . . 77
3.17 c. Expressiveness and Flexibility . . . 77
3.18 Quick Example . . . 78
3.19 Quick Example . . . 78
3.20 Simplifications . . . 79
3.21 Simplifications . . . 80
3.22 Simplifications . . . 80
3.23 : group-4 — cooperative group . . . 81
3.24 Explicit Masking/Condition . . . 83
3.25 Implicit Masking . . . 83

3.26 Experimental: Optional Where-Binding for within . . . 84
3.27 Masking for Multi-Levels . . . 85
3.28 Scalar Matmul (No DMA) . . . 86
3.29 Tile and Flow . . . 88
3.30 Arbitrary-offset windows: view and from . . . 89
3.31 Arbitrary-offset windows: view and from . . . 89
3.32 Strided tiles: .subspan, .step, and .at . . . 90
3.33 Zero-padding: .zfill . . . 90
3.34 Layout reinterpretation: span as . . . 91
3.35 Layout reinterpretation: span as . . . 91
3.36 DMA vs TMA . . . 94
3.37 The main TMA constraint . . . 94
3.38 The main TMA constraint . . . 94
3.39 tma.copy: target tensor movement . . . 95
3.40 Async Operation: the DMA Statement . . . 96
3.41 Async Operation: the DMA Statement . . . 96
3.42 Operation Type . . . 97
3.43 Full Non-Blocking DMA Mode (Chain Mode in Croqtile) . . . 97
3.44 DMA Matmul: Tiles in shared storage . . . 98
3.45 Double buffering with swap . . . 99
3.46 Multi-Buffering Optimization with Croqtile . . . 100
3.47 Example: TMA in a pipelined matmul . . . 101
3.48 Partial Tiles, Guarding, and .zfill . . . 102
3.49 .swiz N : Shared-Memory Layout for the Consumer . . . 103
3.50 Beyond dense FP16: what else the four steps express . . . 105
3.51 Beyond dense FP16: what else the four steps express . . . 105
3.52 Beyond dense FP16: what else the four steps express . . . 105
3.53 Swizzle and layout conflicts . . . 105
3.54 Swizzle and layout conflicts . . . 106
3.55 One group, one tile: the simplest MMA matmul . . . 106
3.56 Widening the scope: cooperative group and mma . . . 107
3.57 Tiling further: two cooperative group sharing operands . . . 109
3.58 Structured concurrent regions with inthreads.async . . . 111
3.59 Inter-region signaling with events . . . 112
3.60 Make it Asynchronous . . . 113
3.61 Event Instances . . . 114
3.62 DeadLocks . . . 114
3.63 DeadLocks . . . 114
3.64 The canonical pattern: 1 producer 1 consumer . . . 115
3.65 A 1P1C matmul skeleton . . . 115
3.66 Persistent scheduling and the if guard . . . 116
3.67 parallel.async: host-level concurrency . . . 118
3.68 Asynchronous program launch and Stream Binding . . . 118
3.69 Synchronization and Asynchronous parallel-by . . . 118
3.70 Synchronization and Asynchronous parallel-by . . . 118
3.71 Stream Binding . . . 119
3.72 Stream Binding without .async . . . 119
3.73 The full 1P1C event pipeline . . . 120
3.74 Difference with C pre-processing . . . 123
3.75 Trigger C Template Instantiation . . . 123
3.76 Trigger C Template Instantiation . . . 123

3.77 Factor Target Specific Details 124
3.78 Factor Target Specific Details 124
3.79 device functions: target alongside Croqtile 125
3.80 storage hints: setmaxnreg 126
3.81 Example Code: Matrix Multiplication with Tiling 128
3.82 Example Code 130

Chapter 1

Getting Started

Chapter roadmap

This chapter takes you from the question of why Croqtile exists to a program you have built and run yourself. It begins with the motivation for the language and the properties it was designed around — zero-cost abstractions, compile-time safety checks, dynamic shapes and symbolic dimensions, and a form that automated tuning can work with — then follows a step-by-step path: the smallest kernel that compiles, progressively richer matmul variants, and the toolchain needed to build and inspect each one. By the end you will be able to reproduce the output of a simple program on your own.

1.1 What is Croqtile

Before installing anything, it is worth knowing what problem Croqtile was built for. This section begins with that problem and the single idea behind the language, then says what Croqtile is, what its constructs give you, and what you need in order to follow the examples.

The programs at the centre of modern workloads are often short. A matrix multiplication, a normalization step, an attention block: each is a few lines of mathematics. Yet turning those lines into a program that runs well has become a specialist's job, and one that has to be done again whenever a shape, a dataflow or a machine changes. The reason is rarely the arithmetic. It is that the same small computation also has to say where its data lives, how it is divided into tiles, in what order those tiles become available, and how concurrent regions agree on progress. In most settings those decisions are scattered across library calls, build options and code a reader has to infer, so changing one of them quietly changes several others. A single family of matrix multiplications already produces a long series of variants, each with its own tiling, movement and coordination; keeping them consistent is where most of the effort goes.

Croqtile begins with a single premise: if those decisions were part of the language itself, a person could state them once, in one place, and let the compiler check them together before producing the program that runs. Croqtile is a C++ embedded DSL for high-performance kernel programming, designed from the ground up around four properties: an easy-to-use syntax whose abstractions cost nothing at run time, comprehensive compile-time safety checks, first-class dynamic shapes and symbolic dimensions, and a form that AI tuning tools can work with. A program states the shapes, iterations, tiles, movement and coordination it needs; the compiler turns that description into runnable code. The examples in this guide grow by one idea at a time, so a later listing usually differs from an earlier one only by the construct that section introduces.

Consider what happens when a computation needs a different tile size. In a program where tiling, copying and coordination are written separately, that one change reaches into the loop bounds, the address arithmetic, the staging buffers and the coordination points. In Croqtile the iteration declares the tile, the tile selects the data, and a movement statement says when it is ready; the change stays local, and the compiler rejects the combinations that do not fit. That is the key benefit of making tiles part of the language: the person writing the program keeps the decisions, while the compiler takes on the bookkeeping that used to tie them together. It also means a complete kernel is short enough to read in one sitting, which is how the examples here are built.

The vocabulary follows the way a kernel is put together. A `__co__` function describes the parallel work and sits beside the ordinary C++ that prepares buffers through the host API. Shaped types built on MDSpan and spanned data carry symbolic dimensions that need not be fixed until the program runs. Iteration constructs declare independent regions and loop over tiles, DMA statements and `chunkat` select and move those tiles, and events order the regions that share them. Where plain C++ is needed again, `__cpp__` blocks, macros and the preprocessor are available in the same file. Appendix A collects the exact spelling of each form, and the chapters that follow put them to work in complete programs.

Prerequisites

- Basic C++ (functions, pointers and arrays) and the idea of a shaped tensor
- Some familiarity with parallel programs: independent work items, tiles, and coordination between concurrent regions
- A machine on which you can build the compiler (see Section 1.3)

Of course, putting those decisions in the language does not make writing a kernel trivial; it moves the difficulty into the compiler, where it can be discharged once instead of rediscovered by every program. The compile-time safety checks have to decide shapes and indices before anything runs. Symbolic dimensions have to survive until the program runs, so one source serves many input sizes. The abstractions have to stay zero-cost, and the constructs have to stay few enough that a person can hold all of them in mind. Those constraints shape almost every rule in the following chapters; giving you a sense of how the compiler meets them is one of the purposes of this guide.

From here the path is step by step. The chapters that follow start with the smallest kernel that compiles and evolve a single running example through progressively richer matmul variants, so each new idea appears in a concrete, working program and every major construct is met before the end. The next step is a practical one: build the compiler so that each of those programs can be run rather than only read. Section 1.2 lists what you need, and Section 1.3 walks through the build.

1.2 System Requirements

Croqtile ships as source, so a working toolchain comes before the first program. This section lists what the build needs and why those versions matter; Section 1.3 gives the commands.

Croqtile is built from source rather than installed as a prebuilt package. The compiler is itself a C++ program whose parser is generated from a grammar, so the first thing to check is the toolchain that builds it. The table below gives the versions that are known to work: a C++17-capable compiler and two parser-generator tools.

Dependency	Minimum Version
GCC	8.1+ (or Clang 6+)
Bison	3.8+
Flex	2.6.4+

Most Linux distributions ship Flex but an older Bison. If the Bison on your system is below 3.8, the setup step described in Section 1.3 fetches a compatible version on its own, so the distribution package does not have to be replaced by hand.

The version floor is not arbitrary. GCC 8.1 and Clang 6 are the first releases with complete C++17 support, and Bison 3.8 and Flex 2.6.4 are the versions whose generated code supports the C++ features the compiler uses. An older Bison is the usual obstacle, and it is the one the setup step resolves for you.

1.3 Building the Compiler from Source

A checkout needs three commands before it is usable: one prepares the dependencies, one builds the compiler, and one runs the test suite. The test result is what tells you the binary is ready, and Section 1.4 gives the smallest program that confirms it end to end.

Clone the repository and run the automated setup:

```
git clone https://github.com/LancerLab/croqtile.git
cd croqtile
make setup-core
```

Listing 1.1: Clone and prepare

`make setup-core` pulls git submodules and downloads the required versions of Flex, Bison, FileCheck, and GoogleTest if they are not already present. Once setup finishes, build the compiler:

```
make
```

Listing 1.2: Build the compiler

Then run the test suite; it is the check that the freshly built compiler is sound:

```
make test
```

Listing 1.3: Run the test suite

If all tests pass, the `croqtile` binary is ready in your build directory. Add it to your `PATH` or invoke it by its full path. Section 1.4 compiles a minimal program to confirm the installation end to end.

1.4 Verifying the Installation

A four-element copy is enough to exercise the whole chain: the compiler parses the program, the generated code compiles and links, and the host program reads back the result. Section 1.5 looks at what else the command line can do.

The smallest useful check is a kernel that copies four integers. Save it as `verify.co`:

```
__co__ s32 [4] identity(s32 [4] input) {
  s32 [input.span] output;
  parallel i by 4
    output.at(i) = input.at(i);
  return output;
}
int main() {
  auto input = croq::make_spandata<croq::s32>(4);
  input[0] = 1; input[1] = 2; input[2] = 3; input[3] = 4;
  auto result = identity(input.view());
  for (int i = 0; i < 4; ++i)
    if (input[i] != result[i]) { std::cerr << "FAIL\n"; return 1; }
  std::cout << "OK\n";
}
```

Listing 1.4: A minimal Croqtile program

Compile and run:

```
croqtile verify.co -o verify
./verify
```

Listing 1.5: Compile and run the check

You should see `OK`. If you do, the compiler, linker, and runtime are all working. Section 1.5 lists the options that change what the command produces.

1.5 The croqtile Command Line

The `croqtile` command turns a `.co` file into a runnable program. This section maps each option to the stage it controls.

Compiler Usage

The `croqtile` command works like `gcc` or `clang`:

```
croqtile program.co                          # compile and link -> a.out
croqtile program.co -o my_kernel             # specify output name
croqtile -es -t target program.co -o out.cu  # emit target source only
croqtile -E program.co                       # preprocess only
```

Listing 1.6: Compiler Usage

Key flags:

Build and Compile

For a first program the same command is enough. Files use the `.co` extension, and the default invocation produces `a.out`:

```
croqtile ele_add.co                             # produces a.out
croqtile ele_add.co -o ele_add                  # specify output name
croqtile -es -t target ele_add.co -o out.cu    # emit target source only
```

Listing 1.7: Compiling a first program

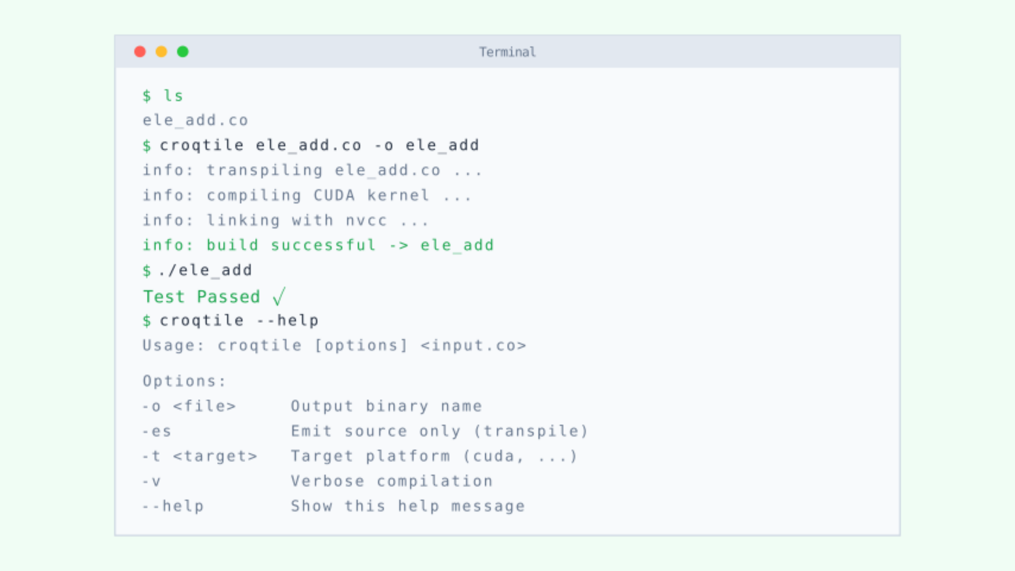
Flag	**Effect**
`-o <file>`	Set output filename
`-t <platform>`	Select target platform (e.g. `target`)
`-es`	Emit target source code without compiling it
`-E`	Preprocess only (expand macros, strip `#if 0` blocks)
`-c`	Compile without linking
`-S`	Emit assembly
`-help`	Show all options
`-help-hidden`	Show advanced/internal options

The `-t` flag selects the target platform.
To see all available options, run `croqtile --help`:

```
croqtile --help
```

Listing 1.8: Listing the options

This prints the full set of compiler flags — output naming, target selection, verbosity, and diagnostic options. It is worth scanning once so you know what is available when you need it.



```
$ ls
ele_add.co
$ croqtile ele_add.co -o ele_add
info: transpiling ele_add.co ...
info: compiling CUDA kernel ...
info: linking with nvcc ...
info: build successful -> ele_add
$ ./ele_add
Test Passed √
$ croqtile --help
Usage: croqtile [options] <input.co>

Options:
-o <file>      Output binary name
-es            Emit source only (transpile)
-t <target>    Target platform (cuda, ...)
-v             Verbose compilation
--help         Show this help message
```

Compiling and running a Croqtile program, plus the `-help` *output.*

Compile Croqtile Program

Croqtile is a source-to-source translator: it turns a `.co` program into C++ source that a downstream compiler consumes. With that compiler in place the chain behaves like an end-to-end compiler, so the command line can offer the usual output forms.

Once the compiler is built, a program can be lowered into several output forms:

- Target source code
- Target object module
- Target executable binary/module

- Target assembly
- Work-script

All but the first and last are the familiar forms from a C++ toolchain. Which of them a build can produce depends on what the downstream compiler supports.

The command itself has the usual shape:

```
croqtile your_program.co
```

Listing 1.9: Default invocation

This command generates an `a.out` executable file. The `-o <filename>` option can be used to specify the output filename when needed. Compiler options like `-c` and `-S` work similarly to those in C++ compilers, provided they are supported by the target platform.

The `-t <platform>` option allows you to specify the target platform for the compilation. Additionally, the `-es` option generates *target source code* without performing the downstream compilation. For instance:

```
croqtile -t target your_program.co -es -o target_source.co
```

Listing 1.10: Emitting lowered source

Notably, Croqtile allows the `-E` option to support Croqtile-only preprocessing. The Croqtile preprocessor handles simple macros and preprocessor directives such as `#if`, `#ifdef`, `#ifndef`, `#else`, and `#endif`, enabling Croqtile functions to be integrated with other C++ code. The `-E` option outputs the preprocessed code, for example, removing code within `#if 0` and `#endif` directives inside Croqtile functions.

Furthermore, in development scenarios, Croqtile can generate *work-script* (using the `-gs` option) to drive further low-level compilation and execution. This facilitates the development process, as many scripts are integrated for easy debugging.

Lastly, options `-help` and `-help-hidden` are available for listing the full option set. Programmers and users can check the list to find their appropriate usage.

Here is an example of the output of `-help`:

```
// --help
Usage: croqtile [options] file...
Options:
  --help                    Display this information.
  --help-hidden             Display hidden options.
  -e, --dump-ast            Dump the Abstract Syntax Tree (AST) after parsing.
  -i, --infer-types         Show the result of type inference.
  -bf16n, --native-bf16     Utilize native bf16 type when target platform
    support.
  -f16n, --native-f16       Utilize native f16 type when target platform support.

  -n, --remove-comments     Remove all comments in non-croqtile code. (Useful
    for FileCheck)
  -t, --target <platform>   Set the compilation target. Use '--help-target' to
    show current supported targets.
  -v, --verbose             Display the programs invoked by the compiler.
  -E                        Preprocess only; do not compile.
  -arch=<processor>         Set the target configuration to execute the binary
    code.
  -c                        Compile croqtile code and the generated target code;
     Without linking.
```

```
-es                          Emit target source file without target source
  compilation.
-gs                          Generate target script.
-o <file>                    Place the output into <file>.
```

Listing 1.11: Full option list

1.6 Development and Test Utilities

Beyond compiling one file, the build tree offers its own test targets and shortcuts. They are worth knowing before the first real program, because every example that follows can be checked with them.

Development Utilities

When you change Croqtile itself, three shortcuts are enough to narrow a failure from the whole suite down to one operator family:

```
make help                                # list all available targets
make sample-test                         # run all sample operator tests
make sample-test-operator OPERATOR=add # test a specific operator
```

Listing 1.12: Everyday development shortcuts

They are the commands to reach for when a build passes but an operator does not, or when a change to the compiler needs checking against the samples.

With the compiler installed and verified, you are ready to write your first real Croqtile program in Section 1.7.

Environment

The build tree has its own set of shortcuts, and `make help` is the index to them:

```
make help
```

Listing 1.13: Listing Makefile targets

A fresh checkout is prepared with `make setup`. It fetches the tools the build and test steps need — Flex, Bison 3.8 or newer, FileCheck and the GoogleTest sources — so the result does not depend on which versions the system already has:

```
make setup
```

Listing 1.14: Preparing a fresh checkout

Once the environment is configured, the compiler is built with:

```
make
```

Listing 1.15: Building the configured tree

Croqtile is still in development, so the unit tests are the check that the binary is sound. A failing run stops and reports the error rather than leaving a build that only breaks later:

```
make test
```

Listing 1.16: Running the unit tests

The sample operators can be tested as a whole family, or narrowed to a single one. The first command runs every sample:

```
make sample-test
```

Listing 1.17: All sample operator tests

The second selects one operator — `add`, `mul`, `relu`, `sigmoid`, `softmax` or `tanh`:

```
make sample-test-operator OPERATOR=add
```

Listing 1.18: One sample operator

1.7 Your First Program

Your first Croqtile program is a complete unit: a `__co__` function and an ordinary C++ `main()`. It is small enough to read in one sitting, and the rest of the guide grows from it.

The shortest program that shows the shape of the language is an element-wise addition of two small matrices. It has no tiling, no explicit data movement and no coordination between regions; those are the constructs every later example adds. What it does have is the full skeleton of a Croqtile program: a `__co__` function that describes the parallel work, and an ordinary C++ `main()` that prepares the inputs, calls it and checks the result. Every program in the guide is built from these two halves.

A Complete Example: Element-Wise Addition

Here is the full program. It adds two `[4, 8]` matrices of 32-bit integers element by element:

```
__co__ s32 [4, 8] ele_add(s32 [4, 8] lhs, s32 [4, 8] rhs) {
  s32 [lhs.span] output;
  parallel {i, j} by [4, 8]
    output.at(i, j) = lhs.at(i, j) + rhs.at(i, j);
  return output;
}
int main() {
  auto lhs = croq::make_spandata<croq::s32>(4, 8);
  auto rhs = croq::make_spandata<croq::s32>(4, 8);
  lhs.fill_random(-10, 10);
  rhs.fill_random(-10, 10);
  auto res = ele_add(lhs.view(), rhs.view());
  for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 8; ++j)
      croq::croq_assert(lhs[i][j] + rhs[i][j] == res[i][j],
                        "values are not equal.");
  std::cout << "Test Passed\n" << std::endl;
}
```

Listing 1.19: A complete element-wise addition

Save this as `ele_add.co`, compile, and run:

```
croqtile ele_add.co -o ele_add
./ele_add
```

Listing 1.20: Compile and run the example

You should see `Test Passed`. The chapters that follow take this same program apart, one construct at a time.

1.8 A Complete Program, End to End

A production `.co` file is three programs in one file: device code, a tileflow `__co__` function, and the host C++ that prepares buffers and reads the result back. The boundaries between them are what the rest of this guide explains.

How `.co` files compile

A `.co` file mixes three kinds of code. The Croqtile compiler treats each differently — and the difference matters for understanding what you can and cannot do at each boundary. Consider this skeleton:

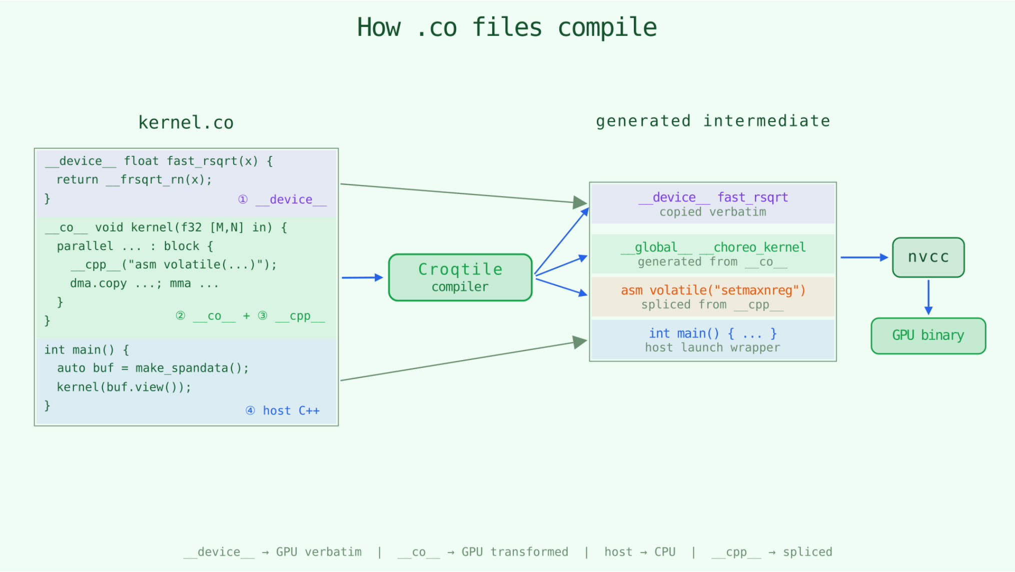
```
__device__ __forceinline__ float fast_rsqrt(float x) {   // 1
  return __frsqrt_rn(x);
}
__co__ void my_kernel(f32 [M, N] input, f32 [M, N]& output) {   // 2
  parallel {bm, bn} by [cdiv(M, 64), cdiv(N, 64)] : block {
    // ... Croqtile orchestration: parallel, dma, mma, events ...
    __cpp__("asm volatile(\"setmaxnreg.dec.sync.aligned.u32 40;\");");   // 3
  }
}
int main() {   // 4
  auto in = croq::make_spandata<croq::f32>(M, N);
  auto out = my_kernel(in.view());
}
```

Listing 1.21: How .co files compile

The compiler splits this into a single intermediate `.cu` file, then hands it to `c++`:
What happens to each region:

- **(1) `__device__` functions** are copied into the generated `.cu` file **verbatim**. They appear in the intermediate source exactly as you wrote them.
- **(2) `__co__` functions** are the heart of Croqtile.
- The compiler does not parse their contents — they must be valid at the splice point in the generated code.
- **(4) Host code** (`main()` and any other non-annotated functions) is passed through as ordinary C++ and ends up in the host compilation path.

All four regions merge into one `.cu` file. Run `croqtile kernel.co -v` (Section 1.9) to see the exact `c++` command the compiler issues.


How .co files compile
kernel.co
generated intermediate
① __device__
② __co__ + ③ __cpp__
④ host C++
Croqtile
compiler
__device__ fast_rsqrt
copied verbatim
__global__ __choreo_kernel
generated from __co__
asm volatile("setmaxnreg")
spliced from __cpp__
int main() { ... }
host launch wrapper
nvcc
GPU binary
__device__ → GPU verbatim | __co__ → GPU transformed | host → CPU | __cpp__ → spliced

```
__device__ float fast_rsqrt(x) {
  return __frsqrt_rn(x);
}
__co__ void kernel(f32 [M,N] in) {
  parallel ... : block {
    __cpp__("asm volatile(...)");
    dma.copy ...; mma ...
  }
}
int main() {
  auto buf = make_spandata();
  kernel(buf.view());
}
```

A Full Croqtile-target C++ Code Example

Here is a complete example that adds two arrays of the same shape and element type. Its three parts are labelled in the listing: a device helper, the tileflow function and the host program that calls it:

```
// Device Program
__device__ void kernel(int * a, int * b, int * c, int n) {
  for (int i = 0; i < n; ++i) c[i] = a[i] + b[i];
}
// Tileflow Program
__co__ s32 [6, 17, 128] ele_add(s32 [6, 17, 128] lhs, s32 [6, 17, 128] rhs) {
  s32 [lhs.span] output; // Use same shape as lhs
  parallel p by 6 {
    with index in [17, 4] { // Tiling factors
      foreach index {
        lhs_load = dma.copy lhs.chunkat(p, index) => local;
        rhs_load = dma.copy rhs.chunkat(p, index) => local;
        local s32 [lhs_load.span] l1_out;
        // Call kernel with loaded data
        call kernel(lhs_load.data, rhs_load.data, l1_out, |lhs_load.span|);
        // Store result back to output
        dma.copy l1_out => output.chunkat(p, index);
      }
    }
  }
  return output;
}
// Host Program
int main() {
  // Define data arrays
  croqtile::s32 a[6][17][128] = {0};
  croqtile::s32 b[6][17][128] = {0};
  // Fill arrays with data
  std::fill_n(&a[0][0][0], sizeof(a) / sizeof(a[0][0][0]), 1);
  std::fill_n(&b[0][0][0], sizeof(b) / sizeof(b[0][0][0]), 2);
  // Call Croqtile function (data movement and device kernel execution)
  auto res = ele_add(croqtile::make_spanview<3>(&a[0][0][0], {6, 17, 128}),
                        croqtile::make_spanview<3>(&b[0][0][0], {6, 17, 128}));
  // Verification: check correctness of results
  for (size_t i = 0; i < res.shape()[0]; ++i)
    for (size_t j = 0; j < res.shape()[1]; ++j)
      for (size_t k = 0; k < res.shape()[2]; ++k)
        if (a[i][j][k] + b[i][j][k] != res[i][j][k]) {
          std::cerr << "result does not match.\n";
          abort();
        }
  std::cout << "Test Passed\n" << std::endl;
}
```

Listing 1.22: A Full Croqtile-target C Code Example

The subsequent sections will explain the different parts of the code.

Host Program - the Control Center

The host program is where the data is created and where the result is checked. It is shown again here with the surrounding calls so that the boundary between ordinary C++ and the tileflow function is visible:

```
int main() {
  // Define data arrays
  croqtile::s32 a[6][17][128] = {0};
  croqtile::s32 b[6][17][128] = {0};
  // Fill arrays with data
  std::fill_n(&a[0][0][0], sizeof(a) / sizeof(a[0][0][0]), 1);
  std::fill_n(&b[0][0][0], sizeof(b) / sizeof(b[0][0][0]), 2);
  // Call Croqtile function (data movement and device kernel execution)
  auto res = ele_add(croqtile::make_spanview<3>(&a[0][0][0], {6, 17, 128}),
                        croqtile::make_spanview<3>(&b[0][0][0], {6, 17, 128}));
  // Verification: check correctness of results
  for (size_t i = 0; i < res.shape()[0]; ++i)
    for (size_t j = 0; j < res.shape()[1]; ++j)
      for (size_t k = 0; k < res.shape()[2]; ++k)
        if (a[i][j][k] + b[i][j][k] != res[i][j][k]) {
          std::cerr << "result does not match.\n";
          abort();
        }
  std::cout << "Test Passed\n" << std::endl;
}
```

Listing 1.23: Host Program - the Control Center

The `main` function is written with standard C++, except for the use of Croqtile APIs. In this program, you first define two arrays, `a` and `b`, and fill them with different values. The API `croqtile::make_spanview` is then used to attach shape information to the data.

The `croqtile::make_spanview` is declared as follows:

```
template <size_t Rank, typename T>
spanned_view<T, Rank> make_spanview(T* ptr, std::initializer_list<size_t> init);
```

Listing 1.24: Host Program - the Control Center

Here is the usage for reference:

```
croqtile::make_spanview<3>(&a[0][0][0], {6, 17, 128});
```

Listing 1.25: Host Program - the Control Center

This API is essential for connecting *host code* to the *croqtile functions*. Essentially, any input buffer (named the `spanned` data) of croqtile function is always associated with its shape. It enables Croqtile to guarantee shape safety at compile and runtime.

Note

The most significant dimension value comes first in the `initializer_list` shape. Thus, a shape of `{6, 17, 128}` corresponds to a C multi-dimensional array like `a[6][17][128]`.

In the example code, croqtile function `ele_add` is called to calculate the element-wise sum in parallel. Afterward, the host code verifies the result buffer `res`.

One important detail is that the output of the croqtile function is of type `croqtile::spanned_data`. Unlike `croqtile::spanned_view`, which does not own the buffer memory of the data it points to, `croqtile::spanned_data` owns the buffer. This ensures that the subsequent data verification process is applied to valid memory. The `croqtile::spanned_view` provides rich APIs, supporting C-style array indexing and shape queries via the member function `.shape()`.

Similarly, the most significant dimension is listed first in this shape array (`res.shape()[0]` in this case). Croqtile follows a '**most-significant-dimension-major**' ordering, also known as '**row-major**' ordering, where the first dimension varies the slowest.

1.9 Inspecting and Debugging a Build

When a kernel compiles but the answer is wrong, the fastest path is to narrow down which layer went wrong: the commands the compiler ran, the runtime checks it can insert, the layout of the data, or the types a debugger sees. The tools below are ordered that way, from cheap checks to expensive ones.

Debugging tip

Before reaching for the compiler's inspection flags, look at the program itself. If something looks wrong after editing a pipeline, verify **event order and trip counts** before chasing MMA layout bugs: producer and consumer must use the same `cdiv(K, TILE_K)` loop bound, and too few stages shifts pressure to `wait full` when the consumer outruns the producer.

Verbose mode: `-v`

Pass `-v` (or `-verbose`) to the compiler to see the external commands it invokes — which preprocessor, which code generator, which `c++` invocation:

```
croqtile kernel.co -v -o kernel
```

Listing 1.26: Verbose mode: -v

This is useful when you suspect the compiler is passing wrong flags to `c++` or when you need to see the exact generated file paths for manual inspection.

Runtime checks: `-rtc`

The compiler supports graduated runtime checking with `-rtc` (or `--runtime-check`):

```
croqtile kernel.co -rtc=high -o kernel
```

Listing 1.27: Runtime checks: -rtc

Levels: `none`, `entry` (default), `low`, `medium`, `high`, `all`. Each level is a superset of the previous. Use `high` or `all` during development, `none` for production.

Debugging MMA-heavy kernels

When the wrong answer comes from a tensor-core path, suspect **layout first** — row versus column major, and whether the RHS is `[N, K]` versus `[K, N]` in memory. Then check **indexing** (which `block_m`, `block_n`, and K slice you attached with `.at` / `chunkat`).

A common mistake is mislabeling `mma.row.row` when the staged data is actually column-major, or using `chunkat` indices that do not align with the MMA tile geometry. If you use swizzled loads (`tma.copy.swiz` / `mma.load.swiz`), a mismatch shows up as a regular pattern in the error (e.g., every sixteenth element correct, the rest shifted).

Debug RTTI and `debugger`

When `print` / `println` is not enough, Croqtile supports debug RTTI (Runtime Type Information) that makes its types visible to `debugger`.

Compile with `-g` to enable debug symbols:

```
croqtile kernel.co -g -o kernel_debug
```

Listing 1.28: Debug RTTI and debugger

The generated code includes RTTI structs for Croqtile types:

Croqtile type	**GDB type**	**Fields**
Shaped tensor <br/> (`s32 [M, N]`)	`croq::rtti::spanned<int, 2>`	`.span.data[]` (dimensions), <br/>`.stride.data[]` (strides), <br/>`.data` (pointer)
Index tuple	`croq::rtti::bounded_ituple<N>`	`.data[]` (values), <br/>`.ub[]` (upper bounds)
Integer tuple	`croq::rtti::ituple<N>`	`.data[]` (values)
Multi-dim span	`croq::rtti::mdspan<N>`	`.data[]` (extents)

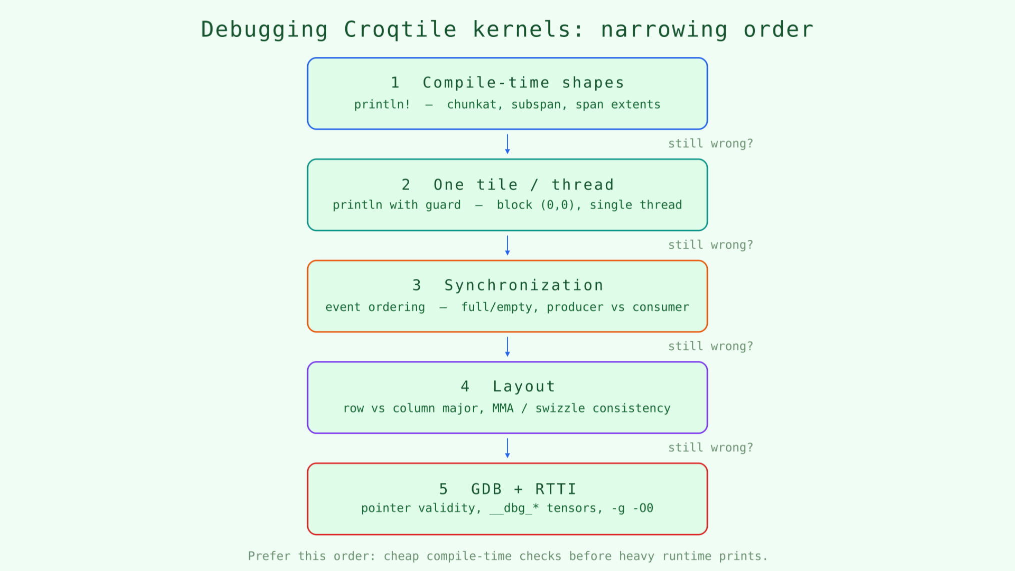
```
debugger -q ./kernel_debug
(gdb) break my_kernel
(gdb) run
(gdb) ptype __dbg_lhs
type = struct croq::rtti::spanned<int, 2>
(gdb) print __dbg_lhs.span.data[0]
$1 = 32
(gdb) print __dbg_lhs.span.data[1]
$2 = 64
(gdb) print __dbg_lhs.data != 0
```

```
$3 = true
```

Listing 1.29: Debug RTTI and debugger

The `__dbg_` prefix on variable names is generated by the compiler — it makes Croqtile variables visible alongside the generated C++ intermediates.

Putting it together


Debugging Croqtile kernels: narrowing order
1 Compile-time shapes
println! – chunkat, subspan, span extents
still wrong?
2 One tile / thread
println with guard – block (0,0), single thread
still wrong?
3 Synchronization
event ordering – full/empty, producer vs consumer
still wrong?
4 Layout
row vs column major, MMA / swizzle consistency
still wrong?
5 GDB + RTTI
pointer validity, __dbg_* tensors, -g -O0
Prefer this order: cheap compile-time checks before heavy runtime prints.

Work from cheap to expensive: `println!` (compile-time shapes) → guarded `println` on one tile (runtime values) → event/stage prints (synchronization) → pattern analysis (layout) → `debugger` (pointer bugs). Guard runtime prints behind `#ifdef` so they vanish in production:

```
#ifdef DEBUG_PRINT
  println("tile_k=", iv_k, " accum=", mc);
#endif
```

Listing 1.30: Guarding debug prints

```
croqtile kernel.co -DDEBUG_PRINT -o kernel    # debug build
croqtile kernel.co -o kernel                   # production build
```

Listing 1.31: Debug and production builds

Tool	**Level**	**Cost**
`print!` / `println!`	Compile-time	Free
`assert(expr, "msg")`	Runtime	Low — aborts on violation
`print` / `println`	Runtime	Medium — serialized `printf`
`-rtc=high`	Runtime	Medium — bounds checks
`-v` / `-verbose`	Compiler	Free — shows subprocess invocations
`-g` + `debugger` + RTTI	Runtime	High — debug symbols, no optimization

In short The tools above answer three questions in order: what the compiler believed at compile time, what the program did at runtime, and which flag exposes each. Start with `print!` when a shape or a type is surprising, move to `assert` when a value is surprising, and only then turn on a verbose build.

The next step is to open a benchmark kernel from the `croqtile/benchmark/` directory, map its regions to the chapters here, change one constant, rebuild, and measure.

1.10 Reading a Production .co File

An unfamiliar `.co` file is easier to read once you know the order its parts appear in: guards and helpers, host setup, the `__co__` function, and the injected C++. The chapters that follow define each of those constructs.

A benchmark kernel looks dense until you know the order in which its parts appear. Read it top down. The file opens with macros and `#error` guards that fix the conditions the rest of it is compiled under. Below them sit `__device__` functions — helper algorithms such as top-K, reductions and shuffle wrappers that Croqtile passes through unchanged. Then comes the host setup, ordinary C++ that creates buffers, chooses the launch configuration and takes timings. The centre of the file is the `__co__` function, where the orchestration lives: `parallel`, `foreach`, TMA/MMA, `inthreads.async` and events, each region mapping back to one of the chapters you have just read. Last are the `__cpp__` islands, usually only a handful of lines. Most production files follow this order, so the same five checks work every time.

Where to go next

With a program running, continue to Chapter 2 to learn what each construct in it means. If you already know the syntax, Chapter 3 is arranged for lookup.

Chapter 2

Language Basics

Chapter roadmap

This chapter introduces the core Croqtile DSL you use to write programs: how a `__co__` function is declared, how shaped data is described, how iteration and tile selection are expressed, how tiles move, and how concurrent regions coordinate. It stays with short, readable programs and asks only what each construct means.

2.1 The Two Parts of a Croqtile Program

A `.co` file reads like one C++ program, but the compiler does not treat every function in it the same way. The `__co__` prefix marks the functions Croqtile rewrites; the rest reaches the downstream compiler as ordinary C++. That split is the first thing to look for when reading an unfamiliar kernel.

At its simplest a program has two parts. A `__co__` function describes computation on shaped tensors, and the compiler translates that function into target code. The host program is standard C++ that prepares the data, calls the function and checks the result. Both live in a single `.co` file, and the `__co__` prefix is all the compiler needs to tell them apart.

Host, Device and Tileflow

Both of those parts are present here, but the ordinary C++ side is worth splitting once more: a `.co` file may also carry a `__device__` helper. The listing is written from the inside out — a device helper, the `__co__` function that calls it, and the host `main()` that calls the `__co__` function.

```
__device__ void device_function(...) {
  // High-performance device kernel implementation
}
// Tileflow program: orchestrating data movement
__co__ void croqtile_function(...) {
  // ... croqtile code ...
  device_function(...);
  // ...
}
// Host program: typically runs on CPU
void main() {
  // ... prepare data ...
  croqtile_function(...);
  // ...
}
```

Listing 2.1: The three regions of one source file

Execution starts at the bottom of the listing, in the host program. Written in standard C++ and run on the CPU, it is the entry point of the module and the caller of the Tileflow program — the `__co__` functions — and it manages the overall workflow of the heterogeneous application. A typical host body prepares the input data, invokes a Croqtile function, handles its return value and advances to the next step.

The Tileflow program called from there is the part Croqtile adds. Built from `__co__` functions, it is the core of a `.co` program: it orchestrates data movement between host and device and between storage levels within a single device. A typical Tileflow body moves data into a buffer at the appropriate storage level, calls a device function to compute on it, and moves the result back to the host. It is the only region the compiler translates.

Device functions sit at the other end of that call chain. A `__device__` function defines computation-intensive operations that run on the target device; the prefix marks it as device code. Like the host program, it passes through Croqtile compilation unchanged. The compiler leaves both sides of ordinary C++ alone; only the `__co__` function between them is rewritten.

2.2 Reading a Kernel Line by Line

An element-wise kernel is four lines long: a signature, an output allocation, one parallel statement and a return. Reading them one at a time shows why the shape in the signature matters — it is what lets the compiler check every indexing operation that follows.

The Croqtile Function

Start with the signature, which declares more than a C++ function prototype would: every parameter and the result carry a shape.

```
__co__ s32 [4, 8] ele_add(s32 [4, 8] lhs, s32 [4, 8] rhs) {
```

Listing 2.2: A kernel signature

The `__co__` prefix marks this as a Croqtile function. Unlike regular C++ functions, the signature carries full shape information: `s32 [4, 8]` means "a 2D tensor of shape 4 × 8 with element type `s32` (signed 32-bit integer)". Both parameters and the return type follow this convention — `type [shape] name`. The compiler uses these shapes to verify that every indexing operation is in-bounds at compile time.

Croqtile supports these element types: `s32` (signed 32-bit int), `f16` (half-precision float), `bf16` (brain float), `f32` (single-precision float), and `f8_e4m3` / `f8_e5m2` (8-bit floats). In the examples that follow, `s32` is the simplest to reason about.

```
s32 [lhs.span] output;
```

Listing 2.3: Allocating the output

This allocates the output tensor. The expression `lhs.span` copies the full shape from `lhs`, so `output` automatically has shape `[4, 8]`. If you later change the input shape, the output adjusts — a pattern that makes Croqtile functions easier to generalize.

```
parallel {i, j} by [4, 8]
  output.at(i, j) = lhs.at(i, j) + rhs.at(i, j);
```

Listing 2.4: The parallel body

`parallel {i, j} by [4, 8]` creates 4 × 8 = 32 parallel instances, one per element position. Each instance runs the body with its own `(i, j)` pair. The `.at(i, j)` accessor indexes into a tensor at position `(i, j)`, and the addition is direct — one element, one thread.

What `parallel` leaves unsaid is how those instances map onto the machine: without an annotation the compiler decides. That mapping is deliberately absent from the language here — the statement describes independent work, not the resources that run it.

```
return output;
```

Listing 2.5: Returning the result

A `__co__` function returns its result tensor. The return type in the signature (`s32 [4, 8]`) must match what you actually return. The caller on the host side receives a `croq::spanned_data` — an owning buffer with shape metadata that you can index into with `[i][j]`.

`__co__ auto` return type

`__co__ auto matmul(...)` lets the compiler infer the return type from `return output`.

Using `s32` with scalar accumulation is a simplified style that isolates the `swap` mechanism (Section 3.6.2); the same return-type deduction applies to the FP16/MMA kernels in Section 3.7.

2.3 The Host-Side API: Inputs and Outputs

The host creates shaped buffers, passes them to a `__co__` function and receives a result back. Each step uses a small API, and the shape attached to a buffer is what makes the call type-safe.

The Host Program

The host side is plain C++ with two Croqtile API calls: one creates a shaped buffer from raw storage, the other hands a view of it to the kernel.

```
auto lhs = croq::make_spandata<croq::s32>(4, 8);
auto rhs = croq::make_spandata<croq::s32>(4, 8);
lhs.fill_random(-10, 10);
rhs.fill_random(-10, 10);
```

Listing 2.6: Creating the shaped inputs

`croq::make_spandata<T>(dims...)` creates an owning tensor buffer on the host. You pass the element type as a template parameter and the dimensions as arguments. `fill_random` populates it with values in the given range.

```
auto res = ele_add(lhs.view(), rhs.view());
```

Listing 2.7: Calling the kernel

Calling a `__co__` function from C++ looks like a normal function call. The `.view()` method produces a non-owning `spanned_view` from an owning `spanned_data` — this is how you pass host tensors into a Croqtile function without transferring ownership. The return value `res` is a `croq::spanned_data` that owns its buffer.

The rest of `main` is ordinary verification: loop over every element and assert equality. The host program is deliberately boring — the interesting work happens in the `__co__` function.

Host Code

The host side uses the same `make_spandata` / `.view()` pattern used by the host code above:

```cpp
int main() {
  auto lhs = croq::make_spandata<croq::s32>(128, 256);
  auto rhs = croq::make_spandata<croq::s32>(256, 256);
  lhs.fill_random(-10, 10);
  rhs.fill_random(-10, 10);
  auto res = matmul(lhs.view(), rhs.view());
  for (size_t i = 0; i < res.shape()[0]; ++i)
    for (size_t j = 0; j < res.shape()[1]; ++j) {
      int ref = 0;
      for (size_t k = 0; k < lhs.shape()[1]; ++k)
        ref += lhs[i][k] * rhs[k][j];
      croq::croq_assert(ref == res[i][j], "values are not equal.");
    }
  std::cout << "Test Passed\n" << std::endl;
}
```

Listing 2.8: Host Code

The loop after the call checks every element against a plain C++ reference computation, so an incorrect result is reported before the program prints anything.

Parameters Passing between Croqtile and C++ Function

Passing arguments between Croqtile and C++ requires the `croqtile.h` header. Normally, programmers combine the raw C++ pointer and associated dimensions info to construct Croqtile spanned data. The code below demonstrates how it works.

```cpp
#include "croqtile.h"
void bar(const float* data, unsigned size) {}
__co__ void foo(f32 mdspan<2> d) {
  parallel p by 6 {
    call bar(d, |d|);     // Call the C++ function
  }
}
void foobar(float* a) {
  foo(croqtile::make_spanview<2>(a, {1, 2}));
}
```

Listing 2.9: Parameters Passing between Croqtile and C Function

In the example, you make use of croqtile utility function (template) ’make_spanview’ to wrap the data, which does not invoke copy. And in Croqtile function ’foo’, the parameter ’f32 mdspan<2>’ is the corresponding entity. Because the data is implicitly cast, you can simply use ’d’ as the first parameter of C++ function ’bar’. And operation ’|d|’ obtains the total size of the spanned data ’d’. These are used for calling C++ function ’bar’.

Similarly, *Scalar Type* data can also be passed from/to Croqtile function. Nevertheless, ituple is only used inside Croqtile function.

Input Parameters

A Croqtile function accepts only two kinds of input parameter, the *spanned data* and the *integer*.

Similar to any C++ program, the parameter may either be named or unnamed:

```
__co__ void foo(f32 [7] a, int) {...}
```

Listing 2.10: Input Parameters

In this example, the second parameter is unnamed, which makes it unable to be reference in the Croqtile function. However, this is still the valid code.

C++ Host: Data Argument with Shape

One obvious gap between C/C++ pointers/arrays and Croqtile *spanned data* is that *spanned data* is shaped. Therefore, to call a Croqtile function, it is necessary to convert pointers or arrays.

One direct method is to use the Croqtile API provided in `croqtile.h`. A program does not have to include `croqtile.h` explicitly in a Croqtile compilation environment; the compiler includes it for every Croqtile program.

```
__co__ void foo(f32 [7, 16] input, f32 [7, 16] output) {...}
void entry(float * input, float* output) {
  // foo(input, output); <-- result in error since missing shape
  foo(croqtile::make_spanned<2>(input, {7, 16}),
      croqtile::make_spanned<2>(output, {7, 16}));
}
```

Listing 2.11: Passing a shaped buffer

In the above example, you make use of `croqtile::make_spanned` function template, providing the rank as template parameter, and typed pointer together with shape list as the function parameters. In this way, a shape is attached with the pointer, which makes a valid Croqtile function parameter. As you may figure out from `croqtile.h`, it results in `croqtile::spanned_view` object, which simply wraps the C/C++ pointer/array and the array-based list. Its simplified definition is:

```
template <typename T, size_t Rank>
class spanned_view {
  T* ptr;                 // typed pointer to the data
  size_t dims[Rank];      // ranked dimensions
};
```

Listing 2.12: The host-side buffer wrapper

Dimensions do not have to be constants; runtime values work as well:

```
__co__ void foo(f32 [7, 16] input) {...}
void entry(float * input, float* output, int M, int N) {
  foo(croqtile::make_spanned<2>(input, {M, N})),
}
```

Listing 2.13: A call built from runtime dimensions

In this example, the `input` shape is runtime variable. But do not worry if wrong parameter is provided. Croqtile will check if `M == 7` and `N == 16` when entering into the tileflow program. **If not, the program will terminate immediately** since all the assumptions about shapes in the Croqtile function does not hold.

One important notice for the *spanned data* inputs is that they are **references to the input data/buffers**. That implies no data copy happens at invoking Croqtile function, though somebody may think from the language perspective it look like copy-semantics.

Return Type and Type Deduction

Similar to the input, *spanned data* and *integer* are valid output types. The *void* type is also allowed if nothing is returned.

In addition to the explicit return type annotation, Croqtile support type inference on returns. For example:

```
__co__ auto foo() {
  f32 [7, 16] result;
  // ...
  return result;
}
```

Listing 2.14: Return Type and Type Deduction

In this code, the return type is annotated as `auto`, which is the same keyword as C++. Croqtile is capable of inferring return type as `f32 [7, 16]`. Therefore, it is recommended to use `auto` whenever possible.

C++ Host: Shape Return

The C++ host code that receives the output of Croqtile function actually receives a buffer. The return has value semantics: the caller receives the buffer itself, not a reference to an object allocated inside the called function. For example:

```
__co__ auto foo() {
  f32 [7, 16] output;
  // ...
  return output;  // semantically copy
}
void entry() {
  auto result = foo(); // move the 'spanned_data' to caller
  // ...
}
```

Listing 2.15: Returning a shaped buffer

In this example, the `output` is semantically copied to the caller. However, in Croqtile's implementation, no copy happens. The returned type is `croqtile::spanned_data`, where its simplified definition is as following:

```
template <typename T, size_t Rank>
class spanned_data {
  std::unique_ptr<T[]> ptr; // move only pointers
  size_t dims[Rank];        // ranked dimensions
};
```

Listing 2.16: The host-side result wrapper

In contrast to `croqtile::spanned_view`, `croqtile::spanned_data` owns the buffer it points to. Thus there is **no copy actually happens when returning from a Croqtile function**. In compiler terminology, this is called the *Return Value Optimization*.

You may use member function like `.rank()`, `.shape()` to query shape related information of Croqtile return. It lets the host work with the result without knowing the details of the Croqtile function that produced it.

2.4 Shapes and mdspan

A shape is a value a program can name and compute with, not just an annotation on a declaration. `mdspan` *defines one, arithmetic derives another from it, and an anonymous dimension leaves a size for the caller to supply.*

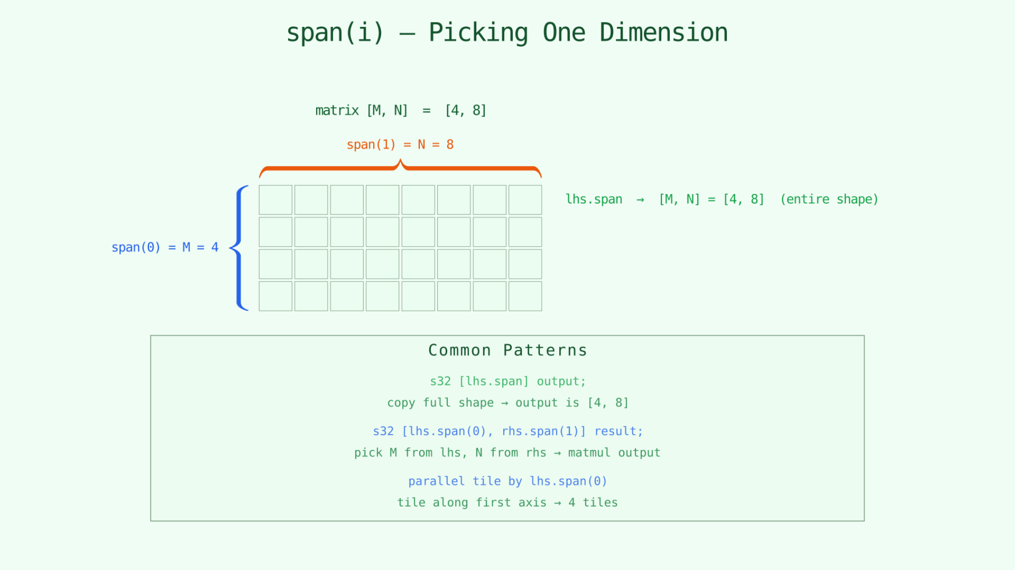
`span` and `span(i)`

`tensor.span` gives the full shape of a tensor. `tensor.span(i)` gives one dimension.

```
s32 [lhs.span(0), rhs.span(1)] output;
```

Listing 2.17: span and span(i)

Deriving one shape from another comes up constantly in data movement code, because output shapes often come from different inputs, and moved tiles frequently need to preserve or recompute shape information.


span(i) — Picking One Dimension
matrix [M, N] = [4, 8]
span(1) = N = 8
lhs.span → [M, N] = [4, 8] (entire shape)
span(0) = M = 4
Common Patterns

```
s32 [lhs.span] output;
copy full shape → output is [4, 8]

s32 [lhs.span(0), rhs.span(1)] result;
pick M from lhs, N from rhs → matmul output

parallel tile by lhs.span(0)
tile along first axis → 4 tiles
```

Use `span` *for the whole shape,* `span(i)` *for a single axis.*

A shape in Croqtile is a value a program can name and compute with, not just an annotation on a declaration. The examples below define one, derive another from it, and leave a dimension open for the caller to fill in.

Defining Shapes with `mdspan`

In Croqtile, shapes can be defined with the `mdspan` keyword, which stands for **Multi-Dimensional Span**. This keyword represents multi-dimensional data for computations.

You can explicitly define `mdspan` variables as follows:

```
mdspan s0 : [7, 8]; // Defines a 2D shape with dimensions [7, 8]
mdspan<1> s1 : [3]; // Defines a 1D shape with dimensions [3]
```

Listing 2.18: Declaring a shape

In this example, the leading keyword `mdspan` indicates the declaration of a *mdspan* variable, followed by the user-provided variable name. Each *mdspan* variable is initialized with an **initialization expression**, which consists of comma-separated integer values enclosed by `[]`. The symbol `:`, which immediately follows the variable name, introduces the *initialization expression*. Therefore, `s0` is defined as a 2D shape with `7` rows and `8` columns, while `s1` is a 1D shape with a dimension of `3`.

It is possible to optionally specify the **rank** of an *mdspan* variable by placing `<>` after the `mdspan` keyword. If the rank is explicitly specified, it informs the Croqtile compiler to check for rank consistency. If the rank value differs from the corresponding *initialization expression*, it triggers a failure at compilation. Here is an example:

```
mdspan<3> s2 : [64, 32]; // error: the rank of mdspan is inconsistent
```

Listing 2.19: A rank mismatch

In addition to explicit mdspan declarations, the Croqtile compiler can infer the type of `mdspan` variable from its *initialization expression*, eliminating the need for the explicit `mdspan` keyword:

```
s3 : [7, 8, 9]
```

Listing 2.20: Omitting the mdspan keyword

In this code, `s3` is an `mdspan` of rank `3` with dimensions `7, 8, 9`. Since Croqtile **requires mdspan to always be initialized within declarations**, the type inference version is preferred in programming practice.

Deriving *mdspan*s

In practical coding, it is common to derive a new shape from an existing one. For example, you might want to perform data **tiling** or **blocking**, which requires dividing the dimensions of a shape. Alternatively, you might want to **pad** specific dimensions, which involves adding delta to the shape dimensions.

In Croqtile, such shape derivations can be easily accomplished using arithmetic operations on *mdspan*. The following code showcases an example:

```
shape : [128, 64]; // initial shape
new-shape0 : shape [(0) / 2, (1) / 4, 1];  // tile and reshape: [1, 64, 16]
new-shape1 : shape [(1) + 2, (0) / 16];    // pad and reshape: [66, 8]
```

Listing 2.21: The original shape

In this example, the `new-shape0` is derived from `shape`, with dimension 0 divided by `2`, dimension 1 divided by `4`. This corresponds to *tiling* operation in high-level semantics.

Additionally, the code adds a new dimension to `new-shape`. In high-level semantics, this operation is often referred to as *reshaping*.

In Croqtile, the definition of `new-shape0` is equivalent to:

```
new-shape0: [shape(0) / 2, shape(1) / 4, 1];
```

Listing 2.22: A derived shape

Here, the initial `shape` is explicitly listed element-wise rather than specified outside `[]`. But similar to the prior version, The **element-of** operation, which is annotated as `()`, is used on top of existing shape to retrieve dimension values. Obviously, this approach requires more code but yields the same result. Thus, the prior version can be considered *syntactic sugar* for the complete *initialization expression* of the new shape.

In the code example, `new-shape1` is also derived from `shape`, it pads dimension 1 by `2` and swaps the dimensions in the derived shape.

Furthermore, you may use the `mdspan` as a whole for derivations:

```
shape : [32, 72]
new-shape0 : shape;  // [32, 72]
new-shape1 : shape + 1;  // [33, 73]
new-shape2 : shape / 4;  // [8, 18]
new-shape3 : [shape, 6]; // [32, 72, 6]
```

Listing 2.23: Rebinding a shape

Arithmetic operations on an *mdspan* variable are applied dimensionally. Thus, the statement
`new-shape1 : shape + 1;`
is equivalent to
`new-shape1 : shape [(0) + 1, (1) + 1];`

The derived definition of `new-shape3` demonstrates that using an *mdspan* variable in an *mdspan initialization expression* results in **concatenation** behavior. Thus, the declaration
`new-shape3 : [shape, 6];`
is equivalent to
`new-shape3 : shape [(0), (1), 6];`

The prior version can be as well deemed as a *syntactical sugar* of a complete definition.

mdspan with Anonymous Dimension

Croqtile supports shape dimensions with unknown values, similar to many high-level machine learning languages. The simplest method is to use **Anonymous Dimension** in `mdspan`:

```
__co__ auto foo(s32 [?, 1, 2] input) { ... }
```

Listing 2.24: A kernel with an unknown dimension

Here, the question mark `?` represents a compile-time unknown value. Any positive integer could be assumed here. However, the value must be provided at runtime, as the following example shows:

```
void bar(int * data, int dim0) {
  foo(croqtile::make_spanview<3>(data, {dim0, 1, 2}));
}
```

Listing 2.25: The caller supplying that dimension

In this example, the first dimension of `input` is provided by `foo`'s *spanned* input, where the value is retrieved from `bar`'s caller at execution. You may assume the value be determined by reading a configuration file, through shape derivation in a machine learning framework, or etc..

The *anonymous dimension* can work properly for this case. However, in certain scenarios, it is not sufficient.

Step Further: Symbolic Dimension

Another approach in Croqtile to support dynamic shape is to use **Symbolic Dimension**. *Symbolic dimensions* are named but as well unknown at compile-time. Here's an example:

```
__co__ auto Matmul(s32 [M, K] lhs, s32 [K, N] rhs) {
  int tile_m = 32, tile_m = 8, tile_k = 16;
  s32 [M / tile_m, K / tile_k] tiled_lhs;
  s32 [K / tile_k, N / tile_n] tiled_rhs;
  // ...
}
```

Listing 2.26: Step Further: Symbolic Dimension

In the code, each dimension is given a symbolic name (`M, K, N` in this case) for the *spanned* inputs. Compared to *anonymous dimension*, *symbolic dimension* approach clearly describes the relationship between shapes. The code looks intuitive and is easy to maintain. Furthermore, it is possible to improve the code safety with the additional information provided. Let us dive deeper for this.

2.5 Integers, I-Tuples, and Bounded Values

Loop bounds and indices are compile-time values with types of their own. This section covers integers, I-tuples and bounded values.

Scalar Types

As described, Scalar types in Croqtile includes the *Integer Type* and *Boolean Type*. The *Integer Type* is similar to C++ type 'int' or 'int32_t'. It is an signed value which takes 32-bits, ranging from -2^{31} to $2^{31} - 1$. The below code illustrates its usage for defining the data and function declaration.

```
int a;
__co__ int foo(int b);
```

Listing 2.27: Scalar Types

Operations like integer arithmetics, shift and logical operation are all supported. And the syntax is identical to C++ builtin operations. In Croqtile, you neither provide equivalence of unsigned scalar integers, nor equivalence of 8-bits, 16-bits, 64-bits scalar integers. This design is simply motivated by the fact: these types are not essential for program control purposes. And normally a 32-bits signed integer can fulfill such work.

The *Boolean Type* in Croqtile is similar to C++ type 'bool'. The operations on top of Boolean, and conversion between Integers, are identical to these of C++.

I-Tuple Types

An integer tuple is an unordered set of integers. As described, it is normally used as a (subscription) index.

To define an i-tuple, simply enclose its elements within '{' and '}' braces. That is,

```
ituple index = {5, 4, 3, 2, 1};  // It defines a tuple of 5 elements
index = {a, b};  // 'a' and 'b' are existing integers
```

Listing 2.28: I-Tuple Types

Operations over *mdspan* and *i-tuple*

In Croqtile, you allow special operation over *i-tuple* and *mdspan*. Below is an example to apply a fixed tiling over an mdspan:

```
sp : [6, 8];
tiling_factor = {3, 2};
spn : sp / tiling_factor;   // spn is defined as [2, 4];
```

Listing 2.29: Operations over mdspan and i-tuple

In Croqtile, mdspan can be defined with such **Tuple-Span Operations**. The supported operations includes:

- *mdspan* / *i-tuple*
- *mdspan* + *i-tuple*
- *mdspan* % *i-tuple*
- *mdspan* ∗ *i-tuple*
- *mdspan* − *i-tuple*

Essentially, these operations can be achieved through mdspan *dimension-wise* definition. However, *tuple-span operations* aid programmers in writing more readable code. This is also the objective that Croqtile aims to achieve.

In this section, you will learn how to define *integer*s and integer-tuples (*i-tuple*s) in Croqtile code and understand their usage.

Integers for Program Control

In high-performance computation kernels, integers are typically used for program control rather than computation. In Croqtile *tileflow programs*, integers are used exclusively for loop control and array indexing. Given the limited value ranges in these scenarios, Croqtile has provided a single integer type — the *32-bit signed integer* — to simplify the design for this domain-specific use.

In Croqtile, an **integer**-typed variable is defined similar to C/C++:

```
int a = 1;
```

Listing 2.30: Integers for Program Control

However, since Croqtile can infer the type from the *initialization expression*, you can also define an integer variable in another style:

```
a = 1;
```

Listing 2.31: Integers for Program Control

Similar to C/C++, you can define multiple integers in a single line:

```
int a = 1, b = 2, c = 3;
d = 4, e = 5;
```

Listing 2.32: Integers for Program Control

In Croqtile, the *mdspan*-typed shape can not be used as the function parameter. However, integers can be passed from the host to the Tileflow program, and from the Tileflow program to the device:

```
__co__ void foo(int a) {
  // ...
  call kernel(a, 3);
}
```

Listing 2.33: Integers for Program Control

Unlike C/C++, Croqtile does not allow the declaration of integers without initialization or the reassignment of an integer. For example:

```
int a;  // error: declaration without initialization
int b = 1;
b = 3;  // error: re-assignment
```

Listing 2.34: Integers for Program Control

This enforces that **integers must be initialized at declaration and cannot be reassigned**. In some contexts, these integers are referred to as "**immutable**" or "**constant**".

Defining I-Tuples

In Croqtile, multiple integers can be grouped into an **integer-tuple**, or **i-tuple** (or **ituple**). The keyword `ituple` can be used to define an *i-tuple*. The below code showcases the usage:

```
ituple a = {1, 2, 3};
b = {4, 5, 6};  // utilize the type inference
```

Listing 2.35: Defining I-Tuples

Since Croqtile can infer types from the *initialization expression*, programmers can often omit the `ituple` keyword. However, without explicit type annotation, an *ituple* variable definition might look similar to an *mdspan* definition if you are not yet familiar with Croqtile. To distinguish them:

- The *initialization expression* of an *ituple* follows an assignment operation `=` (like integers), whereas an *mdspan* is initialized after `:`;
- The *initialization expression* of an *ituple* is enclosed by `{}`, not `[]` as for *mdspan*.

Similar to `mdspan`, you may enforce rank check for *ituples* at compile time:

```
ituple<3> a = {1, 2};  // error: inconsistent rank
```

Listing 2.36: Defining I-Tuples

Operations on I-Tuples

Operations on *i-tuples* are similar to those on *mdspan*. You can either use the *element-of* operation `()` to retrieve the element values or use *ituple* as a whole:

```
a = {3, 4};
b = {a(0), 1, a(1)};  // '()' to retrieve the element value
c = a {(0), (1), 2};  // syntax sugar
d = {a, 5, 6};        // concatenate
e = a + 1;            // addition is applied elementwise
```

Listing 2.37: Operations on I-Tuples

Note that, similar to *integer* and *mdspan*, **an *ituple* variable must be initialized and cannot be reassigned**.

In practice, *ituples* are often used with *mdspans*. For example:

```
shape : [7, 18, 28];
tiling_factors = {1, 2, 4};
tiled_shape : shape / tiling_factors;
padded_shape : shape + {2, 0, 2};
```

Listing 2.38: Operations on I-Tuples

In this code, a mdspan-typed shape is divided by an ituple-typed tiling factor to derive a `tiled_shape`. Additionally, a `padded_shape` is derived from the addition of the initial `shape` and an anonymous *ituple*. Note that when a *mdspan* is added by an *ituple*, their rank must be consistent. Or else, an error will occur.

```
shape : [7, 8, 9] + {1, 2}; // error: inconsistent rank
```

Listing 2.39: Operations on I-Tuples

In short In this section, you learned how to define *integers* and *i-tuples* in Croqtile. The *Integers* follow C/C++ syntax. *i-tuples* follow a syntax similar to *mdspan*, but they can operate on *mdspan* to derive new values.

Both types **require an __initialization expression__ and cannot be reassigned**, making them appear as **constant** or **immutable** values. In fact, all variables defined in Croqtile tileflow programs are immutable, except for those of specific types. Therefore, **the assignment operator = is used for variable initialization in most cases**, which programs must be aware of.

2.6 Spanned Data and Storage Specifiers

Data in Croqtile is spanned*: a buffer together with its shape. This section shows how spanned data is declared, stored and initialised.*

Fully-Typing

A *mdspan* can not be applied alone to define the data for computation. In Croqtile function, a data definition must be fully-typed, which consists of a fundamental type and a *mdspan*. The below code showcases how it works.

```
ndims : [20, 15];
f32 [10, 10] d0;
f16 [ndims] d1;
```

Listing 2.40: Fully-Typing

There are fundamental types that Croqtile has supported, including:

- Unsigned Integers: *u8/u16/u32*
- Signed Integers: *s8/s16/s32*
- Floating-points: *f16/bf16/f32*

Note 's32' and 'int' are different in Croqtile. 's32' is a fundamental type, which can not be applied for a fully typing.

The Storage Qualifier

A spanned-typed data in Croqtile is usually large.

In croqtile, the language defines three storage qualifiers to annotate the data being defined, including:

- **global**,
- **shared**,
- and **local**.

The below code showcases the usage.

```
ndims : [20, 15];
local f32 [10, 10] d0;
shared f16 [ndims] d1;
```

Listing 2.41: The Storage Qualifier

By default, when no storage qualifier appears, the data defined is considered as from the *global* storage.

Croqtile *tileflow program* describes how to move data around. Therefore, declaring or defining the data or buffer are fundamental. In this section, you will learn related syntax.

Defining a *Spanned Data*

Defining a *spanned data* requires both the element type and its shape. For example,

```
f32 [32, 16] data;
```

Listing 2.42: Defining a Spanned Data

This defines a buffer with a shape of `[32, 16]`, where the element type is `f32`, which is the *IEEE-754* single-precision float. It is also possible to use a named `mdspan` to define the buffer:

```
shape : [72, 14];
s32 [shape] data0;              // using the named mdspan ‘shape‘
u32 [shape(0), shape(1)] data1; // same shape as the above
u16 [shape + 6] data2;          // shaped as [78, 20]
s16 [shape, 8] data3;           // shaped as [72, 14, 8]
```

Listing 2.43: Defining a Spanned Data

Here the previous example used the named *mdspan* `shape` to define a 2D buffer named `data0`, where the elements are 32-bit integers. Note that the named *mdspan* `shape` is placed inside `[]`. Additionally, it is allowed to use *element-of*, arithmetics, or concatenation on *mdspans* to specify the buffer's shape.

In Croqtile terminology, the element type is referred to as the **fundamental type**. Croqtile supports multiple fundamental types, including:

- **Unsigned Integers**: `u8`, `u16`, `u32`
- **Signed Integers**: `s8`, `s16`, `s32`
- **Floating Points**: `f16`, `bf16`, `f32`

The prefixes `u` and `s` stand for "unsigned" and "signed", respectively, while the number suffix indicates the bit width of the type. `f16` refers to the *IEEE-754* standard *half-float*, and `bf16` refers to the 16-bit *binary float*, commonly used in machine learning scenario.

Note `s32` is different from `int`. Although they occupy the same amount of storage, in Croqtile, `s32` can not be used alone to define programming entities. For example:

```
s32 a;  // error: the fundamental type can not be used for variable definition
    alone
```

Listing 2.44: Defining a Spanned Data

This results in a compile-time failure. Consequently, `s32` cannot be used for program control like `int`, and type conversion between them is not possible.

In practical code, some buffer declarations must specify a **storage specifier**, like so:

```
global f32 [32, 7, 2] a;
shared u8 [512, 144] b;
local u8 [72, 1024] c;
```

Listing 2.45: Storage Specifier

Since Croqtile handles storage in a heterogenous context, a buffer definition without a storage specifier defaults to the storage type of the host program, i.e., CPU memory. Other storage specifiers are defined by the target.

- *global*: Refers to the device's global storage.
- *shared*: Refers to the device block's shared storage
- *local*: Refers to the thread-private storage.

Other targets may have different definitions. Furthermore, buffers with different storage types have limitations in their declarations. You will explore this in later chapters.

Initialization

It is possible to initialize a buffer at the declaration site. For example:

```
local s32 [17, 6] b1 {0};        // elements are initialized to 0
shared f32 [128, 16] b2 {3.14f}; // elements are initialized to 3.14f
```

Listing 2.46: Initialization

The syntax for the *initialization expression* of a *spanned data* is straightforward: it encloses an initial value inside brackets following the variable name. However, its functionality is limited — it always set all elements to a fixed value.

Declaring the Parameters and Return Values

A *spanned data* is passed between host and tileflow programs. Therefore, a croqtile function can have *spanned data*s as its parameters. The following code showcases an example:

```
__co__ f16 [7, 8] foo(f32 [16, 17, 5] input) {...}
```

Listing 2.47: Declaring the Parameters and Return Values

The syntax is similar to variable definitions, except that operations on an existing *mdspan* is not possible. Additionally, no storage specifier is allowed or initialization is allowed.

One useful built-in member function for the *spanned* parameters is `.span`, which provides the associated *mdspan* of the *spanned data*. The following code demonstrates how to use it for a buffer declaration:

```
__co__ auto foo(f32 [16, 17, 5] input) {
  f32 [input.span / {4, 1, 5} ] buffer;  // declare a buffer with tiled shape
  // ...
}
```

Listing 2.48: Declaring the Parameters and Return Values

In short This section covered defining and managing data/buffers in Croqtile tileflow programs, including declaration syntax, initialization, storage specifiers, and efficient buffer lifetime management by the compiler. This enables us to step further to the next essential topic: the dynamic shape support.

1. `global`

- The default memory space for data in Croqtile, if no storage qualifier is specified.

```
global f32 [100, 200] matrix;
```

Listing 2.49: 1. global

2. `shared`

Example:

```
shared f32 [10, 10] tile_data;
```

Listing 2.50: 2. shared

3. `local`

It is particularly useful when each thread requires a small amount of data, such as scalar values or small arrays that do not need to be shared with other threads. Example:

```
local f16 [5] thread_local_data;
```

Listing 2.51: 3. local

Default Behavior and Usage

This default behavior simplifies memory management for users who do not need to explicitly define where their data should be stored, but it also means that accessing such data could incur higher latency, especially when working with large arrays or parallel processing units.

```
f32 [100, 100] d0;
```

Listing 2.52: Default Behavior and Usage

If you want to place data in a different memory space (shared or local), you need to specify the appropriate storage qualifier.

```
local f32 [10, 10] d0;     // Small data, local to the thread
shared f16 [ndims] d1;     // Intermediate data, shared between threads in a
    block
```

Listing 2.53: Default Behavior and Usage

Example Code: Using Storage Qualifiers

To better understand how storage qualifiers work, consider the following code example:

```
ndims : [20, 15];
local f32 [10, 10] d0;
shared f16 [ndims] d1;
global f32 [100, 200] matrix;
// Perform computations on these arrays, utilizing the appropriate storage space
dma.copy d0 => d1;
dma.copy d1 => matrix;
```

Listing 2.54: Example Code: Using Storage Qualifiers

In this code:

2.7 Iteration: parallel-by, with-in, and foreach

Iteration in Croqtile declares independence rather than a schedule. This section introduces `parallel`, `with-in` and `foreach` through small programs.

Croqtile's Two-Layer Parallelism Model

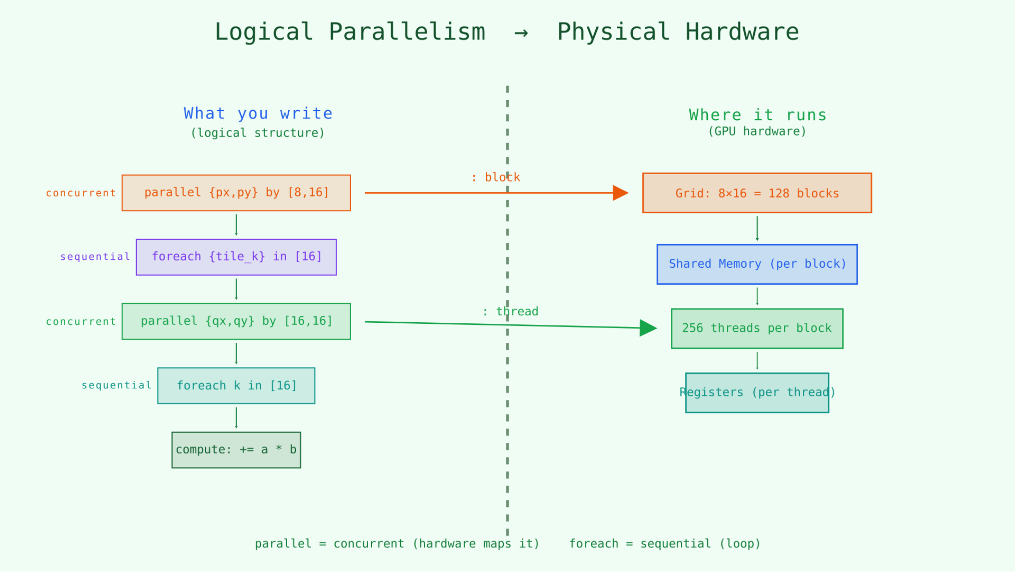
Croqtile gives you two separate knobs:

1. `parallel` — declares that iterations are independent and *may* run concurrently. This is a logical statement.
2. **Space specifiers** (`: block`, `: thread`, `: group`, `: group-4`) — tell the compiler each parallel axis maps to. This is a physical statement.

And there is `foreach`, which is the opposite of `parallel`: iterations run **sequentially**, in order. Use `foreach` when later iterations depend on earlier ones — an accumulation loop, a K-dimension reduction, a pipeline that loads tile k before computing with it.

```
parallel {px, py} by [8, 16] : block
  foreach {tile_k} in [16]                 // sequential -- K-dimension loop
    parallel {qx, qy} by [16, 16] : thread  // concurrent -- mapped to threads
      foreach k in [16]                      // sequential -- inner accumulation
        output += lhs * rhs;
```

Listing 2.55: Croqtile s Two-Layer Parallelism Model


Logical Parallelism → Physical Hardware
What you write
(logical structure)
Where it runs
(GPU hardware)
concurrent
parallel {px,py} by [8,16]
: block
Grid: 8×16 = 128 blocks
sequential
foreach {tile_k} in [16]
Shared Memory (per block)
concurrent
parallel {qx,qy} by [16,16]
: thread
256 threads per block
sequential
foreach k in [16]
Registers (per thread)
compute: += a * b
parallel = concurrent (hardware maps it)
foreach = sequential (loop)

Left: the logical nesting the programmer writes (parallel = concurrent, foreach = sequential). Space specifiers bridge the two.

If you omit the space specifier — just `parallel p by 8` — the compiler picks a default mapping. That is what Chapters 1 and 2 did, and it worked fine for simple cases. Explicit specifiers are how you take control when performance matters.

Both `parallel` and `foreach` support multi-dimensional indices (`{a, b}` syntax), the compose operator (`#`), and the extent operator (`#p`). Everything from Chapters 1 and 2 still applies.

`: block` — parallel blocks

```
parallel {px, py} by [8, 16] : block
```

Listing 2.56: : block — parallel blocks

This creates an 8 × 16 = 128 block grid. Blocks cannot communicate with each other during execution — they are truly independent.

Use `: block` for the outermost tiling of your output. If your output is `[128, 256]` and each block handles a `[16, 16]` tile, you need `128/16 x 256/16 = 8 x 16 = 128` blocks.

`: thread` — Threads Within a Block

```
parallel {qx, qy} by [16, 16] : thread
```

Listing 2.57: : thread — Threads Within a Block

This creates 16 × 16 = 256 threads within the enclosing block.

Use `: thread` for the finest-grain parallelism — the per-element work within a tile.

Nesting Specifiers

Real kernels nest these levels. A common pattern:

```
parallel {px, py} by [8, 16] : block {
  parallel {qx, qy} by [16, 16] : thread {
    // 128 blocks x 256 threads = 32,768 threads total
  }
}
```

Listing 2.58: Nesting Specifiers

The outer `parallel` creates the block grid. The inner `parallel` creates threads within each block. The braces `{px, py}` introduce multi-dimensional indices — shorthand for two nested `parallel` lines.

`with tile_k in 16`

Opens a scoped region and binds `tile_k` as a tile axis with extent 16. Inside the block, `tile_k` is the chunk index for `chunkat` along K, and `#tile_k` is 16.

`foreach tile_k(1:)`: sliced iteration

`(1:)` means tile indices `1, 2, ...` through the end. Tile 0 was loaded in the prologue.

Parallel Region: the 'parallel-by' Block

In systems like CPU, it allows of asynchronous thread to realize the parallel execution. However, in Croqtile, it employs the Single Instruction Multiple Data (SPMD) model as it way to realize parallelization.

However, the syntax of constructing a parallel region is quite different. It employs the C-style bracket and encloses the code for parallel execution within the 'parallel-by' block.

```
parallel p by 6 {
  // SPMD code
}
```

Listing 2.59: Parallel Region: the parallel-by Block

The above code snippet illustrates the method to create a parallel region with Croqtile keyword 'parallel' and 'by'. Here, it assume there are 6 execution threads. Each of the thread execute the same SPMD code inside but with a different 'p' value. Alternatively, if you are more familiar with sequential C/C++ programming, you may consider 'p' as the iteration variable of a loop with 6 iterations. (But any iteration may go first to be executed!)

Despite parallelism, there is one more implication of 'parallel-by'. In the statement, 'p' is an integer associated with its bound [0, 6). You name 'p' as a **bounded integer** instead of a simple integer. In some special operations like 'chunkat' (explain later), it requires the *bounded-integer* to work properly since the bound is essential for its computation.

The 'with-in' Block and 'where' Clause

Similar to 'parallel-by', 'with-in' statement can also bind *i-tuples* to a *mdspan*. The below code shows an example.

```
with index in [10, 10] {
  // index is ituple with 2 elements
}
```

Listing 2.60: The with-in Block and where Clause

Here, 'index' is a *i-tuple* with 2 elements. Sometimes programmers prefer that the 2 elements being named. This is possible by using the below syntax.

```
with {x, y} in [10, 10] {...}
```

Listing 2.61: The with-in Block and where Clause

Or even to name both the *i-tuple* and its elements.

```
with index = {x, y} in [10, 10] {...}
```

Listing 2.62: The with-in Block and where Clause

You name 'index' as a **bounded ituple** in such scenarios.

While 'with-in' may seem like 'parallel-by', it operates differently. One significant difference is 'with-in' statement does not have implication for parallelism. The code block inside 'with-in' statement is sequentially executed. It does nothing more than creating the *bounded-ituple*.

In addition, programmers could append a 'where' clause. For example,

```
with {m, n} in [M, N], {n_p, k} in [N_P, K] where n_p <-> n {
  // matmul implements with m,n,K. n_p is no long useful.
}
```

Listing 2.63: The with-in Block and where Clause

The code snippet requires 'n' and 'n_p' to have an identical value in all iterations. Thus inside the 'with-in' block, it is possible to replace'n' whenever 'n_p' is required, or the opposite. Such a facility is useful to program many AI kernels. Programmers should use operation '<->' to establish such relations.

The 'foreach' Block

Once the *bounded-ituple* is defined by the 'with-in' clause, programmers can loop over the bounded-ituples/bounded-integers. In Croqtile, this is simple.

```
with x in [10] {
  foreach x {
    // do something with each x
  }
}
```

Listing 2.64: The foreach Block

Basic Syntax: `parallel-by`

The basic syntax for a `parallel-by` block is as follows:

```
parallel p by 6 {
  // SPMD (Single Program, Multiple Data) code
}
```

Listing 2.65: Basic Syntax: parallel-by

You name the code inside `{}` as the SPMD-style **parallel execution block**, and the instances of the code as the **parallel thread**. In this example:

- `parallel`: This keyword initiates the *parallel execution block*.
- `p`: It is the **parallel variable**. It contains an integer value identifying the current *parallel thread*. Each *parallel thread* runs the same code but with a different value of *parallel variable* `p`.
- `by 6`: It indicates that there are totally 6 *parallel threads*. Thus, the *parallel variable* `p` ranges from `0` to `5` across different threads.

In some scenarios, programers may not require an explicit *parallel variable* for a simple parallel construct. It is possible to omit it:

```
parallel by 2 { ... }
```

Listing 2.66: Basic Syntax: parallel-by

This invokes two parallel threads to execute. However, it is not possible for the two threads to work on different data, which is normally required by SPMD programs for a data-parallel processing purpose.

Multiple-Level Parallelism

In Croqtile, you can define multiple levels of parallelism. Here's an example:

```
parallel p by 2 {
  // parallel-level-0
  parallel q by 12 {
    // parallel-level-1
  }
}
```

Listing 2.67: Multiple-Level Parallelism

This code defines two levels of parallelism with values `2` and `12`, respectively. This means a total of `2 x 12 = 24` parallel threads are invoked for the task. Each thread is identified by a unique combination of `p` and `q`. In Croqtile, you refer to the level of `p` as *parallel-level-0* and the level of `q` as *parallel-level-1*.

One key difference observable by programmers is the **synchronization cost**. This hierarchical design encourages programmers to place the most divergent code (that requires synchronization) within the **block** level to enhance overall performance.

In Croqtile syntax, you can define multiple-level parallelism in a single line:

```
parallel p by 2, q by 12 { ... }
```

Listing 2.68: Multiple-Level Parallelism

This achieves the same parallelization as the previous example. It uses the **comma-separated** *parallel-by* expression, assuming there is no code in the between two parallel levels.

Sub-Level Parallelism

Targets like allow a single parallelism level to be divided into up to three sub-levels. In Croqtile, `ituple` and `mdspan` are used for this syntax:

```
parallel {px, py, pz} by [1, 1, 2] {
  // higher synchronization cost
  // ...
  parallel t_index = {qx, qy} by [3, 4] {
    // lower synchronization cost
    // ...
  }
  // ...
}
```

Listing 2.69: Sub-Level Parallelism

In this example, two levels of parallelism are defined. The first level (*level-0*) is subdivided into three sub-levels, annotated by the `{px, py, pz} ituple`. The second level (*level-1*) is subdivided into two sub-levels, annotated by the `{qx, qy} ituple`, or `t_index` as a whole. For each sub-level, the parallelization count corresponds to the respective value inside the `mdspan`.

Note that the first element inside the `ituple` represents the **least significant parallel variable**. Croqtile adopts this ordering to align with developers' conventions, rather than for internal consistency.

In this section, you will learn about constructing *bounded variable*s using the `with-in` statement and creating loops with the `foreach` statement.

Loops in Croqtile

In C++, loops are typically constructed using `for`, `while`, and `do-while` blocks. These loops include conditions for termination and usually involve variable manipulations related to loop control. Programmers have the freedom to define exit conditions, loop stepping, and other aspects, allowing for flexible loop structures.

In contrast, Croqtile, being a domain-specific language for data movement, supports a more constrained loop structure. The current loop construction in Croqtile is as follows:

```
foreach index {
  // loop body
}
```

Listing 2.70: Loops in Croqtile

Here, `index` is a *bounded variable* (introduced in the previous section). If `index` has an upper bound of `6`, the above code is equivalent to:

```
for (int index = 0; index < 6; ++index) {
  // loop body
}
```

Listing 2.71: Loops in Croqtile

While *bounded variables* may seem restrictive, they are sufficient for data movement tasks, as you typically won't move data with a negative or out-of-bound index.

In Croqtile, *bounded variables* defined within a `parallel-by` statement are immutable and cannot be used in `foreach` statements. Instead, programmers should use the `with-in` statement to define the *bounded variables* suitable for loops.

Defining *Bounded Variables*:

A `with-in` statement allows you to define either a *bounded integer* or a *bounded ituple*. Syntactically, it resembles the `parallel-by` statement but only defines *bounded variables* without invoking multiple instances for execution. Below are examples:

```
with x in 128 {
  // 'x' is a bounded integer
}
with y in [512] {
  // 'y' is a bounded ituple
}
with index in [10, 10] {
  // 'index' is a bounded ituple
}
```

Listing 2.72: Defining Bounded Variables:

Here, `x` represents a bounded integer with an upper bound of `128`. Both `y` and `index` represent bounded ituples whose upper bounds are `512` and `10, 10` respectively. Similar to the `parallel-by` statement, you can name the elements of the ituple for clarity or declare both the bounded ituple and the associated bounded integers, as shown below:

```
with {x, y} in [10, 10] {
  // x and y are explicitly named elements of the ituple
}
with index = {x, y} in [10, 10] {
  // 'x', 'y', and 'index' can be used within the block
}
```

Listing 2.73: Defining Bounded Variables:

Like the `parallel-by` statement, multiple `with-in` declarations can be combined using a comma-separated syntax, as demonstrated below:

```
with index = {x, y} in [10, 10], idx in [100, 10] { }
```

Listing 2.74: Defining Bounded Variables:

In this way, two bounded ituples are defined. Note that either `index` or `idx` can only be referenced within the following `with-in` block.

The `with-in` statement defines *bounded variable*s, which can then be iterated over using the `foreach` statement. The basic syntax is as follows:

```
with index in [6] {
  foreach index {
    // Perform operations with each index
  }
}
```

Listing 2.75: Basic Syntax

In this example, the `foreach` block iterates 6 times, with the value of the **iteration variable** `index` ranging from `0` to `5` incrementally.

It is also possible to iterate over *bounded ituple*s. For example:

```
with index = {x, y} in [6, 17] {
  foreach index { }
}
```

Listing 2.76: Basic Syntax

This code defines a *bounded ituple* `index`. Iterating over `index` is equivalent to a nested loop structure:

```
for (int x = 0; x < 6; ++x)
  for (int y = 0; y < 17; ++y) { }
```

Listing 2.77: Basic Syntax

Notice the ordering of the loop nesting: the **left-to-right** order of the *bounded variable*s within a *bounded ituple* corresponds to the **outer-to-inner** nesting of the loops. This ordering is essential for correct code behavior constructing sometimes.

Furthermore, This rule also applies to the *comma-separated bounded variable list* that follows the `foreach` statement:

```
with index = {x, y} in [6, 17], iv in [128] {
  foreach iv, index { }
}
```

Listing 2.78: Basic Syntax

In this code, two bounded variables are defined. The following `foreach` block is equivalent to a multi-level nested loop:

```
for (int iv = 0; iv < 128; ++iv)
  for (int x = 0; x < 6; ++x)
    for (int y = 0; y < 17; ++y) { }
```

Listing 2.79: Basic Syntax

Syntactic Sugar

```
foreach x in 128 { }
foreach idx in [10, 20] { }
foreach {y, z} in [8, 16] { }
```

Listing 2.80: Syntactic Sugar

The code above is equivalent to the following code:

```
with x in 128 {
  foreach x { }
}
with idx in [10, 20] {
  foreach idx { }
}
with {y, z} in [8, 16] {
  foreach y, z { }
}
```

Listing 2.81: Syntactic Sugar

Deriving the Loop From a Bounded Integer

In certain scenarios, such as pipelining data movement, it may be necessary to modify loop iterations. In Croqtile, this can be achieved by deriving a loop from a *bounded integer* within the `foreach` statement. For example:

```
with {x, y} in [6, 17] {
  foreach x, y(1::) { }
}
```

Listing 2.82: Deriving the Loop From a Bounded Integer

In this case, the **Range Expression** `y(1::)` is used to derive the loop. This results in equivalent C/C++ code:

```
for (int x = 0; x < 6; ++x)
  for (int y = 1; y < 17; ++y) { }
```

Listing 2.83: Deriving the Loop From a Bounded Integer

As seen in the code, the `y`-loop starts at `1`. The *range expression* consists of a *bounded variable*, followed by braced enclosing three colon-separated integer values, like the below form:

```
  bounded-variable(lower-offset:upper-offset:stride)
```

Listing 2.84: Deriving the Loop From a Bounded Integer

This derives a loop from the `bounded-variable`, where the `bounded-variable` serves as the *iteration variable* of the loop. Specifically:

- The initial value of the *iteration variable* is the `lower-offset` plus the lower bound of the `bounded-variable`. Since all bounded integers have a lower bound of `0` in Croqtile, the `lower-offset` sets the initial value of the loop.
- The loop terminates when the *iteration variable* is equal to or greater than the upper bound of the `bounded-variable` plus `upper-offset`. Negative values are typically used for `upper-offset`.
- The *iteration variable* increments by `stride` at the end of each iteration.

Thus, `y(1:-1:2)` results in a loop like `for (y = 0 + 1; y < 17 - 1; y += 2)` in the example above. If any field of the *range expression* is not specified, it results in default values: `0` for `lower-offset` and `upper-offset`, and `1` for `stride`.

Note that *range expressions* only apply to the *bounded variables*. *Range expression* over the *bounded ituple* triggers an error at compile time.

In short In this section, you explain the use of `with-in` and `foreach` statements in Croqtile to define and iterate over *bounded variables*, which are essential for data movement tasks. You introduced the syntax and behavior of these constructs, including how to apply range operations to modify loop iterations and the importance of understanding the current and upper-bound values of bounded variables.

The loop deriving part is important for implementing multi-buffering data movement, which is essential for building high-performance kernels and will be introduced in optimization chapters later.

2.8 Tile Selection: chunkat, subspan, and view/from

A tile is a slice of a larger buffer. This section shows the ways to select one — ***`chunkat`****,* ***`subspan`****,* ***`view().from()`*** *— and the compose operator* ***`#`****.*

`chunkat(...)`: equal partitioning

```
lhs.chunkat(tr, tc)
```

Listing 2.85: chunkat(...): equal partitioning

`chunkat` divides each dimension into equal chunks based on the surrounding parallel structure. If `lhs` has shape `[64, 128]` and the kernel uses `parallel {tr, tc} by [4, 8]`, then `lhs.chunkat(tr, tc)` selects a `[16, 16]` tile.

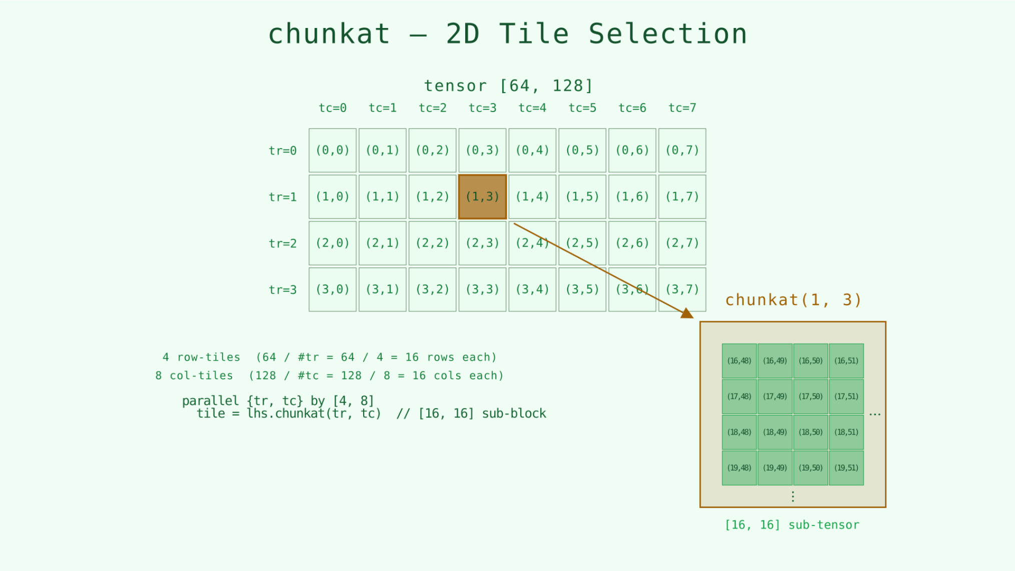
That makes `chunkat` the natural tool when your tiling follows the parallel decomposition directly.

`chunkat(1, 3)` selects one equal tile from a regular tiling grid.

`subspan(...).at(...)`: explicit tile extents on a tile grid

```
lhs.subspan(64, 64).at(block_m, iv_k)
```

Listing 2.86: subspan(...).at(...): explicit tile extents on a tile grid


chunkat — 2D Tile Selection
tensor [64, 128]
tc=0
tc=1
tc=2
tc=3
tc=4
tc=5
tc=6
tc=7
tr=0
(0,0)
(0,1)
(0,2)
(0,3)
(0,4)
(0,5)
(0,6)
(0,7)
tr=1
(1,0)
(1,1)
(1,2)
(1,3)
(1,4)
(1,5)
(1,6)
(1,7)
tr=2
(2,0)
(2,1)
(2,2)
(2,3)
(2,4)
(2,5)
(2,6)
(2,7)
tr=3
(3,0)
(3,1)
(3,2)
(3,3)
(3,4)
(3,5)
(3,6)
(3,7)
chunkat(1, 3)
(16,48)
(16,49)
(16,50)
(16,51)
(17,48)
(17,49)
(17,50)
(17,51)
(18,48)
(18,49)
(18,50)
(18,51)
(19,48)
(19,49)
(19,50)
(19,51)
[16, 16] sub-tensor
4 row-tiles (64 / #tr = 64 / 4 = 16 rows each)
8 col-tiles (128 / #tc = 128 / 8 = 16 cols each)

```
parallel {tr, tc} by [4, 8]
  tile = lhs.chunkat(tr, tc)  // [16, 16] sub-block
```

`subspan` says what the tile shape is. `.at(...)` says which tile instance you want. This is the better fit when the tile shape is an explicit algorithmic choice, such as a GEMM tile of `[64, 64]`, rather than something you want inferred from `parallel by ...`.

`view(...).from(...)`: explicit tile extents with explicit element offsets

```
lhs.view(TILE_M, TILE_K).from(offset_m, offset_k)
```

Listing 2.87: view(...).from(...): explicit tile extents with explicit element offsets

`view(...).from(...)` is the most general selection form introduced in this part of the guide. `view(...)` states the shape of the region you want, and `.from(...)` gives its starting coordinate in element space.

This is the right tool when the tile origin is already available as element offsets rather than tile indices, or when the region is not most naturally described as an equal chunk. It is especially useful for dynamic and irregular access patterns where `chunkat(...)` would be awkward or impossible.

You can think of the three forms like this:

- Use `chunkat(...)` when the surrounding parallel decomposition already defines an equal tiling.
- Use `subspan(...).at(...)` when you want an explicit tile shape and tile coordinates.
- Use `view(...).from(...)` when you want an explicit tile shape and element-space starting offsets.

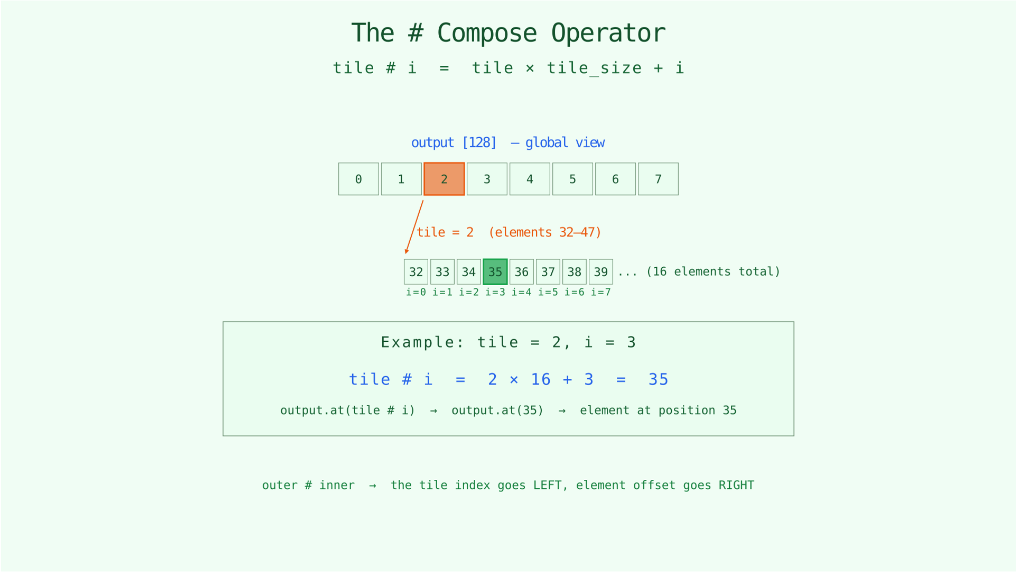
Compose: `outer # inner`

```
output.at(tr # i, tc # j)
```

Listing 2.88: Compose: outer inner

This combines a tile coordinate with an in-tile coordinate to produce a coordinate in the full tensor.

Read `tr # i` as "row `i` inside tile `tr`."


The # Compose Operator
tile # i = tile × tile_size + i
output [128] — global view
0
1
2
3
4
5
6
7
tile = 2 (elements 32–47)
32 33 34 35 36 37 38 39 ... (16 elements total)
i=0 i=1 i=2 i=3 i=4 i=5 i=6 i=7
Example: tile = 2, i = 3
tile # i = 2 × 16 + 3 = 35
output.at(tile # i) → output.at(35) → element at position 35
outer # inner → the tile index goes LEFT, element offset goes RIGHT

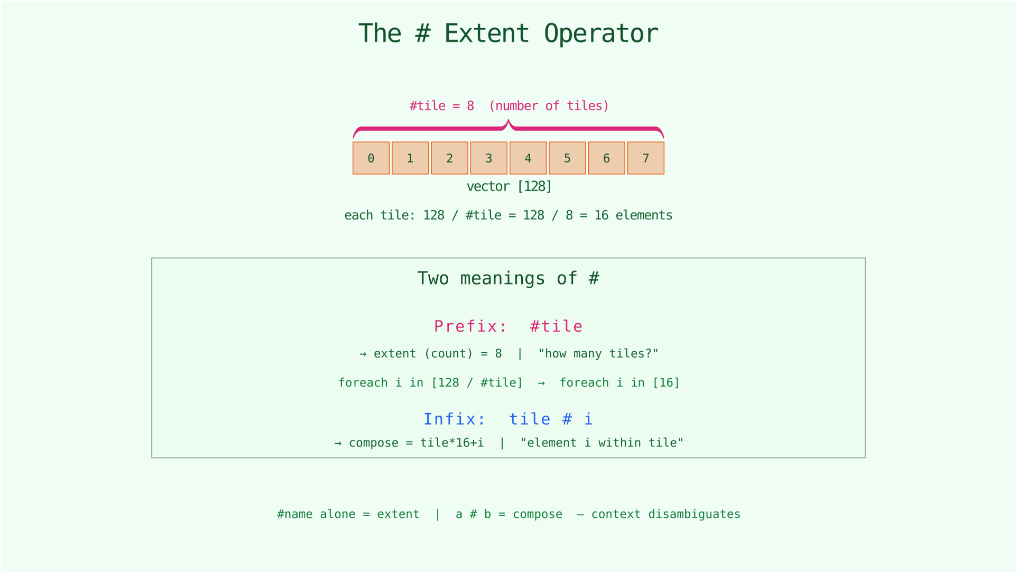
Compose a tile index with an in-tile offset to recover a global coordinate.

Extent: `#name`

```
foreach i in [128 / #tile]
```

Listing 2.89: Extent: name

Here `#tile` means the extent of the `tile` axis: how many tiles exist along that dimension. If `parallel tile by 8`, then `#tile` is 8.


The # Extent Operator
#tile = 8 (number of tiles)
0
1
2
3
4
5
6
7
vector [128]
each tile: 128 / #tile = 128 / 8 = 16 elements
Two meanings of #
Prefix: #tile
→ extent (count) = 8 | "how many tiles?"
foreach i in [128 / #tile] → foreach i in [16]
Infix: tile # i
→ compose = tile*16+i | "element i within tile"
#name alone = extent | a # b = compose — context disambiguates

Prefix `#` asks for an extent. Infix `#` composes coordinates.

Bounded-ituple/integer and 'chunkat' Operation

'chunkat' is an operation performed on spanned data. It creates a new *mdspan* over the existing data. Thus in certain systems, it is referred to as 'subview'. However, as 'chunkat' accepts bounded-ituple and bounded-integer as parameters, it is named differently in Croqtile.

```
global f32 [6, 10, 100] data;
parallel p by 6 {
  with index = {x, y} in [10, 10] {
    // for every data move, the stride into 'data' is
    //    stride = p*1000 + x * 100 + y * 10
    //
    // for each chunk, the dimensioned size for the movement is {1, 1, 10}
    f = dma.copy data.chunkat(p, index) => local;
  }
}
```

Listing 2.90: Bounded-ituple/integer and chunkat Operation

The example above shows one typical use of `chunkat`. Here the spanned data has type `f32 [6, 10, 100]`. The operation takes two parameters — the *integer* `p` and the *ituple* `index` — and divides the data into 6 × 10 × 10 pieces, one for each combination of their ranges. Each piece holds 10 consecutive elements along the least significant dimension and is copied into local storage.

This section introduces the *chunkat expression* for the DMA statement, which is the core concept of Croqtile that simplifies data movement coding.

Ubound Operation

In the previous chapter, you learned that a *bounded variable* defined by a `parallel-by` or `with-in` statement is either an *integer* or an *ituple* with one or more bounds. You also learned how to use the *current value* and *upper-bound value* of the *bounded variable* to construct loops in `foreach` blocks, where the *upper-bound* is implicitly used.

However, in Croqtile, you can use the **ubound operation** to explicitly retrieve the upper bound of a bounded variable. *Ubound* is a unary operation that uses the prefix symbol `#` before a bounded variable. For example:

```
with size in [32] {
  local u8 [#size, 16] a; // buffer shape: [32, 16]
  local u8 [size, 16] b; // error in shape: [0, 16]
}
```

Listing 2.91: Ubound Operation

In this code, the local buffer `a` is shaped as `[32, 16]`, since `#size` evaluates to the upper-bound of `size`. However, the declaration of `b` uses the *current value* of `size`, resulting in an incorrect shape definition.

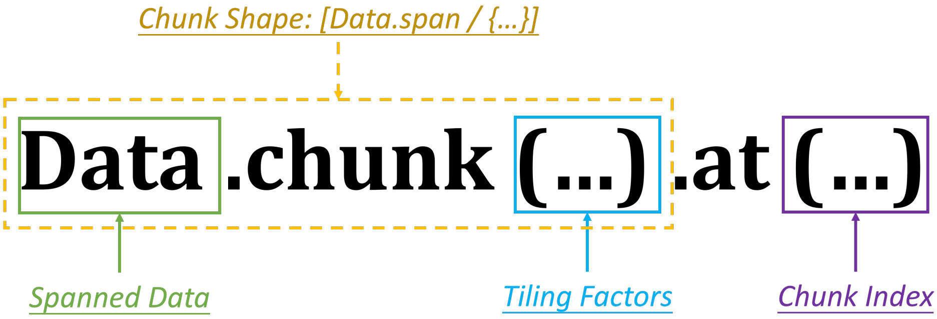
Use `chunkat` Expression to Tile Data

In all the examples from the previous section, the *data expressions* of the *DMA statement* directly utilized the *spanned* data or *storage* annotation. However, programmers can use the **chunkat** expression to enable tiling behavior in practice. For example:

```
__co__ void foo(f32 [36] input) {
  parallel p by 6 {
    dma.copy input.chunkat(p) => shared;
  }
}
```

Listing 2.92: Use chunkat Expression to Tile Data

In this example, the built-in member function `.chunkat` is used to create a tile or block for the source expression of the DMA statement. Each tiled block is called a **chunk** in Croqtile. The `.chunkat(...)` data expression can be considered a simpler form of `.chunk(...).at(...)`, where the semantics of each part are illustrated in the figure below:



Considering the expression `input.chunkat(p)`, which is equivalent to `input.chunk(#p).at(p)`:

- `input` is the *spanned data* to be tiled.
- `#p` indicates the **tiling factor**. It also specifies the total **chunk count** for each dimension to process. In this case, the 1D input is tiled by `6`, resulting in `6` chunks to be processed.
- `.chunk(#p)` determines the **chunk shape**. The chunk is a 1D data segment tiled from `input`, with its size being the data *mdspan* divided by the *upper-bound* of the bounded variable. In this example, the chunk shape is calculated as `[input.span / #p] = [6]`.
- `.at(p)` determines the **chunk index**. Here, `p` ranges from `[0, 6)`, allowing parallel threads to process different chunks of `input`. Specifically, parallel thread `1` processes a chunk with elements `[6, 12)` from `input`, while parallel thread `5` processes a chunk with elements `[30, 36)`, and so on.

In many scenarios, it is unnecessary to use separate `.chunk` and `.at` expressions, as `.chunkat` takes bounded variables as parameters. The upper-bounds of these variables act as the *tiling factors*, and their current values act as the *chunk index*.

Use Multiple Bounded Variables

The following code showcases an example that utilizes multiple bounded variables:

```
__co__ void foo(f32 [36, 14, 8] input) {
  parallel p by 6 {
    with index in [7, 4] {
      foreach index {
        dma.copy input.chunkat(p, index) => shared;
      }
    }
  }
}
```

Listing 2.93: Use Multiple Bounded Variables

In this example, there are two bounded variables: `p` is the *bounded integer* defined by the `parallel-by` statement, while `index` is the *bounded ituple* with two integers. The `chunkat` expression concatenates them to form a *bounded ituple* with upper-bounds `{#p, #index}` and current values `{p, index}`. Thus, the shape of each chunk is calculated as `[36, 14, 8] / {#p, #index} = [36, 14, 8] / {6, 7, 4} = [6, 2, 2]`.

In addition, since `p` is the bounded variable for parallelization, there are `6` threads working in parallel, each handling a large chunk of `[6, 14, 8]`. Within a single parallel thread, it iterates `7x4` times, each time moving a `[6, 2, 2]` small chunk to a compiler-allocated `shared` storage.

With this approach, you can apply tiling to data movement, where each iteration in each parallel thread selects only a part of the data/buffer for transfer. The Croqtile compiler uses information about the *spanned* data and the *bounded variables* to transpile the DMA statement into C++ subscripting, DMA configurations, etc., simplifying the trivial and error-prone work that C++ programmers typically face. This makes learning the Croqtile syntax worthwhile.

In short In this section, you learned about using the `chunkat` expression in Croqtile for tiling data in DMA statements, which allows dividing data into smaller chunks for efficient processing. You explored how `chunkat` can be applied to either the source or destination of a DMA statement to manage data movement between memory and processors.

Now, you may try to write a full tileflow program. Wish you good luck!

2.9 Data Movement: the DMA Statement and Futures

Moving a tile is a statement in the language, not a library call. This section introduces the DMA statement, futures, and synchronous versus asynchronous copies.

The DMA Statement Family

All of Croqtile's tile-movement operations use the same basic form:

```
future = engine.operation.modifiers source => destination;
```

Listing 2.94: The DMA Statement Family

The important pieces are:

Part	Meaning
`future`	Optional handle to the transfer result. Use `future.data` to access the moved tile and `future.span` to query its shape.
`operation`	`copy`, `transp<...>`, or `pad<...>`.
`modifiers`	Optional flags such as `.async`, `.zfill`, or `.swiz<N>`.
`source => destination`	The source view and the destination memory or destination view.

Minimal examples:

```
f = dma.copy input.chunkat(block) => shared;
```

```
g = dma.transp<1, 0> matrix => local;
h = dma.pad<{2, 1}, {3, 2}, {0, 0}, 0> tile => shared;
```

Listing 2.95: The DMA Statement Family

The key shift from Section 1.7 is that these statements move shaped regions, not individual elements.

First Example: Tiled Addition Through shared storage

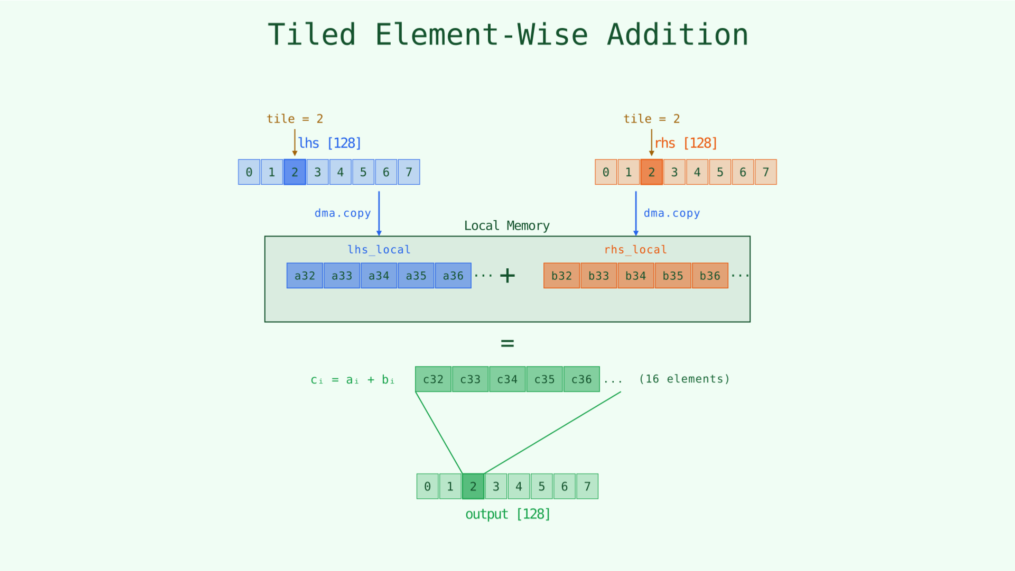
```
__co__ s32 [64, 128] tiled_add_2d(s32 [64, 128] lhs, s32 [64, 128] rhs) {
  s32 [lhs.span] output;
  parallel {tr, tc} by [4, 8] {
    lhs_load = dma.copy lhs.chunkat(tr, tc) => shared;
    rhs_load = dma.copy rhs.chunkat(tr, tc) => shared;
    foreach {i, j} in lhs_load.span
      output.at(tr # i, tc # j) =
          lhs_load.data.at(i, j) + rhs_load.data.at(i, j);
  }
  return output;
}
```

Listing 2.96: First Example: Tiled Addition Through shared storage

The arithmetic is still element-wise addition. What changed is the data path:

1. `chunkat(tr, tc)` selects one logical tile from each input.
2. `dma.copy ... => shared` stages that tile into block-visible fast memory.
3. `tr # i` and `tc # j` map tile-local coordinates back to global output coordinates.

This is the core Croqtile pattern: select a tile, move it, compute on the moved tile.


Tiled Element-Wise Addition
tile = 2
lhs [128]
0 1 2 3 4 5 6 7
tile = 2
rhs [128]
0 1 2 3 4 5 6 7
dma.copy
dma.copy
Local Memory
lhs_local
a32 a33 a34 a35 a36 ···
+
rhs_local
b32 b33 b34 b35 b36 ···
=
cᵢ = aᵢ + bᵢ
c32 c33 c34 c35 c36 ...
(16 elements)
0 1 2 3 4 5 6 7
output [128]

Load a tile, compute on the staged data, then write results back using global coordinates.

Futures, `.data`, and `.span`

The result of a DMA or TMA statement is a future-like handle:

```
f = dma.copy input => shared;
```

Listing 2.97: Futures, .data, and .span

That handle gives you two important things:

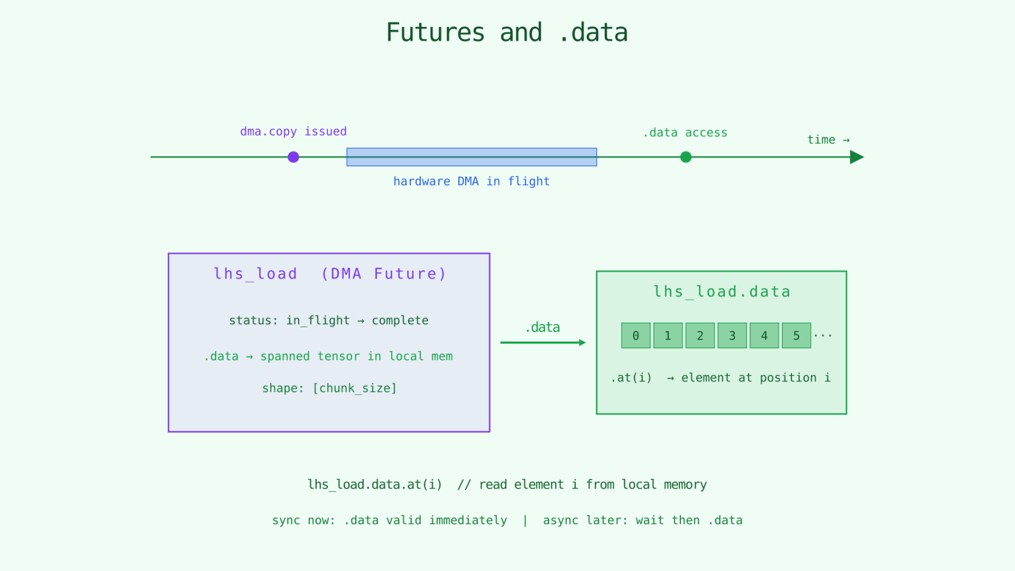
- `f.data`: the moved tile, as a spanned tensor.
- `f.span`: the shape of that tile.

Example:

```
f = dma.copy input => shared;
foreach idx in [f.span]
  output.at(idx) = f.data.at(idx);
```

Listing 2.98: Futures, .data, and .span

Even synchronous copies are worth naming when you want to pass the moved tile to later code.


Futures and .data
dma.copy issued
.data access
time →
hardware DMA in flight
lhs_load (DMA Future)
status: in_flight → complete
.data → spanned tensor in local mem
shape: [chunk_size]
.data
lhs_load.data
0
1
2
3
4
5
···
.at(i) → element at position i
lhs_load.data.at(i) // read element i from local memory
sync now: .data valid immediately | async later: wait then .data

The transfer result is a handle. Read the copied tile through `.data` *and its shape through* `.span`*.*

Synchronous and Asynchronous Copies

By default, copies are synchronous:

```
f = dma.copy input => shared;
dma.copy f.data => output;
```

Listing 2.99: Synchronous and Asynchronous Copies

Add `.async` when you want the transfer to proceed in the background:

```
f = dma.copy.async input => shared;
wait f;
dma.copy f.data => output;
```

Listing 2.100: Synchronous and Asynchronous Copies

The same pattern applies to TMA:

```
f = tma.copy.async input => shared;
wait f;
tma.copy f.data => output;
```

Listing 2.101: Synchronous and Asynchronous Copies

Rule: if you read `f.data` from an async transfer, wait first.

Later chapters use async copies to overlap load and compute. In this part of the guide, the important part is the contract: `.async` creates an in-flight transfer, and `wait` makes it safe to consume.

`dma.transp<...>`

```
fa = dma.transp<1, 0, 2> a => local;
dma.copy fa.data => o;
```

Listing 2.102: dma.transp ...

This permutes dimensions during the movement itself. You do not first copy and then transpose. The transfer produces data in the transposed layout.

`dma.pad<...>`

```
f = dma.pad<{2, 1}, {3, 2}, {0, 0}, V>.async input => shared;
wait f;
dma.copy f.data => output;
```

Listing 2.103: dma.pad ...

Padding creates a larger destination tile. The template arguments describe low padding, high padding, internal stride-style padding, and the fill value. Use `pad` when you want explicit border values, not just out-of-bounds protection.

Tileflow Program: Orchestrating the Data Movement

The *Tileflow Program* consists of *Croqtile functions*. As described earlier, it manages the movement of data between the host and the target device, ensuring that data is copied correctly across different storage locations.

For convenience, let's revisit the code:

```
__co__ s32 [6, 17, 128] ele_add(s32 [6, 17, 128] lhs, s32 [6, 17, 128] rhs) {
  s32 [lhs.span] output; // Use same shape as lhs
  parallel p by 6 {
    with index in [17, 4] { // Tiling factors
      foreach index {
        lhs_load = dma.copy lhs.chunkat(p, index) => local;
        rhs_load = dma.copy rhs.chunkat(p, index) => local;
        local s32 [lhs_load.span] l1_out;
        // Call kernel with loaded data
        call kernel(lhs_load.data, rhs_load.data, l1_out, |lhs_load.span|);
        // Store result back to output
        dma.copy l1_out => output.chunkat(p, index);
      }
    }
  }
  return output;
}
```

Listing 2.104: Tileflow Program: Orchestrating the Data Movement

In this code, the `__co__`-prefixed croqtile function accepts two inputs `lhs` and `rhs`, both with the shape `[6, 17, 128]` and the element type of `s32` (signed 32-bit integer). And the output is defined to have the same shape and type as the inputs.

The `parallel p by 6 {...}` block indicates that the enclosed code runs in parallel. Specifically, six instances of the code are executed concurrently. This implies a transition in the execution environment from the host to the device.

Inside the `parallel-by` block, a `with-in` block binds the symbol `index` to two values, `17` and `4`. In Croqtile, `index` is referred to as a `bounded-ituple` with two `bounded` values, which can be used in `foreach` statements. (You will explain `bounded` types in later chapters.)

The `foreach index {...}` statement is equivalent to the following C code:

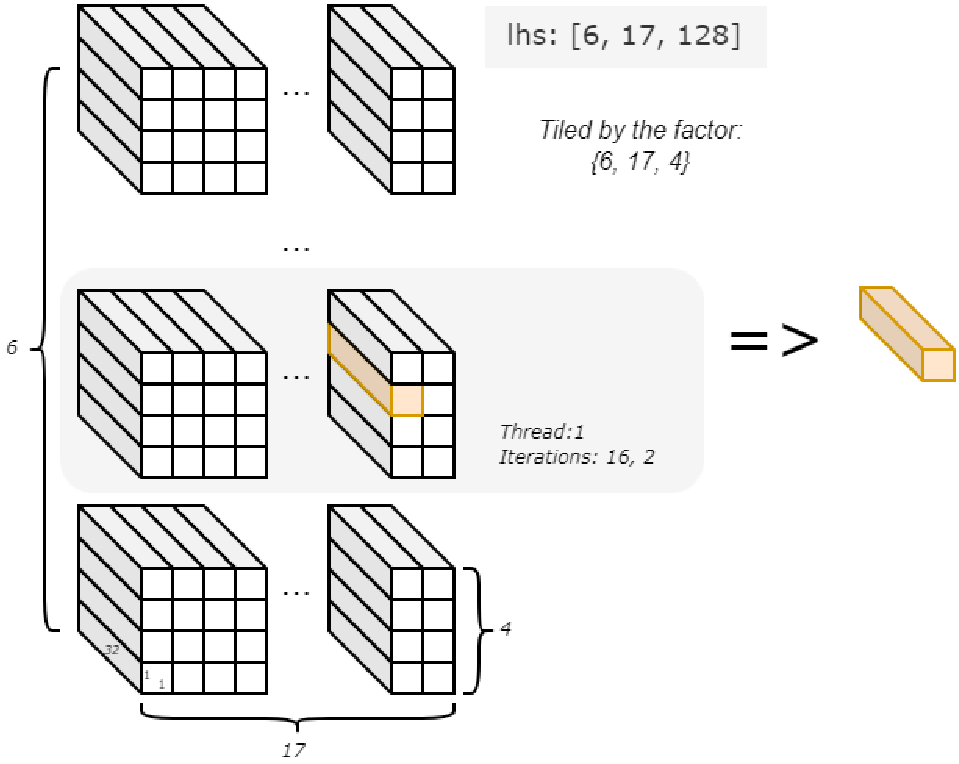
```
for (int x = 0; x < 17; x++)  // assume 'x' represents the 1st element of 'index'

  for (int y = 0; y < 4; y++) { ... }  // and 'y' represents the 2nd element of '
    index'
```

Listing 2.105: Tileflow Program: Orchestrating the Data Movement

Within the `foreach` block, the `dma.copy` statement describes how data movement occurs. For example, consider the statement `lhs_load = dma.copy lhs.chunkat(p, index) => local;`:

- The symbol `lhs_load` is called **future** of the DMA operation, which contains information about the DMA destination.
- `dma.copy` invokes a direct DMA data transfer without transforming the shape of the data. The expression on the left-hand side of `=>` represents the DMA source, while the right-hand side represents the destination.
- In this case, the destination is specified as a `local` buffer, which is automatically allocated by the Croqtile compiler.
- The source expression `lhs.chunkat(p, index)` is referred to as the **chunkat expression** in Croqtile. Here, `p` and `index` tiling factors for the `lhs`. Given that `lhs` has a shape of `[6, 17, 128]` and the upper bounds of `p` and `index` are `6`, `17`, and `4`, the data chunk size is `1x1x32` (`6/6, 17/17, 128/4`). In each iteration, a single data chunk is used as the source, with the exact chunk determined by the current values of `p` and `index`. For example, in parallel thread `1` and the iteration `{16, 2}`, the chunk's offset is set to `{1, 16, 2}`.

This is illustrated in the below figure:



With the DMA statement, different chunks of data tiled from `lhs` are moved iteratively and in parallel from the host to the device's `local` storage. Similarly, the DMA statement manages `rhs` by moving it in small chunks to `local` storage for processing.

Next, the statement `local s32 [lhs_load.span] l1_out;` defines a per-parallel-thread buffer. Note here it takes its shape from the expression `lhs_load.span`, which represents the tiled block. The buffer is used to store the output data in the subsequent `call` statement. The `call` statement invoke the *device function* named `kernel` to perform computations. Once the computation is complete, another DMA statement moves the output data from the `local` buffer back to host. This complete one iteration.

In this code, each parallel thread runs `17x4` iterations, with each iteration handling a `1x1x32`-sized chunk of data. The *croqtile program* terminates when all `6` parallel threads have completed their iterations. It then returns the output buffer to its caller, the host program.

The data movement is abstracted as the most complicated statement in Croqtile. This section will introduce you the basic *DMA statement* structure and the future variable they produce.

A DMA statement in Croqtile can be an asynchronous entity, meaning that the code following an **asynchronous DMA** executes in parallel with the DMA statement. For a *asynchronous DMA*, explicit synchronization is required before using the DMA result.

To support these features, the syntax is organized as follows:

```
future = dma-op src-expr => dst-expr;
```

Listing 2.106: Basic Syntax

The statement defines a **future**-typed variable, which appears on the left side of the operator `=`. In Croqtile, **a *future* represents both the handle of the asynchronous execution instance and the DMA result**. Therefore, the definition of *future* can be ignored for *synchronized DMA* statements.

The right-hand side of the **DMA operation** includes the **operation type** (`dma-op`), the **data expression** for the operation source (`src-expr`), and the *data expression* for the destination (`dst-expr`). The source and destination are separated by the symbol `=>`, which indicates the direction of data flow.

Synchronous and Asynchronous

Asynchronous DMA, or **Async-DMA**, is also known as **Non-Blocking DMA** because it does not block the execution of subsequent code. Conversely, **Synchronous DMA**, or **Sync-DMA**, is referred to as **Blocking DMA** because the code does not proceed until the DMA operation is complete. These terms will be used interchangeably in the following text.

In Croqtile, the `.async` suffix in a **DMA operation** indicates that the operation is asynchronous. The following code provides an example:

```
f0 = dma.copy.async data0 => shared; // non-blocking
f1 = dma.copy data1 => shared;       // blocking
...
wait f0;                             // synchronization
```

Listing 2.107: Synchronous and Asynchronous

In this example, the DMA operation that produces the future `f0` is asynchronous, allowing the program to continue executing while the data transfer occurs in the background. The DMA operation that produces `f1` is synchronous, meaning the program will pause until the data is fully transferred from `data1` to `shared` storage. Meanwhile, the data copy of `data0` might still be in progress.

Executing the statement `wait f0;` causes the program to check if `data0` has been fully transferred to `shared` storage. This synchronizes the main execution flow with the asynchronous DMA of `f0`. Therefore programmers must make sure there is no "dangling future* left in your code.

Data Expression

So far, you have only used simple data expressions, either a defined **spanned data** or a **storage location**. The **storage location** is used solely for the **destination buffer** declaration, as it requires the Croqtile compiler to allocate memory for it. The following code provides an example:

```
global f32 [32, 16] input;
shared f32 [32, 16] output;
dma.copy input => output;  // Copy to a user-declared buffer
dma.copy input => shared;  // Same, but to a compiler-allocated buffer
```

Listing 2.108: Data Expression

In this example, the destination of the second DMA operation is specified as the storage location `shared`. This requires the compiler to allocate storage accordingly, making it equivalent to the first DMA operation. In practice, you recommend programmers use storage locations as destinations, as Croqtile can deduce the destination shape from the **DMA statement**.

However, the use of simple **data expressions** is rare in practice. In the next section, this section introduces the **ChunkAt** expression to demonstrate how to implement tiling in data expressions.

Future and Wait

Both *Sync-DMA* and *Async-DMA* operations can be used to define a *future*. As seen in earlier examples, a *future* acts as a handle for asynchronous operations and is typically used as a parameter of the `wait` statement to synchronize an *Async-DMA*.

Croqtile allows the `wait` statement to take multiple *futures*. However, using a *future* from a **Sync-DMA** in a `wait` statement will result in an error. The following code provides an example:

```
f0 = dma.copy.async input0 => shared;
f1 = dma.copy.async input1 => shared;
f2 = dma.copy input2 => shared;
...
wait f0, f1;  // Multiple wait
wait f2;      // Error: cannot wait on a sync-dma
```

Listing 2.109: Future and Wait

In Croqtile, another use of a *future* is to retrieve the destination buffer of the associated DMA statement. This is also the reason why **Sync-DMA** is allowed to define a future. Croqtile provides two built-in member functions for *future* variables:

- `.span` to retrieve the **mdspan** of the DMA destination buffer.
- `.data` to retrieve the reference of the DMA destination buffer (spanned data).

For example:

```
f0 = dma.copy.async input0 => shared;
f1 = dma.copy input2 => shared;
...
local f32 [f0.span] buffer;
call device_kernel(f1.data, |f1.span|);
```

Listing 2.110: Future and Wait

In this example, you use `.span` built-in member function to retrieve the shape of `f0`'s destination buffer, and `.data` to retrieve the *spanned* data of `f1`'s destination. In addition, the **ElementCount** operation on the *mdspan* of the destination buffer of `f1`. The **ElementCount** operation returns the number of elements in an **mdspan**, which is useful in many scenarios.

In short You explored both synchronous and asynchronous operations, and saw how Croqtile supports advanced data handling with various *DMA Operation Types*. Finally, you discovered how futures help manage async operations and retrieve buffers, ensuring smooth data flow and synchronization.

In the next section, you will step further to the *Data Expression* to see how to make data tiling/blocking happen.

2.10 Coordination: inthreads, Events, and Async Blocks

Concurrent regions have to agree on who may proceed. This section introduces `inthreads`, events and the asynchronous forms through short examples.

Event arrays for multi-stage pipelines

For pipelines with multiple buffered stages, declare event arrays:

```
shared event full[STAGES], empty[STAGES];
```

Listing 2.111: Event arrays for multi-stage pipelines

Each physical buffer slot gets its own `full`/`empty` pair. The ring index `stage = iv_k % STAGES` maps the unbounded K iteration to a fixed number of physical slots. With four stages, the producer can run several tiles ahead before blocking on `wait empty`.

Bootstrap protocol

The consumer must **seed** the `empty` credits before the K-loop starts:

```
foreach {s} in [STAGES] {
  trigger empty[s];
}
```

Listing 2.112: Bootstrap protocol

Without this bootstrap, the producer's first `wait empty[0]` blocks forever — a deadlock, not a mysterious MMA bug. This is a common pitfall: every bounded buffer protocol requires initial credits.

Section 3.8.5 develops the full double-buffered and multi-stage pipeline kernels that put these primitives to work. The examples there compose `inthreads.async`, events, `swap`, and `mma.commit` into complete, runnable matmul pipelines.

Parallel Perspective of Tileflow Programs

In Croqtile, a *divergent* code region is generated with the keyword `inthreads`, where a compare expression followed.

Let us have a look at the following example:

```
__co__  void foo() {
  // sequential code
  // start of the SPMD code region
  parallel p by 6 {
    // some uniform code for all threads
    // ...
    inthreads (p <= 2) {
      // divergent code for thread 0, 1, 2 (thread-mask: 000111)
    }
    // threads are forced to be synchronized
    // some uniform code for all threads
    // ...
  }
  // end of the SPMD code region
  // sequential code
}
```

Listing 2.113: Parallel Perspective of Tileflow Programs

You may now re-examine Croqtile tileflow programs from the perspective of parallelism. It consists of

- *Sequential Code*
- *SPMD-Styled Parallel Code* enclosed in `parallel-by` block, which consists of:
  - *Uniform Code*
  - *Divergent Code* enclosed in `inthreads` block.

In this example, `p <= 2` initiates the *divergent code block*, where from the implementation perspective, it creates a thread mask of `000111`, which allows only thread `0`, `1` and `2` to execute the code inside. And thread `3`, `4` and `5` will skip this path. **All threads are forced synchronized after the `inthreads` block**. In this case, it implies thread `3`, `4`, and `5` waits for the completion of the others before stepping forward.

In Croqtile, the `inthreads` block looks like a `if` conditional block in C/C++. However, `inthreads` explicit claims the code divergence, which is a concept in SPMD program, rather than the conditional block for sequential code. Therefore, the `inthreads` block:

- Can only appear in the SPMD code region, which is enclosed in the `parallel-by` block.
- Can only have a comparison result related to thread identifiers.

Croqtile compiler has the checks for both the comparison expression and `inthreads`-block placement.

Implicit Masking

In Croqtile, code between multiple `parallel-by` levels can be considered as a type of implicit masking. The below code showcases an example:

```
__co__ void foo() {
  parallel p by 6 {
    // implicit masking: 'inthreads (q == 0) { ... } '
    parallel q by 2 {
      // code here is for all threads
    }
    // implicit masking: 'inthreads (q == 0) { ... } '
  }
}
```

Listing 2.114: Implicit Masking

In this example, there exists two levels of parallelization. The code inside `parallel p by 6` block but outside `parallel q by 2` blocks is conceptually for the outer parallelization only. This means, only one thread from the sub-level parallelization execute the code, which is equivalent to `inthreads (q == 0) { ... }`. Programmers must be aware there is only single thread instance to execute the code part. Only the code inside the inner `parallel-by` is executed by all threads.

As `inthreads` introduce MPMD programming pattern in Croqtile, it is necessary to synchornize between different asynchronous code. You can use event in Croqtile to achieve that.

Event Basics

There are different definitions of `event` in different programming language. In Croqtile, the **Event** is used to handle communication and interaction between asynchronous code blocks at different level.

An event in Croqtile is either *SET* or *UNSET*. When declaring an event, it is initialized with the *UNSET* state. For example, the following code initialize a `shared` event:

```
shared event e;
```

Listing 2.115: Event Basics

Note that since it is a `shared` event, it can only be declared inside `parallel-by` block and used for parallel threads. Similarly, `global` and `local` events can be defined as well. There is normally restrictions syntactially for either the declarations and value references according to the target platforms.

In addition, you may declare an event array following C-style syntax:

```
shared event e[4], e1;
```

Listing 2.116: Event Basics

Note it is allowed to use comma-separated expression to define either event and event array in a single line.

In Croqtile, the `trigger` statements changes the `event` from *UNSET* to *SET* state:

```
shared event e[4], e1;
trigger e, e1;
```

Listing 2.117: Event Basics

The `trigger` statement triggers each event in an event array `e`, as well as the single event `e1`, as showed in the comma-separated list.

Similarly, the `wait` statement can wait for an event in a blocking mannar. The wait statement blocks currentthreads for execution if the event is *UNSET*, until the event state is changed to *SET*.

Therefore, triggering an event is used to unblocking code for the *wait statement*. For example,

```
parallel p by 2 {
  shared event e;                          // init ’e’ to ’UNSET’
  inthreads.async (p == 1) { wait e; }     // wait-thread
                                           // once unblocked, change ’e’ to ’
    UNSET’
  inthreads.async (p == 0) { trigger e; }  // trigger-thread
                                           // convert ’e’ to ’SET’
  sync.shared;                             // sync point
}
```

Listing 2.118: Event Basics

You have observed similar code in last section, except for that there are `wait` and `trigger` statements. In this code snippet, once the event `e` is triggered by the *trigger-thread*, the *wait-thread* is no longer blocking. As a consequent, the two asynchronous threads followed an order to reach the synchronization point defined by `sync.shared`, where thread `0` completes its `inthreads` block before thread `1`. From another perspective, the events here **chains the asynchronous `inthreads`**. This is useful in many scenarios.

Note that, in Croqtile semantics, **an event is always auto-reset by its `wait` statement**. That means when a `wait` statement is unblocked, the events that are waited are changed to *UNSET* state.

In short In this section, you learnt 'event' that can ochestrate asynchronous `inthreads` blocks. The `trigger` statements can set events and event arrays to *SET* state, while `wait` statements block execution, and reset events to *UNSET* until they are triggered.

Programmers must be careful to use Croqtile events, or else it is possible to result in deadlocks, which is hard to debug.

2.11 Branching and Early Return

Not every branch is known at compile time. This section covers runtime predicates and early return from a parallel region.

Predicated execution with `if`

Croqtile's `if` behaves like its C counterpart:

```
if (tile_id < total_tiles) {
  // body executes only when the condition is true
}
```

Listing 2.119: Predicated execution with if

All threads in scope evaluate the condition. Threads where it is false skip the body. — Threads on the skipped side sit idle while the taken side runs, then vice versa. This is the price of runtime flexibility.

When to use `if`: data-dependent decisions that cannot be resolved at compile time. Bounds checks, partial-tile guards, conditional accumulation. The condition can depend on runtime values — loop indices, input data, tile coordinates.

The practical rule: keep `if` conditions (all 32 threads agree) whenever possible.

Early return from `parallel`

Sometimes you need to early return from a `parallel` block. The `yield` keyword will simply generate a `return;` instruction to get out of device code.

```
parallel p by BLK_CNT: block {
  parallel q by THR_CNT : thread {
    if (cond) yield;
    ...
  }
}
```

Listing 2.120: Early return from parallel

Common pitfall

The `yield` keyword currently will **not** do any compile-time nor runtime checks. Be careful about synchronization safety.

2.12 The Four-Step MMA Syntax

Contraction is written as four explicit steps. This section reads that syntax once, with the smallest example that shows all four.

Every tensor-contraction kernel follows the same rhythm:

1. `mma.fill 0.0`
2. `mma.load`
3. `mma.row.row mc, ma, mb` — issue the contraction: $\mathbf{C} \mathrel{+}= \mathbf{A} \times \mathbf{B}$ into `mc`.
4. `mma.store mc, dst`

You loop steps 2–3 over K (loading the next K-slice each iteration, accumulating into the same `mc`), then run step 4 once to flush the completed output tile. — You do not declare per-lane layouts; the compiler derives them from the target and your layout choice (`row.row` here).

The `.row.row` suffix is a **layout contract** — Both operands are row-major. The full set of layout combinations is `row.row`, `row.col`, `col.row`, and `col.col`; in practice, `row.row` and `row.col` cover most kernels. Choosing the wrong variant is a correctness bug, not a performance hint

2.13 C++ Embedding, Calls, Macros, and the Preprocessor

Croqtile lives inside C++: a `.co` file can call helpers, inject source text and use the preprocessor. This section covers each mechanism in turn.

`__cpp__`: verbatim C++ injection

Whatever you place there must be valid at the splice point. The Croqtile compiler does not parse or rewrite the contents.

- `__cpp__("...")` — Ordinary string; best for short one-liners.
- `__cpp__(R"(...)")` — Raw string literal; use for `asm volatile` where escaping every `"` would be painful.

Macros inside `__cpp__` strings

A common mistake: assuming the preprocessor expands macros inside string literals.

```
#define PRODUCER_MAXNREG 40
inthreads.async (p1 == 0) {
  __cpp__(R"(asm volatile("setmaxnreg.dec.sync.aligned.u32 PRODUCER_MAXNREG;");)");
}
```

Listing 2.121: Macros inside cpp strings

This fails — the preprocessor does not expand inside strings. Use numeric literals inside `__cpp__` and `#if` / `#error` outside to enforce consistency:

```
#define PRODUCER_MAXNREG 40
#if PRODUCER_MAXNREG > 50
#error "Producer maxnreg too high for this tile config"
#endif
inthreads.async (p1 == 0) {
  __cpp__(R"(asm volatile("setmaxnreg.dec.sync.aligned.u32 40;");)");
}
```

Listing 2.122: Macros inside cpp strings

Croqtile's preprocessor runs before the main compiler pass. Macros expand in both `__co__` regions and host C++ in the same `.co` file, so one definition keeps tile geometry and host-side checks aligned.

Directive	**Role**
`#define NAME value`	Object-like macro: textual replacement
`#if` / `#elif` / `#else` / `#endif`	Conditional inclusion
`#ifdef` / `#ifndef`	Shorthand for whether a macro is defined
`#error message`	Force a compile-time failure with a message

Function-like macros (`#define MAX(a, b) ...`) are not supported. Use `constexpr` helpers in ordinary C++.

Tile geometry as macros

Production matmul sources centralize tile dimensions at the top:

```
#define MATMUL_WARP_M 64
#define MATMUL_WARP_N 128
#define MATMUL_TILE_K 64
#define MATMUL_WARP_K 16
#define MATMUL_STAGES 4
```

Listing 2.123: Tile geometry as macros

The same names appear in `parallel`, shared-memory declarations, `foreach` bounds, and host-side verification.

Compile-time assertions with `#error`

```
#if MATMUL_SWIZ != (2 * MATMUL_TILE_K)
#error "MATMUL_SWIZ must equal 2 * MATMUL_TILE_K for f16 kernel"
#endif
#if MATMUL_WARP_M != 64
#endif
```

Listing 2.124: Compile-time assertions with error

Conditional code paths

```
#define PATH0
__co__ foo() {
  #ifdef PATH0
    // path 0 code
  #else
    // path 1 code
  #endif
}
```

Listing 2.125: Conditional code paths

The preprocessor keeps one branch and discards the other before Croqtile parses the `__co__` body. Command-line defines: `croqtile kernel.co -DMATMUL_TILE_K=128`.

Function Calls and Call Croqtile Function

Croqtile functions are not allowed to call another Croqtile function. However, inside a Croqtile function, it is normal to have function calls to C++ kernels.

```
void bar() {...}    // C++ kernel function
__co__ void foo() {
  parallel p by 6 {
    call bar();     // Call the C++ function
  }
}
```

Listing 2.126: Function Calls and Call Croqtile Function

In the above example, program calls the existing C++ function 'bar' using Croqtile keyword 'call', which is intuitive.

Macros have long been a fundamental feature in C/C++ programming. To enable seamless manipulation of both C++ and *tileflow* programs, macro expansion is essential. This section demonstrates Croqtile's macro processing capabilities.

Object-Like Macros

Croqtile preprocessor supports C/C++ **object-like macros** to connect both host code and tileflow code. The *object-like macro* performs pure text replacement and accepts no parameters. The below code gives an example:

```
#define M 256
#define N 32
#define K 64
__co__ auto matmul(f32 [M, N] lhs, f32 [N, K] rhs) { /*...*/ }
void foo() {
  croqtile::f32 a[M][K];
  croqtile::f32 b[N][K];
  // ...
  auto res = matmul(croqtile::make_spanview<2>(a, {M, K}),
                    croqtile::make_spanview<2>(b, {N, K}));
}
```

Listing 2.127: Object-Like Macros

In the code snippet, the inputs of croqtile function `matmul` are not dynamically shaped. Instead, Croqtile pre-processor substitutes `M`, `N`, `K` with values `256`, `32`, `64` ahead of Croqtile compilation, which produces tileflow function with statically shaped inputs. In this way, programmers makes different code consistent and easy to manipulate.

Note that, till now Croqtile does only support macros with parameters. It means code like:

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Listing 2.128: Object-Like Macros

would not work.

Comments

Croqtile support C-style comments, either `/*...*/` or `//...`, leveraging the capability croqtile preprocessor has provided.

Conditional Compilation

Croqtile also support conditional compilation which C/C++ programs used a lot. This includes `#if`/`#ifdef`/`#ifndef`/`#else`/`#endif`. The below code snippet showcases the usage:

```
#define PATH0
// some host code
__co__ foo() {
#ifdef PATH0
// some code related to PATH0
#else
// other code
#endif
}
// host control
#ifdef PATH0
// ...
#else
// ...
#endif
```

Listing 2.129: Conditional Compilation

In this way, it also make host and croqtile code be controlled within the same preprocess method.

Note that, the capability of croqtile preprocessor is still enhancing. But it is likely that you would not implement full C preprocessing support. Croqtile would not pick up existing C features unless people find it is necessary.

In current implementation of Croqtile functions, it invokes *device function* to implement the specified target computations. This section will learn the detail about the *call statements* in Croqtile.

Call Device Functions

To call a device function in the Croqtile function, it requires the call statement explicitly led by `call`.

Basic Syntax of Call Statements

The general syntax for a `call` statement in Croqtile is:

```
call func-name <optional-template-args> (arguments);
```

Listing 2.130: Basic Syntax of Call Statements

Here, the keyword `call` is followed by a function name `func-name`, an optional `<>` enclosed template arguments, which are comma-separated. The `arguments` are also comma-separated and listed inside `()` like normal C/C++ functions.

Note that **Croqtile transpilation process would not apply any check between the callers and the callees**, including the function existence, function signature consistency, and parameter consistency. It delegates such duties to the target compilation process. Programmers must be careful about the conventions between Croqtile function and device function consequently.

Therefore, in the following example, `call bar();` in `mou` is illegal code since it tries to call device function in a code location which is assumed to be host code area.

```
__co__ void foo() {
  parallel p by 1 {
    call bar();  // ok
  }
}
__co__ void mou() {
  call bar();  // error: not able to call device function from host
}
```

Listing 2.131: Basic Syntax of Call Statements

The Convention: Allowed Argument Types

In current implementation, the argument must be either:

- *Spanned Data* type, or
- Scalar *Integer* type, or
- Scalar *Floating-Point* type.

Here, the scalar *integer* type only includes `int`, whereas the scalar *floating-point type* includes `float`, `double`, `half`, `bfp16`, and `half8`. However, since the target-platform differs in floating-point support, the *floating-point type* parameter also varies for different target platforms.

In addition, **operations on floating-point are not supported** by Croqtile till now, considering the floating is not useful for tileflow programs. Instead, they are typically values for device computation, which the *tileflow programs* pass such values directly to device kernels.

The below code showcases an example:

```
__device__ void bar(float *p, int m, int n) {}
__device__ void foo(float *, int, unsigned, double, float) {}
__co__ void foobar(f32 [M, 24] input, int N, float padding) {
  parallel p by 1 {
    shared f32 [14, 7] buffer;
    call bar(input.data, M, buffer.span(0));
    call foo(buffer.data, N, 3, 3.14, padding);
  }
}
```

Listing 2.132: The Convention: Allowed Argument Types

In this example, it passes different data types as arguments from Croqtile function to the device functions, which matches the device parameters exactly. Note that in the device function, the **corresponding parameter type of the *spanned data* argument is simply the pointer of its _element type_**, where the shape information is dropped. For example, the `foo`'s argument `p` is `float*`, which corresponds the *spanned data* argument `input.data`. In Croqtile, you names the parameter as a **decayed** pointer of the *spanned data*. If using other form like `float p[]`, it will trigger failure in target compilation stage.

For some types like `f16`, and `bf16`, there may not be native target support of such types, it is possible to utilize `croqtile::f16` and `croqtile::bf16` to handle such types.

In short In this section, you learned the syntax to invoke device functions, including normal ones and those triggers function template instantiations. Programmers must code in cautious since no check is applied at transpilation time, which may result in more confusing error reporting since the check is applied at target compile time with generated functions.

2.14 Printing, Assertions, and Compile-Time Inspection

The language has its own print and assert forms, at compile time and at runtime. This section shows what each one reports and when to reach for it.

`print!` and `println!`: compile-time inspection

The bang variants (`print!`, `println!`) run at **compile time**, not at runtime. They print to the compiler's output and are useful for inspecting shapes, extents, and type information without launching a kernel:

```
__co__ void check_shapes(f32 [3, 2] b) {
  print!("shape of b: ");
  println!("b.span = ", b.span);
}
```

Listing 2.133: print and println : compile-time inspection

When you compile this file, the compiler emits:

```
shape of b: b.span = [3, 2]
```

Listing 2.134: print and println : compile-time inspection

String literals are concatenated (`"wor" "ld!"` becomes `"world!"`).

`print` and `println`: runtime device output

```
__co__ void inspect(s32 [4, 8] data) {
  foreach i in [data.span] {
    println("element ", i, " = ", data.at(i));
  }
}
```

Listing 2.135: print and println: runtime device output

Each call takes a comma-separated mix of string literals and expressions. The output order across threads is **nondeterministic**

Guarding output. For a specific check, guard the print with a condition:

```
parallel {px, py} by [8, 16] : block
  parallel {qx, qy} by [16, 16] : thread {
    // ... compute ...
    if (px == 0 && py == 0 && qx == 3 && qy == 5) {
      println("partial sum = ", accum);
    }
  }
```

Listing 2.136: print and println: runtime device output

Without the guard you get one line per thread — thousands of lines for a large grid.

`assert`: runtime invariant checks

Croqtile's `assert` builtin checks an invariant at runtime and aborts the kernel with a message if it fails:

```
assert(stage < MATMUL_STAGES, "stage index out of bounds");
```

Listing 2.137: assert: runtime invariant checks

On the device, this compiles to a `printf` of the message followed by an abort. Use assertions to catch index-out-of-range, null pointer, and other "this should never happen" conditions early.

2.15 Syntax Quick Reference

The constructs introduced so far are gathered here as a compact reference, for looking up a spelling or a meaning without re-reading the examples.

Checklist

By the end of this part of the guide, the new pieces of syntax should mean the following:

Syntax	Meaning
`dma.copy.async ...`	Start a non-blocking DMA transfer.
`wait f`	Wait until future `f` is ready to consume.
`future.data`	Access the moved tile.
`future.span`	Access the moved tile's shape.
`dma.transp<...>`	Permute dimensions during transfer.
`dma.pad<...>`	Pad during transfer with an explicit fill value.
`.zfill`	Zero-fill out-of-bounds positions in partial tiles.
`.swiz<N>`	Choose a shared-memory swizzle layout.
`chunkat(...)`	Select one equal tile from a regular tiling.
`subspan(...).at(...)`	Select a tile by explicit tile extents and tile coordinates.
`view(...).from(...)`	Select a tile by explicit extents and explicit element offsets.
`tile # i`	Compose tile and in-tile coordinates into a global coordinate.
`#tile`	Ask for the extent of the `tile` axis.

The next chapter builds on this by overlapping movement with compute. Once you can think of a kernel as a loop that repeatedly stages tiles and computes on staged tiles, double buffering and pipelining become natural extensions rather than new ideas.

Summary of New Syntax

Syntax	Meaning
`parallel p by N`	N-way parallelism, index `p` runs 0..N-1 concurrently
`parallel {px, py} by [M, N]`	Multi-dimensional parallel level (`p` and `q` correspond to one parallel level)
`parallel p by A, q by B`	Comma-separated parallel levels (`p` and `q` correspond to different parallel levels)
`parallel q by N : thread`	Map to threads within a block
`foreach index = {m, n, k} in [a, b, c]`	Named tuple destructuring in a `foreach`
`lhs.span(0)`	Extract one dimension of a tensor's shape
`p # m`	Compose outer tile index `p` with inner offset `m`

The next chapter replaces the scalar `.at()` arithmetic in the inner loop with **tensor-core** operations

Syntax	Meaning
`if (expr) { ... }`	Predicated execution — runtime conditional, divergent threads masked
`inthreads.async (condition)`	Structured concurrent region — compile-time thread partitioning
`shared event name[N]`	Declare N synchronization tokens
`trigger name`	Signal that a condition is met
`wait name`	Block until the corresponding `trigger` fires
`tile_id = tile_iter # block_id`	Compose indices for tile striping
`int total_tiles = expr`	Local integer variable
`yield`	Early return from `parallel` block

Syntax at a glance

Syntax	Meaning
`dma.copy.async src => dst`	Non-blocking copy (returns immediately)
`dma.any`	Placeholder future (no transfer in flight yet)
`swap(f0, f1)`	Exchange two future handles without copying data
`rotate(f0, f1, f2)`	Cycle three future handles
`with tile_k in N { ... }`	Scoped tile axis binding with extent N
`foreach tile_k(1:)`	Iterate starting from index 1
`__co__ auto fn(...)`	Return type inferred from `return` statement

Syntax at a glance

Where to go next

You can now read most `.co` files. Chapter 3 explains why the constructs are shaped this way and covers the advanced forms; Appendix A is a compact lookup table.

Chapter 3

Programming Guide

Chapter roadmap

This chapter organises the same language by function. It explains the structure of a Croqtile program, its type system, symbolic shapes, parallelism, tile selection, data movement, contraction and coordination, and the reasoning behind them. Complete kernels sit beside the feature they demonstrate, so each section also stands on its own.

3.1 Program Structure and Compilation Models

A Croqtile program is transpiled into host and device code. This section explains the resulting three-layer structure and the compilation models the language supports.

This reference covers every aspect of programming with Croqtile:

1. **Program structure**: How Croqtile programs are organized — the `__co__` function, host-side APIs, and C++ interoperability.
1. **Shaped data**: Croqtile's type system for multi-dimensional tensors — shapes, MDSpan, spanned data, symbolic dimensions, and dynamic shapes.
1. **Loop and parallelism**: SPMD parallelism, loop control, tiling, and iteration constructs.
1. **Data movement**: DMA statements, tiling with `chunkat`, TMA, swizzle, and advanced data movement patterns.
1. **C++ embeddings**: Input/output conventions, `__cpp__` inline blocks, macros, and preprocessor integration.
1. **Optimization patterns**: Tileflow optimizations, async DMA patterns, multi-buffering, and performance tuning techniques.

This section introduces the fundamental structure of *Croqtile* programs and their associated terminology.

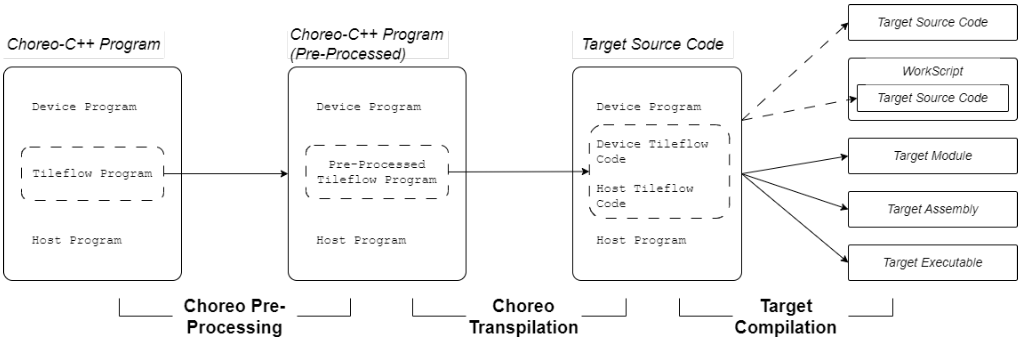
Construct a Croqtile Program

A typical *Croqtile* program is composed of multiple parts, depending on the target platform.

- The Device Program
- The Host Program
- The Tileflow Program

Transpilation and Compilation

The Croqtile compilation process typically involves three major steps: **Pre-processing**, **Transpilation**, and **Target Compilation**. To better understand how different parts of a Croqtile program work together, the full compilation workflow is illustrated below:


Choreo-C++ Program
Device Program
Tileflow Program
Host Program
Choreo-C++ Program (Pre-Processed)
Device Program
Pre-Processed Tileflow Program
Host Program
Target Source Code
Device Program
Device Tileflow Code
Host Tileflow Code
Host Program
Target Source Code
WorkScript
Target Source Code
Target Module
Target Assembly
Target Executable
Choreo Pre-Processing
Choreo Transpilation
Target Compilation

As shown in the figure, immediately after *pre-processing*, Croqtile *transpiles* (*source-to-source* compilation) the *tileflow program* into target code, leaving the user-provided *host program* and *device program* unchanged. The *tileflow program* is *transpiled* into host and device source code, which you refer to as *croqtile-generated code*. The compiler then combines the user-provided code and *croqtile-generated* code to perform the *target compilation* process. This process can generate various outputs, such as transpiled source code, workscripts, target modules, target assembly, and executables.

Thus, the Croqtile compiler functions as a end-to-end compiler, with the key step being the transpilation of the *tileflow program* into *croqtile-generated code*.

One notable feature of Croqtile compilation is its support for both:

- **Single Source Compilation Model**: Similar to , where the target compiler allows device and host programs to be in a single source file for target compilation.
- **Separate Source Compilation Model**: Similar to , where host and device code must be compiled separately.

The code shown above naturally supports the *Single Source Compilation Model*. However, to support the *Separate Source Compilation Model*, Croqtile requires wrapping the Device Program with the `__cok__` block, as shown below:

```
__cok__ {
  void device_function(...) { ... }
} // end of __cok__
__co__ void croqtile_function(...) { ... }
void foo() { ... }
```

Listing 3.1: Transpilation and Compilation

This is the code structure for a program. The `__cok__ {}` wrapper enables the Croqtile compiler to handle user-provided device code properly. It helps Croqtile separate device and host code from a single Croqtile source file for different compilation processes. Therefore, do not be surprised if you encounter `__cok__` in certain Croqtile code; it is necessary for integrating support for the *Separate Source Compilation Model*.

Device Program: Parallel Computation

The kernel is designed to operate on input data, process it in parallel, and generate the output.

You repeat the code as below:

```
__device__ void kernel(int * a, int * b, int * c, int n) {
  for (int i = 0; i < n; ++i) c[i] = a[i] + b[i];
}
```

Listing 3.2: Device Program: Parallel Computation

For targets that only support the _Separated Source Programming Model_, the code may need to be wrapped within a `__cok__ {}` block. The equivalent code is shown below:

```
__cok__ {
  extern "C" void kernel(int * a, int * b, int * c, int n) {
    for (int i = 0; i < n; ++i) c[i] = a[i] + b[i];
  }
} // end of __cok__
```

Listing 3.3: Device Program: Parallel Computation

Here, the `extern "C"` annotation replaces the `__device__` keyword used in target, as requires C-linkage for the device functions.

Programmers must be aware that the device program follows the *Single-Program-Multiple-Data (SPMD)* paradigm. However, unlike traditional programs, the device program does not manage data movement —whether between the host and device or across multiple storage levels within the device. Instead, the *tileflow program* orchestrates these tasks in a much simpler and safer manner.

In short You are now aware that a Croqtile program consists of three parts, with the *tileflow program* being the core. This part is transpiled into target source code during the compilation process. The call chain typically flows from the host to the tileflow program, and then from the tileflow program to the device code.

You may have noticed that Croqtile not only abstracts DMA operations into higher-level semantics but also combines iteration and tiling for ease of use. This makes Croqtile code concise and expressive. In the following chapters, you will delve deeper into Croqtile's syntax and semantics to explore its full potential.

Heterogeneity: Transpiling to program launch

For targets like , the `parallel-by` block not only specifies the number of parallel threads but also defines the boundary between host and device code, leveraging Croqtile's ability to manage heterogeneity. Specifically:

- Code **outside** the `parallel-by` block is transpiled into **host code**.
- Code **inside** the block is transpiled into **device code**, as illustrated below:

```
__co__ void foo(...) {
  // Generate host code
  // ...
  parallel p by 6 {
    // Generate device code
    // ...
  }
  // Generate host code
  // ...
}
```

Listing 3.4: Heterogeneity: Transpiling to program launch

For those familiar with , this is analogous to a occurring at the `parallel-by` statement. However, Croqtile abstracts these details with the unified SPMD parallelism model, allowing developers to focus on their algorithms without worrying about the underlying heterogeneity.

3.2 Syntax Design and the Type System

The type system has a small number of value families. This section explains how they were designed and what the storage and bounded qualifiers contribute.

Type System Aside: `mdspan` and `ituple`

Before you build the matmul, two features of Croqtile's type system are worth mentioning. You have already used them implicitly; now is a good time to name them.

`mdspan` — every tensor in Croqtile carries its shape as part of its type. When you write `s32 [128, 256] lhs`, the shape `[128, 256]` is not a runtime value — it is a compile-time property. The compiler uses it to verify that every `.at()`, `chunkat`, and `dma.copy` is dimensionally consistent. If you try `lhs.at(i, j, k)` on a 2D tensor, the compiler rejects it. The expression `output.at(px#qx, py#qy)` is verified to produce indices within the output's bounds.

The `span(i)` syntax extracts one dimension: `lhs.span(0)` is 128, `rhs.span(1)` is 256. The shorthand `lhs.span` copies the entire shape.

`ituple` — the `{px, py}` syntax in `parallel {px, py} by [8, 16]` introduces an `ituple`, a compile-time integer tuple. The compose operator `px#qx` combines two ituple elements into a global index. The extent operator `#p` extracts the upper bound of a parallel variable. These are all resolved at compile time — no runtime overhead.

For the full details, see Sections 2.4 and 2.5. In the examples that follow, the key takeaway is: Croqtile catches shape mismatches at compile time, not at runtime.

It aims to simplify the daily tasks of engineers responsible for crafting high-performance kernels. By leveraging Croqtile, the programming process becomes more accessible, allowing engineers to optimize these critical processes with greater ease and efficiency.

Embedding Croqtile within C++

Croqtile is the DSL code embedded within C++. The Croqtile compiler performs the source-to-source translation of the Croqtile function to be C++ code. It implicitly includes "croqtile.h", which bridges the translated code to work properly with other C++ code. The below code snippet showcases an example.

```
// some C++ code
__co__ void croqtile_function() {
  // croqtile code
}
// another C++ code
void foo() {
  croqtile_function();
}
```

Listing 3.5: Embedding Croqtile within C

Note, a Croqtile function is prefixed with the "\___co\___" keyword. All code in the function scope is translated by the Croqtile compiler. Other C++ code calls the Croqtile function by its name as declared.

In addition, certain Croqtile-specific types are introduced to ensure consistency between Croqtile function arguments and their respective callers. Considering that there are some important programming elements missing, you will defer the introduction of such details until the data types are explained.

Variables and Types

Croqtile introduces four fundamental type categories: scalar-type, spanned-type, integer-tuple-type (ituple-type) and bounded-type. These types serve for different purposes.

- **Scalar Type**. It is designed to fullfil the requirement of program control. It consist of *Integer Type* and *Boolean Type*.
- **Spanned Type**. It represents the data type (normally the tensor) for computation. Apart from referencing the raw data, a *spanned type* also associates data with a shape representing by multi-dimensional ranges. The 'multi-dimensional ranges' is named 'mdspan'. It is useful for tiling, etc, which is introduced soon.
- **Integer Tuple (I-Tuple) Type**. It represents a group of integer values. A common usage of *i-tuple* is to introduce the bound of multi-dimensional data.
- **Bounded (Integer/ITuple) Type**. This is the special type that is used to simplify data (sub-zone) reference. And it works with loop construct to iteratively process data.

Among the four type categories, *scalar* and *ituple* could be well accepted consider it maps to elements of existing general purpose programming languages. However, the *spanned type*, and *bounded type* are specific to the domain. The following sections will show the detail.

Spanned Types

A spanned type is a **Composite Type**. It consists of a **fundamental type**, and a **multi-dimensional-span(mdspan) type**. Neither the *fundamental type* nor the *mdspan* is a *complete type*. It implies that neither of them can type a data with storage. In Croqtile, you name them as *partial types*.

The reason for making the type be composition of two is to specifically manipulate *mdspan* alone. This is crucial in ML scenarios where data are always organized as multi-dimensional. It is very common for the programmers to tile the data into smaller blocks/chunks, and move them across storage hierarchy. Therefore, manipulating the _shape_ (represented by _mdspan_) of multi-dimensional data plays a vital role.

In the following sections, this section shows how to define such partial types and how to compose partial types into a complete type for declaring/defining multi-dimensional data purpose.

The Partial Type: mdspan

Unlike most type systems, *mdspan*, comes as an partial entity for typing. Here you claims it as *partial* because Croqtile program is unable to define data as a *mdspan* type. However, *mdspan* itself can be defined solely. Such design is based on the oberservation that loop tiling/blocking cares nothing but the shape of multiple dimension data. Therefore, Croqtile allows programmers to manipulate *mdspan* regardless the fundanmental type it associated with, to ease the programming about shapes.

Meanwhile, croqtile compiler could apply type checking on *mdspan* to reveal code errors as early (ahead of execution, when applies) as possble.

In Croqtile, a mdspan is braced by '[' and ']'. The upper-bound of dimensions inside are separated by ','. That is,

```
mdspan sp : [7, 8];           // defined a mdspan of 2-dimensions
mdspan<4> mds : [a, 3, 4, 1]; // 'a' is an existing integer
d : [c, 4, 28];               // 'd' is not explicit annotated. Type is deduced.
```

Listing 3.6: The Partial Type: mdspan

As illustrated, a 'mdspan' can be explicitly defined with *mdspan* partial-type keyword as annotated, optionally with a '<' '>' enclosed total dimension value. It is also possible to define a mdspan without type annotation. As shown in definition of 'd' above. (Note: 'mds' and 'd' are of the **dependent type**, considering the type depends on the evaluation of 'a' and 'b'. Programers may provides 'a' and 'b' with runtime values. However, in this way, some runtime check are paid consequently)

For C++ programmers, you may think a mdspan as the trait of a multiple dimensional array, where it describes a multi-level-range. For example, 'sp' in the above example defines two-level of ranges, ranging from 0 to 6, and 0 to 7 separately.

When a mdspan is defined, it is possible to get the integer dimension value using operator '()' over the mdspan.

```
sp : [7, 8];
int b = sp(0) + sp(1);    // 'b' equals to 15 (7 + 8)
```

Listing 3.7: The Partial Type: mdspan

In the above example, the expression 'sp(0)' reasons about the first dimension value of mdspan 'sp'. The dimension values are in essence integer values, which are suitable for integer arithmetics.

As a conseqeunce, you may derive a mdspan from existing one as the below code snippet:

```
sp : [6, 8];
spn : [1, sp(0)/2, sp(1)/4]; // define a new mdspan from the existing one.
```

Listing 3.8: The Partial Type: mdspan

With such a facility, it is very convinient to apply tiling over multiple-dimensional spans in your Croqtile code. Considering that tiling operation is always required in constructing the high-performance kernels, Croqtile has provided syntax suger to make the work even easier:

```
sp : [6, 8];
spn : sp [1, (0)/2, (1)/4];  // spn is defined as [1, 3, 2]
```

Listing 3.9: The Partial Type: mdspan

This code works the same way as the previous one, but obviously in a much simpler syntax. This follows one of Croqtile's design philosophy — To enable functionalities with minimal code whenever possible.

Note, a mdspan can only be defined. No modification to an existing mdspan is allowed. Further, a mdspan can only be defined once.

Except for defining a mdspan in the above *dimension-wise* way, Croqtile also support to define mdspan definition with other methods. You will introduce such methods when *i-tuple* is revealed.

Bounded Types

Bounded types consists of **Bounded Scalar** and **Bounded ITuple**. Bounded Scalar takes a range of [0, ub), where 'ub' represents its upper bound. Therefore, if an integer 'p' is set as bounded, it should also be associated with a specific upper bound. To establish such associations, programmers must code explicitly inside the *Control Structures* of 'parallel-by' and 'with-in', which will be introduced later.

Similarly, since *ITuple* is a group of *Integer*s, it can also be associated with a group of bounds. Specifically, in Croqtile, the *Bounded ITuple* is associated with a *mdspan* value, where a group of upper bounds are settled. In later sections, you shall illustrate the detailed syntax.

In short Croqtile introduces a novel approach to SPMD programming. It favors C++-style coding and is embedded within C++. However, its primary focus is on alleviating the burden of low-level programming details, particularly those related to data manipulation across various memory layers through DMA operations. At times, it is also referred to as the dataflow programming DSL. You developed this tool to support the daily task of constructing high-performance kernels. Your aim is to enable programmers to focus less on the intricacies of language construction and more on higher-level conceptual thinking.

You wish you find it functions as expected. And you are looking forward to any feedback for continuous improvement.

Group of Ranges and Group of Integers

In Croqtile, *mdspan* can be used to represent dimensional shapes. However, as introduced, *mdspan* stands for **Multi-Dimensional Span**. The elements inside are *Dimensional Spans*, which imply ranges of values.

For example, an *mdspan* `[7, 14]` represents a group of two ranges `[0, 7), [0, 14)`. Therefore, an *mdspan* does not represent a list of integer values. However, grouping integer values is useful in certain scenarios:

- When considering the delta of two mdspans, multiple integers are needed. In high-level semantics, the delta may reflect padding differences when adjusting shape dimensions.
- When tiling a shape, the multiple tiling factors are integers, not ranges.

These situations motivate Croqtile to have a type for grouping integer values.

Look-Ahead: Bounded I-Tuples

I-tuples are not frequently used in real Croqtile code. However, their variants, **bounded i-tuple**s, which bind an *i-tuple* with an *mdspan* to indicate its possible range, are essential for constructing Croqtile loops. You will cover *bounded i-tuples* in more detail in later chapters.

Spanned: Data and Buffers

The primary focus of a Tileflow program is to manipulate large datasets by moving them around. In some terminology, the input and output of a Croqtile function are classified as "input data" and "output data," respectively. Meanwhile, any other storage is referred to as "buffers." This conceptually distinguishes between "external" and "internal" memory from a function's perspective. However, both "data" and "buffer" refer to storage positions. In Croqtile, both are typed as **spanned**, reflecting that they are data/buffers associated with an *mdspan*-represented shape. And in the following context, Croqtile calls it *spanned data*, or simply *spanned*.

Buffer Lifetime Management

The lifetime of storage allocated inside a *croqtile function* is managed by the Croqtile compiler for efficient use. The compiler attempts to reuse buffers as much as possible if their lifetimes do not overlap. Therefore users of Croqtile are not necessary to do buffer management.

Bounded Variable

In a `parallel-by` block, the variables defined after the `parallel` keyword are referred to as **Bounded Variables**. Specifically:

- If the variable is an *integer*, it is called a **bounded integer**.
- If the variable is an *ituple*, it is called a **bounded ituple**.

This terminology indicates that the variable not only has a value but also has defined bounds. For example, in the statement `parallel p by 6`, `p` is a *bounded integer* with a bound `[0, 6)`, where `6` is the exclusive upper bound.

The concept of bounded variables will be explored further in later sections, where their usage will be detailed.

Values of Bounded Variables

Understanding the values associated with a bounded variable is crucial. A bounded variable has two key values:

- **Current Value**:
  - Within a `foreach` statement, the current value is determined by the loop iteration, as the bounded variable serves as the iteration variable.
  - Outside of a `foreach` statement, the bounded variable always has a value of zero.
- **Upper-Bound Value**: This is specified in the `with-in` or `parallel-by` statements.

Programmers may encounter issues when using the current value of a bounded variable outside of a `foreach` statement, particularly after the loop has completed. By definition, the current value of a bounded variable is immutable except within a `foreach` loop. The following code illustrates this:

```
with x in 6 {
  // x's current value is 0
  foreach x {
    // x's current value is either 0, 1, 2, ..., 5
  }
  // x's current value is 0, NOT 6
}
```

Listing 3.10: Values of Bounded Variables

Memory in Croqtile

In Croqtile, the memory system is designed to handle large datasets effectively, which is crucial for high-performance computing scenarios, especially in machine learning and scientific computing.

Spanned Types in Croqtile

A **spanned type** is a composite type that includes both the fundamental data type (such as `f32`, `s32`, or `f16`) and a **multi-dimensional span (mdspan)** that describes the shape and organization of the data. The spanned type allows Croqtile to define large arrays or matrices with an efficient layout, while also enabling the programmer to manipulate the shape (i.e., dimensions) independently of the data type. This flexibility is essential for managing multi-dimensional datasets that are frequently encountered in parallel computation and machine learning workloads.

However, the spanned type by itself does not allocate storage or memory for the data. Instead, it serves as a blueprint for defining how the data will be organized in memory, allowing for runtime flexibility and optimizations.

Storage Qualifiers in Croqtile

In Croqtile, **storage qualifiers** annotate the memory allocation and specify which memory region the data will reside in. Croqtile defines three primary storage qualifiers:

3.3 Symbolic Shapes, Shape Evaluation, and Value Numbering

Shape dimensions may be symbolic rather than literal. This section explains how the compiler evaluates them, how value numbering works, and what safety that buys.

First-Class Citizen: Shape

Croqtile's primary function is to manage data movements, which are crucial for efficiently organizing and processing large datasets, especially in machine learning and high-performance computing scenarios. However, most C++ programming environments handle data in a casual manner —either as flat (pointers) or hierarchical structures (arrays), without a native representation of associated shapes.

In contrast, Croqtile enforces code safety and simplifies the programming of shaped data by requiring that **any data declared or used must be associated with a shape**. This motivates Croqtile to treat shape as a first-class citizen.

Evaluation of *mdspan*

So far, you have seen mdspan with constant values, which is sufficient for many scenarios, as high-performance device kernels often require fine-tuning based on fixed input data shapes. In these cases, the `mdspans` are evaluated at compile-time, meaning their values do not incur extra execution time or storage overhead.

However, in some scenarios, a runtime shape (where some dimensions are determined at execution) is required for building the kernel. Croqtile supports this with **Symbolic Dimensions** for `mdspan`, which will be introduced later. In such scenarios, runtime evaluation of dimension values may be necessary. Fortunately, Croqtile manages this evaluation immediately after entering a croqtile function in the Croqtile-generated host code, resulting in negligible startup cost. **Therefore, programmers can ignore overheads related to __mdspan__ normally**.

Evaluation of Integers and I-Tuples

Similar to mdspan, both *integer*s and *i-tuple*s incur minimal runtime cost. Their values are evaluated at compile time whenever possible, so programmers can generally ignore their cost.

n this section, you will learn about the *anonymous dimension* and *symbolic dimension* in Croqtile and how they support dynamic shapes required by certain systems.

Fixed and Dynamic Shape

Data with fixed shapes is commonly used in high-performance computing kernels, allowing for aggressive optimization with known dimensions. However, certain scenarios require handling shapes with runtime dimensions, known as dynamic shapes. This necessitates kernel generality to manage inputs of varying shapes, a feature highlighted by many machine learning frameworks.

Improved Code Safety

The reason for improved code safety of using *symbolic dimension* is that more comprehensive code checks can be applied by Croqtile compiler. For example, consider an *anonymous dimension* version of the `Matmul` function:

```
__co__ auto Matmul(s32 [?, ?] lhs, s32 [?, ?] rhs) {
  int tile_m = 32, tile_m = 8, tile_k = 16;
  s32 [lhs.span / {tile_m, tile_k}] tiled_lhs;  // 'mdspan' and 'ituple': rank
    must be same
  s32 [rhs.span / {tile_k, tile_n}] tiled_rhs;  // 'tiled_rhs' can not have a
    zero dimension
  // ...
}
```

Listing 3.11: Improved Code Safety

Croqtile performs both compile-time and runtime checks to ensure safety:

- **Compile-time Checks**: Croqtile verifies rank consistency. In this example, `lhs.span` has a ranked of 2, matching the rank of `{tile_m, tile_k}`, so there are no issues.
- **Runtime Checks**: Croqtile generates code to validate dimensions at runtime. For instance, in the declaration of `tiled_lhs`, its shape must not have a dimension of `0`, which would result in an invalid zero-sized buffer. Croqtile's *transpilation* process generates the following *target host code*:

```
void __croqtile_transpiled_Matmul(croqtile::span_view<2, croqtile::s32> lhs,
                                  croqtile::span_view<2, croqtile::s32> rhs) {
  croqtile_assert(lhs.shape()[0] / 32 > 0,
                  "the 0 dimension of 'lhs' may result in spanned data "
                  "with a dimension value of 0.");
  // ...
}
```

Listing 3.12: Improved Code Safety

Here, the `croqtile_abort` function aborts program execution if the condition is not met. This early check helps Croqtile detect issues promptly.

For the *symbolic dimension* version, Croqtile compiler generates additional checks in the *target host code*:

```
void __croqtile_transpiled_Matmul(croqtile::span_view<2, croqtile::s32> lhs,
                                  croqtile::span_view<2, croqtile::s32> rhs) {
  // additional check happens only when using symbolic dimensions
  croqtile_assert(lhs.shape()[1] == rhs.shape()[0],
                 "dimension 'K' is not consistent.");
  croqtile_assert(lhs.shape()[0] / 32 > 0,
                 "the 0 dimension of 'lhs' may result in spanned data "
                 "with a dimension value of 0.");
  // ...
}
```

Listing 3.13: Improved Code Safety

In addition to the checks performed in the *anonymous dimension* version, the *symbolic dimension* version verifies the consistency of dimensions between `lhs` and `rhs`, resulting in safer code.

Therefore, it is recommended that programmers use symbolic dimensions not only for ease of use but also to enhance code safety.

Cost and Limitations

To support dynamic shapes, Croqtile compile has to be conservative at compile time. This limitation restricts certain compile-time checks and necessitates some checks to be performed at runtime instead. Although the implementation aims to minimize overhead by placing these checks in the host code, they **remain more conservative compared to code with fixed-sized shapes**. Programmers should be mindful of this situation and may need to add user-level assertions to enhance code safety when necessary.

A significant limitation of both *anonymous dimension* and *symbolic dimension* is that they can **only be applied to the *spanned data* in parameter lists**. Dimensions of buffers declared within Croqtile functions cannot be made dynamic. For example, the following code will result in a compile-time error:

```
__co__ void foo() { f32 [M] d;} // compile-time error: can not applied to the
    spanned buffer
```

Listing 3.14: Cost and Limitations

In short In this section, you discussed *anonymous dimension* and *symbolic dimension* in Croqtile, which support dynamic shapes required in certain scenarios. Compared to *anonymous dimension*, using *symbolic dimension* enhance code safety through additional compile-time checks, making them the recommended choice for ease of use and improved safety.

a. Improved Readability and Clarity

Using symbolic dimensions like `M` and `N` explicitly in the shape definition makes the program **easier to read** and understand. Instead of seeing a `?` or `-1` and wondering what the dimension is supposed to represent, the reader can immediately grasp that `M` and `N` are variables that depend on runtime values. This improves **maintainability** and reduces the cognitive load for developers who are reading and modifying the code.

For example, if a program uses `?` for dynamic dimensions, it's unclear whether it represents an uninitialized value, a wildcard dimension, or something else. This can lead to confusion when trying to reason about the program. In contrast, symbolic dimensions are explicit and show exactly how the shape is determined:

```
// Using symbolic dimensions
mdspan sp : [M, N];
mdspan spn : sp [1, M / 2, N / 4];
// Clear how the new shape is derived
// Using '?' or '-1'
mdspan sp : [? , ?];
```

Listing 3.15: a. Improved Readability and Clarity

The latter version is ambiguous and lacks the clarity that symbolic dimensions provide.

b. Type Safety and Compile-Time Checking

One of the major benefits of symbolic dimensions in Croqtile is that they are **type-safe** and **checked at compile-time**. Croqtile allows you to declare symbolic dimensions and even manipulate them algebraically (e.g., `M / 2`, `N / 4`), ensuring that they adhere to the expected types and constraints. This eliminates the need for runtime checks, which can often lead to errors or performance penalties.

For example, using `?` or `-1` as dynamic dimensions in other systems often leads to runtime checks for consistency, such as verifying that the dimensions are properly initialized before use. These checks can introduce overhead and make the program harder to optimize. In contrast, symbolic dimensions allow Croqtile to **check constraints at compile-time**, ensuring that shape expressions are valid before execution:

```
mdspan sp : [M, N];
// M and N are symbolic dimensions
mdspan spn : sp [1, M / 2, N / 4];
// Compiler ensures validity of shape expressions'
```

Listing 3.16: b. Type Safety and Compile-Time Checking

This ensures type consistency and reduces the potential for errors during runtime.

c. Expressiveness and Flexibility

Symbolic dimensions enable a **richer** set of expressions for describing data shapes. Instead of using arbitrary placeholders like `?` or `-1`, which are limited to basic dimensionality, symbolic dimensions can be **constrained** using complex arithmetic and relationships. For example, you can directly express how one dimension depends on another, or how it is scaled or tiled, in a much more natural way.

In contrast, using `?` or `-1` often leads to brittle code, where developers need to manually calculate or infer the actual size at runtime, introducing potential errors and making the code harder to reason about.

For example, you can easily express relationships between dimensions using symbolic variables:

```
mdspan sp : [M, N]; // Symbolic dimensions
mdspan spn : sp [1, M / 2, N / 4]; // Derived shape based on symbolic
    expressions
```

Listing 3.17: c. Expressiveness and Flexibility

If you were using `?` or `-1`, expressing the same relationships would require convoluted workarounds or additional runtime logic.

d. Compile-Time Optimization

One of the significant benefits of symbolic dimensions is that they can help with **compile-time optimizations**. Since Croqtile allows symbolic dimensions to be evaluated at compile time, it can potentially optimize memory layouts and kernel execution plans based on these dimensions. This can result in more efficient code generation and better performance compared to using `?` or `-1`, which may require runtime evaluation and dynamic memory allocation, hindering optimization opportunities.

Introduction to Croqtile Value Numbering

In Croqtile, the value numbering process serves to facilitate Shape Inference by either directly inferring shapes known at compile time or generating shape computation expressions.

While many value numbering approaches focus on eliminating redundant computations (expressions that consistently produce the same value), Croqtile's value numbering aims to establish a progressive method for simplifying the Croqtile domain-specific construct 'mdspan'. In this context, the application of constant propagation, algebraic simplification, and other techniques becomes more essential.

In terms of the implementation method, Croqtile utilizes a simple **scope-based** value numbering approach, an extension of the local value numbering method, complemented by a lexical-scoped value number table[1]. Unlike other value numbering methods, Croqtile does not necessitate the construction of basic blocks, pruned-SSA, or a compiler backend, as it applies value numbering directly onto the Abstract Syntax Tree (AST). However, to facilitate this approach, certain restrictions must be imposed on Croqtile's syntax rules, particularly to prevent value mutation.

Quick Example

The value numbering process assigns a value number to all entities containing a list of values, including for mdspan and ituple. For example,

```
mdspan a : {1, b, c + 3}
```

Listing 3.18: Quick Example

The value numbering process would produce values including:

```
   VN #0:  const_1
   VN #1:  b
   VN #2:  c
   VN #3:  const_3
   VN #4:  +:#2:#3
   VN #5:  #0,#1,#4
```

Listing 3.19: Quick Example

In this illustration, the number after "VN #" is the value number of an expression. I either represents a constant integer value (known at compile time), a symbolic value (like variable 'a', 'b'), or composition of other value numbers. Here, value number #0 and #3 represents a reference to constant value, where #1, and #2 represents reference to the integer-typed variable "b" and "c". #4 represents the value of an addition expression, which gives the sum of #2 and #3. That is the expression of 'c + 3' when dereferencing all the value numbers. And lastly, value number #5 represents the list of values defined by mdspan 'a' symbolically. With such a definition, you may translate the indexing operations, e.g. "a(0)", results in value number #0 and thus a constant of 1.

From Expressions to Value Numbers

In current implement, the value numbering process in a syntax-directed way. It traverse the Abstract Syntax Tree nodes and generate value numbers when the node contains an expression.

The method of generating value numbers is simple. For each expression, it takes the expression as the input, and returns corresponding value number.

More specifically, it:

- Generates a signature from an expression.
- If the signature has already existed, return the associated value number by looking up the value number table.
- Or else, it associates the signature with a new value number in the value number table, and return it.

Notably, for simplicity, current signature is an string of expression. This can be improved by hashing later (like most value numbering process does).

In addition, to handle the symbol definition and symbol reference, it follows the below rules:

- When an expression is to initialize a symbol (variable or partial type), the symbol is assign the same value number of the expression. More specifically, it associate the signature of the *scoped* symbol name with the value number in the value number table.
- When an expression references a symbol, it looks up its corresponding signature in the value number table, and return it.

Note the symbol must use its *scoped name* for signature generation, since identical names are allowed to be defined in different scope.

Simplifications

Every time the value numbering process generates a signature, it tries to simplify the expression. For example,

```
    int a = 0 + 1;
```

Listing 3.20: Simplifications

Given the simple integer expression, the value numbering process could generates signatures and associated value number as illustrated:

```
VN #0:  const_3
VN #1:  const_8
VN #2:  +:#0:#1
Alias a -> #2
```

Listing 3.21: Simplifications

Here, without optimizations, two constant values #0 and #1, and a new value number #2, which is an addition of #0 and #1 are generated. #2 has the signature of "+:#0:#1". However, considering that addition of constants can be evaluated at compile time, simplification could be performed. The below output showcases the optimized value numbering process:

```
VN #0:  const_3
VN #1:  const_8
VN #2:  const_11
Alias a -> #2
```

Listing 3.22: Simplifications

The signature of #2 now represents a constant, which implies the algebraic simplification is applied successfully.

From Value Numbers to Expression

After value numbering and simplification, each expression is associated with its simplified value number. In Croqtile, this is important step for the shape inference process of the mdspan type.

The *mdspan* type is a partial type that defines count of fundamental-typed elements. And in programming, together with a fundamental type, e.g, it decides the memory size an *spanned* object takes.

The shape detail of *mdspan* could be runtime decided. However, in many scenario, the shape is possible to be fully evaluated to fixed numbers at compile time. In such a scenario, static memory allocation can also be triggered at compile time to best utilize the system memory. Or else, the it requires to generate code to calculate the shapes.

Consequently, except the constant values, it is required to generate expressions back from value numbers.

Lexical Scopes and Restriction

In Croqtile, you applying the value numbering *GLOBALLY* but on top of AST. As it is a syntax-directed method without basic blocks and thus lack data flow analysis, it is impossible to inference a shape with varying dimensions.

Limitations

More specifically, it has to set two limitations:

1. Any mdspan can not have element with bounded integer type, or produced from bounded ituple.
2. Simple integer type and ituple type can not be re-defined.

Reference

[1] Briggs P, Cooper K D, Simpson L T. Value numbering[J]. Software: Practice and Experience, 1997, 27(6): 701-724.

3.4 Parallelism, Space Specifiers, and Masking

Parallelism in Croqtile is declared in layers. This section covers space specifiers, masking and the language rules that keep multi-level iteration well defined.

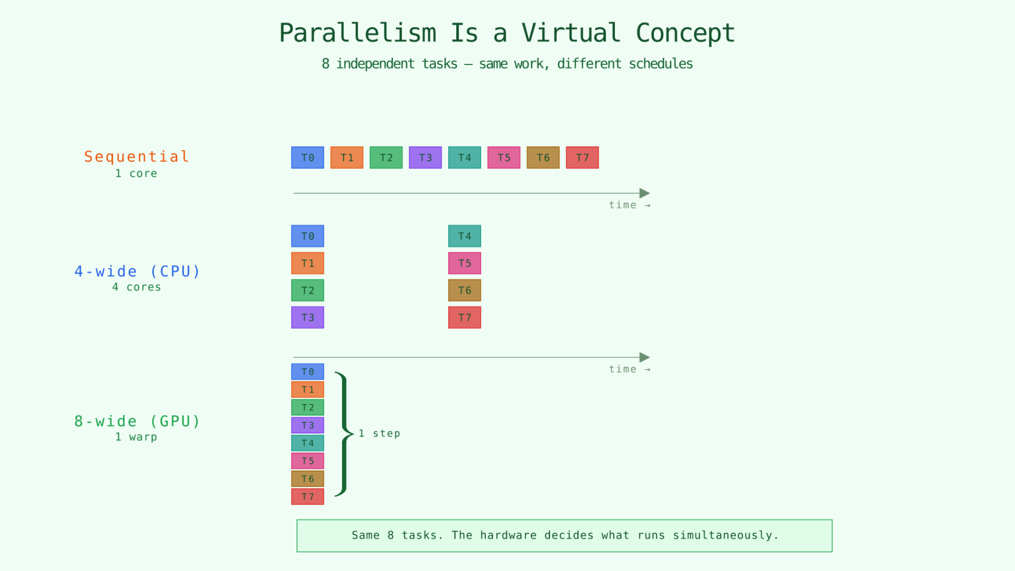
Parallelism: How Croqtile Represents Parallel Work

The previous two chapters kept things simple: `parallel {i, j} by [4, 8]` created 32 instances, each handling one element, and you never asked where those instances actually ran. That was fine for element-wise addition and for understanding data movement — the compiler picked sensible defaults.

How You Think About Parallel Tasks

Before you touch any Croqtile syntax, let's talk about what "parallel" actually means.

Suppose you have eight independent tasks — eight tiles to add, eight rows to process, whatever. You can describe all eight at once and call them "parallel." But that statement says nothing about how many actually run at the same time. On a single CPU core, they execute one after another. On a 4-core CPU, maybe four run simultaneously.


Parallelism Is a Virtual Concept
8 independent tasks — same work, different schedules
Sequential
1 core
T0
T1
T2
T3
T4
T5
T6
T7
time →
4-wide (CPU)
4 cores
time →
8-wide (GPU)
1 warp
1 step
Same 8 tasks. The hardware decides what runs simultaneously.

The tasks are identical

The point is that parallelism is a **virtual** concept.

If you want to change the tiling, you also have to change the launch configuration. The logical and physical are tangled together.

Croqtile untangles them.

: `group-4` — cooperative group

```
parallel g by 2 : group-4
```

Listing 3.23: : group-4 — cooperative group

Section 3.7 puts this to work with tensor-core operations.

Matrix Multiply: Putting It Together

With `parallel`, `foreach`, `dma.copy`, `chunkat`, `#`, and space specifiers, a tiled matrix multiply is within reach.

The plan: multiply a `[128, 256]` matrix by a `[256, 256]` matrix to get a `[128, 256]` result.

Control Structures

Croqtile has significant difference with C++ on parallelization, loop, etc.

Sequentialism and Parallelism

Repetitive tasks are common in programming.

Currently, the most widely used parallelization paradigm is **Single Program, Multiple Data (SPMD)**. In this paradigm, code is written once but executed as multiple instances simultaneously. Each instance operates independently unless synchronization is explicitly implemented. Croqtile adopts this paradigm and abstracts parallelism using the **parallel-by** block.

Looping, on the other hand, is a more straightforward concept and is the most common method for writing sequential code. In Croqtile, programmers can construct loops using the **with-in + foreach** block. This structure supports *do-while* style iterations, which are particularly useful in scenarios involving multiple buffers, given Croqtile's primary focus on orchestrating data movement.

In the following sections, this section examines the details of constructing loops and implementing parallelism in Croqtile.

This section will guide you through using the `parallel-by` block to implement parallel code in Croqtile.

Parallel Execution Block

In Croqtile, the code run in parallel are placed into the *parallel execution block*, which showcasing there are multiple instances of the same code that are executed simultaneously.

In short In this section, you explored Croqtile's approach to parallelism in heterogeneous computing environments, focusing on the `parallel-by` block syntax and the management of multiple and sub-level parallelism.

However, while parallelism enhances efficiency, loops remain essential. In the next section, this section introduces how to build loops in Croqtile.

Parallelization and Masking

It is usually necessary to mask off some *parallel thread* in programming SPMD code. Programmers may invoke fixed number of parallel threads where the tasks can not evenly divided. Thus masking is necessary to close this gap: it makes only parts of the threads to work, while leaving the other threads wait the working threads..

In Croqtile code, the code may be implicitly masked, or explicit masked. Let us dive into the detail.

Explicit Masking/Condition

Thread masking is a concept when programs have a full view of all instances of SPMD code. However, from the SPMD coding practise, which program is limited in single thread view, the thread masking appears as a boolean expression which typically is a comparison result related to thread identifiers.

In the examples that follow, Croqtile experimentally supports explicit masking of the `foreach` block, which mask off loop code as a whole. For example:

```
__co__  void foo() {
  parallel p by 6 {
    with q in [2] {
      foreach q if (p == 0) {
        // this iteration only work for parallel thread 0
      }
    }
  }
}
```

Listing 3.24: Explicit Masking/Condition

Implicit Masking

There are some code structures in Croqtile that implies implicit masking. One typical situation is the multi-level `parallel-by` block. The below code showcases an example:

```
__co__  void foo() {
  parallel p by 6 {
    // parallel-level-0:
    //   code here is as if implicitly masked 'if (q == 0)'
    parallel q by 2 {
      // parallel-level-1:
      //   code here is for all 'q' threads
    }
    // parallel-level-0:
    //   code here is as if implicitly masked 'if (q == 0)'
  }
}
```

Listing 3.25: Implicit Masking

In this example, there exists two-level of parallelization. The code inside `parallel p by 6` block but outside `parallel q by 2` blocks is only for *parallel-level-0*. However, for target like , it could invoke 2 parallel threads in practise to execute either code inside *parallel-level-0* or *parallel-level-1*. Therefore, the *parallel-level-0*-only part is as if guarded with a C++ block `if (q == 0)`.

Conditional `foreach` Block Since It is possible to guard the `foreach` loop with conditions. One direct

Experimental: Optional *Where-Binding* for `within`

You can append a *where-clause* to impose **Where-Binding** constraints among different bounded variables defined in a `with-in` statement. This allows different bounded variables to be treated as aliases for each other. Below is an example:

```
with {m, n} in [M, N], {n_p, k} in [N_P, K]
where n_p <-> n {
  // matmul implements with m, n, K. ’n_p’ always refers to the same value as ’n’.
}
```

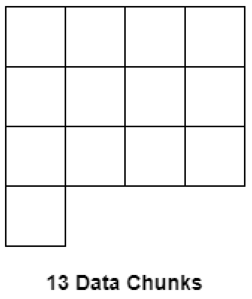
Listing 3.26: Experimental: Optional Where-Binding for within

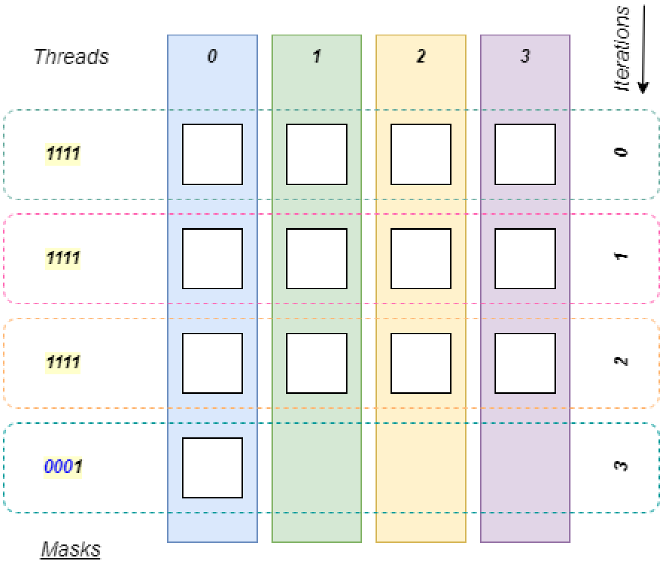
Syntactically, the `<->` operation establishes the *where-binding* between two bounded variables. Within the `with-in` block, any reference to `n_p` refers to `n`, and vice versa.

While Croqtile primarily targets SPMD (Single-Program-Multiple-Data) programming, it also supports MPMD (Multiple-Program-Multiple-Data) execution through thread masking. In this section, you demonstrate how to write divergent code using `inthreads` block in Croqtile.

Masking in SPMD

It is typically necessary to mask off some *parallel threads* in SPMD programming. For example, programmers often divide data into chunks for parallel or iterative processing. However, there may be a remainder that cannot be evenly distributed across all threads.





The figure aboves showcases a tiling of 13 data chunks, where 4 parallel threads are invoked for the processing of each chunk. In this case, 4 iterations are required to process all the data chunks. However, for the last iteration, only *thread-0* is required to work. You may observe the **thread mask** for each iteration are `1111`, except for the last iteration (`1000`). The thread mask is a combination of boolean bits, where `1` stands for **active**, and `0` stands for **inactive**. So in this case, only *thread-0* is set to be *active* to avoid unnecessary data chunk processing in the last iteration.

Uniform and Divergence

Masking is an implementation-level concept. From the perspective of the SPMD programming, you typically claim the code that processes data chunks **diverges** because not all the thread instances execute the same code due to masking. And programmers should be aware of that their SPMD code is a combination of:

- **Uniform Code**: *All threads* execute the same code path.

- **Divergent Code**: Only the *masked threads* execute the *divergent* code path.

It is necessary to have the *divergent* code in SPMD code to handle different real-world requirements. And from a perspective of different programming paradigm, applying *divergent* code in SPMD programs can even mimic MPMD (Multiple Program Multiple Data) behavior. In MPMD, different threads execute different code. Therefore, *divergence* can extend SPMD to MPMD scope conceptually.

In Croqtile, you are able to create divergent code via `inthreads` block. Let us dive into the detail.

Masking for Multi-Levels

Thread masking can be applied at different parallel levels. The following example demonstrates this:

```
__co__  void foo() {
  parallel p by 3, q by 4 {
    inthreads (p < 2 && q == 0) {
      // divergent code for thread (0, 0), (1, 0)
      // thread-mask: 011-0001
    }
    inthreads (q == 1) {
      // divergent code for thread (0, 1), (1, 1), (2, 1)
      // thread-mask: 111-0010
    }
  }
}
```

Listing 3.27: Masking for Multi-Levels

This code uses two levels of parallelization:

- Outer level (`p`): Parallel count of 3
- Inner level (`q`): Parallel count of 4

The first `inthreads` predicate (`p < 2 && q == 0`) restricts execution to:

- Outer parallel IDs: `0, 1` (excluding 2)
- Inner parallel ID: `0` only

The thread mask (`011-0001`) reflects these constraints, where the two levels are separated by a hyphen (-).

The second predicate (`q == 1`) imposes no restriction on the outer level (`p`), allowing all outer IDs (`0, 1, 2`) while restricting the inner level to `q == 1`.

In short In this section, you examined the motivation for thread masking, which enables divergent code execution for MPMD-style programming within Croqtile's SPMD paradigm. Developers can use either explicit thread masking via `inthreads` blocks or implicit control through `parallel-by` constructs to restrict execution to specific threads.

While thread masking provides flexibility, it introduces thread asynchrony that complicates coordination. The next section will explore advanced techniques for managing these synchronization challenges.

3.4.1 Worked Example: Tiled Matrix Multiplication

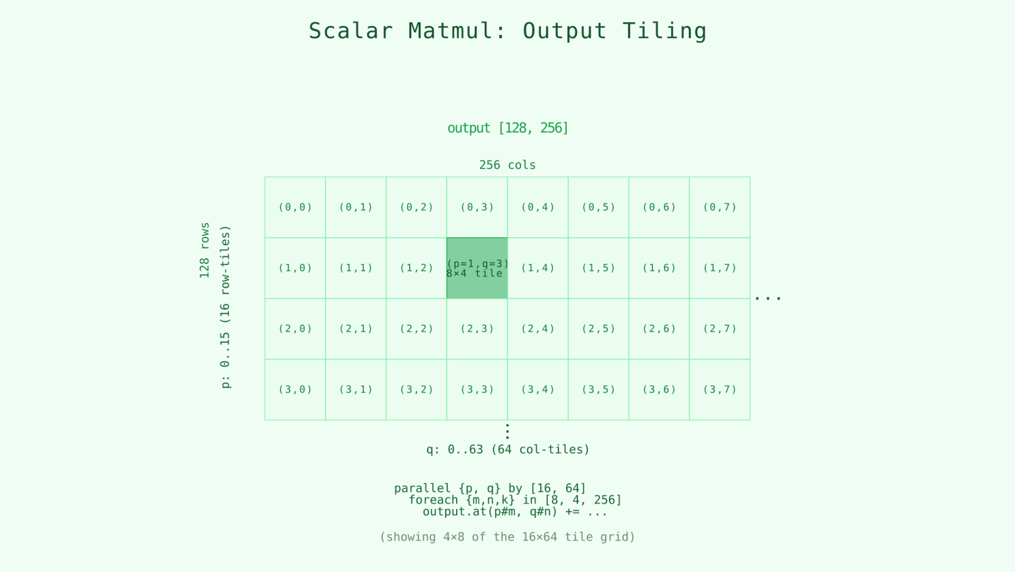
A tiled matrix multiplication puts the parallelism constructs to work. This section walks through the kernel and the host code that launches it.

Scalar Matmul (No DMA)

Start with just `parallel` and `.at()`

```
__co__ s32 [128, 256] matmul(s32 [128, 256] lhs, s32 [256, 256] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {p, q} by [16, 64] {
    foreach index = {m, n, k} in [128 / #p, 256 / #q, 256]
      output.at(p#m, q#n) += lhs.at(p#m, k) * rhs.at(k, q#n);
  }
  return output;
}
```

Listing 3.28: Scalar Matmul (No DMA)


Scalar Matmul: Output Tiling
output [128, 256]
256 cols
128 rows
p: 0..15 (16 row-tiles)
(0,0) (0,1) (0,2) (0,3) (0,4) (0,5) (0,6) (0,7)
(1,0) (1,1) (1,2) (p=1,q=3) 8×4 tile (1,4) (1,5) (1,6) (1,7)
(2,0) (2,1) (2,2) (2,3) (2,4) (2,5) (2,6) (2,7)
(3,0) (3,1) (3,2) (3,3) (3,4) (3,5) (3,6) (3,7)
...
⋮
q: 0..63 (64 col-tiles)

```
parallel {p, q} by [16, 64]
  foreach {m,n,k} in [8, 4, 256]
    output.at(p#m, q#n) += ...
```

(showing 4×8 of the 16×64 tile grid)

The [128, 256] output divided into a 16 × 64 tile grid. Each (p, q) pair owns one tile.

This is dense — here is what each piece does.

Output shape from operand dimensions. `s32 [lhs.span(0), rhs.span(1)] output` builds the output shape from the inputs: `lhs.span(0)` is 128 (rows of the left matrix) and `rhs.span(1)` is 256 (columns of the right matrix).

Multi-axis parallel. `parallel {p, q} by [16, 64]` declares two parallel indices of one parallel level with a comma. This creates a 16 × 64 = 1024-way parallel grid. Each `(p, q)` pair owns a tile of the output.

Named tuple destructuring. `foreach index = {m, n, k} in [128 / #p, 256 / #q, 256]` introduces three nested loop indices — `m`, `n`, `k` — bound to a tuple called `index`. The trip counts are:

- `128 / #p = 128 / 16 = 8` rows per tile
- `256 / #q = 256 / 64 = 4` columns per tile

- `256` for the full contraction dimension

Composed global indices. `p#m` composes the tile index with the within-tile offset. `p` selects which of the 16 row-tiles, `m` runs 0..7 within that tile, so `p#m` covers 0..127 across all tiles.

The arithmetic. `output.at(p#m, q#n) += lhs.at(p#m, k) * rhs.at(k, q#n)` is the textbook dot product: for each output element, sum products along K.

3.4.2 Tracing One Output Element

Following one output element through a kernel makes the index arithmetic concrete. This section traces it step by step.

Pick global position `(row=37, col=50)` in the output. Which block and thread own it?

Blocks partition 128 rows into 8 tiles of 16: `px = 37 / 16 = 2`, offset `qx = 37 % 16 = 5`. Columns partition 256 into 16 tiles of 16: `py = 50 / 16 = 3`, offset `qy = 50 % 16 = 2`. So block `(2, 3)`, thread `(5, 2)`.

For that thread, the K-loop runs 16 iterations. On iteration `tile_k = 0`, `dma.copy` loads `lhs` rows 32..47 and K columns 0..15 into shared `lhs_load`, and `rhs` K rows 0..15 and columns 48..63 into shared `rhs_load`. The inner `foreach k in [16]` accumulates `lhs_load.data.at(5, k) * rhs_load.data.at(k, 2)` for k = 0..15 into `output.at(37, 50)`. Then `tile_k = 1` loads the next K-strip, and so on. After all 16 iterations, `output.at(37, 50)` holds the complete dot product — same answer as the scalar reference.

3.5 Tile Selection and Data Expressions

Tiling is expressed with data expressions rather than raw pointer arithmetic. This section covers the composition operator and the forms a data expression may take.

`chunkat`, `subspan`, and `view().from()`: Three Ways to Select a Tile

Croqtile gives you three common ways to describe the tile you want to move.

`#`: Compose and Extent

Croqtile uses `#` in two related ways.

Handling irregular access

The tiling primitives introduced so far — `chunkat`, `subspan(...).at(...)` — assume tensors that divide evenly into tiles. This works for textbook GEMM where M, N, K are multiples of the tile dimensions. Production kernels are rarely so tidy. Expert batches start at dynamic offsets. Convolution windows overlap. The last K-tile is almost never an exact multiple of TILE_K. Some inputs arrive as flat buffers that need reshaping before MMA can consume them.

Croqtile provides four tools for these cases. Each one modifies how the compiler interprets a tensor's addressing without touching the pipeline structure — DMA, TMA, MMA, events, swizzle all continue to work unchanged.

Tile and Flow

It is possible to use the `chunkat` expression for either the source or the destination of a *DMA statement*. When `chunkat` is used only for the source, it typically divides the input or buffer into smaller chunks to move it to storage closer to the processor. In contrast, when `chunkat` is used only for the destination, it typically moves the buffer to compose part of the output. The following code showcases an example:

```
__co__ auto foo(f32 [36] input) {
  f32 [36] output;
  parallel p by 6 {
    f0 = dma.copy input.chunkat(p) => shared;  // tile the source
    dma.copy f0.data => output.chunkat(p);     // un-tile the destination
  }
  return output;
}
```

Listing 3.29: Tile and Flow

In this example, the statement applies `chunkat` to the destination, moving the data block in `shared` and "un-tiling" each block to the output. Note that the Croqtile compiler **requires the source data and destination data to have the same shape and element type** if not using *storage* as the destination *data expression*.

However, using the `chunkat` expression for both the source and destination is rarely supported by any target platform at this time.

Compose Two Bounded Variables for Tiling with

In the example above, there is a need to tile the matrices in a manner that involves **two bounded variables**. This is where the **# operator** comes into play, allowing the program to compute the **Cartesian product** of two bounded variables (**not commutative**).

The # Operator:

- The code defines the tiling parameters with four bounded variables: `q`, `n`, `h`, and `w`, whose upper bounders are `6`, `N/6`, `H/16`, and `W/512`.
- Suppose `N` is 18, `q` represents physical thread index. You want each thread to process data that is contiguous in the first dimension. That is, the first thread handles input[0~2][xxx], the second thread handles input[3~5][xxx], and so on.
- `q # n` will result in a new bounded variable implicitly, whose upper bound is `N`: bound of `q` multiply bound of `n`.
- For every data move, the stride of each dimension is 1, 3, 16, 512. In the last dimension, the index is `w`. In the first dimension, the index is `q * (bound of n) + n`.

Why this is Important:

- The combination of `q` and `n` ensures that physical and virtual parallelism can work together efficiently.
- This helps optimize **memory access patterns**, especially when working with large matrices, where efficient memory use is crucial.
- Unlike higher-level loop scheduling techniques like in TVM, which abstract away the actual data movement, Croqtile explicitly links **virtual loops** to **physical DMA operations**. This results in more efficient data movement and computation.

3.5.1 Arbitrary-Offset Windows: view and from

A tile sometimes starts at an offset computed at runtime. This section shows how `view` and `from` describe such a window.

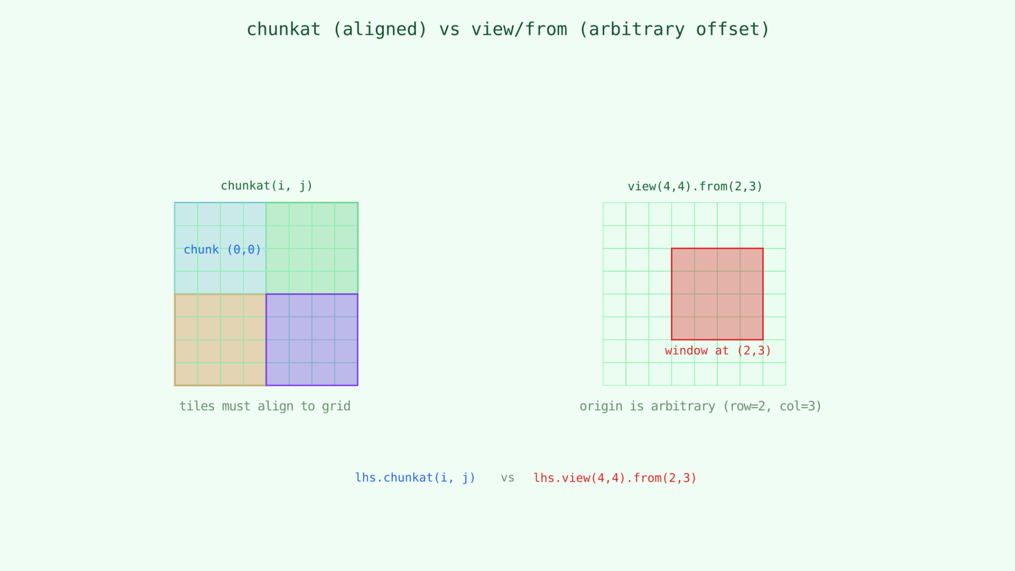
`chunkat` divides a tensor into a regular grid of equal-sized tiles. But what if the slice you need does not start on a tile boundary? Consider a mixture-of-experts (MoE) kernel: each expert processes a different number of tokens, so the starting row for each expert's operand is determined at runtime. You cannot precompute a fixed tile grid.

`view(M, N).from(row, col)` defines an `M x N` rectangle starting at any `(row, col)` — no alignment requirement:

```
patch = matrix.view(16, 16).from(37, 50);
```

Listing 3.30: Arbitrary-offset windows: view and from

This is a `[16, 16]` slice starting at row 37, column 50. The origin can be any runtime value.



The power of `view` / `from` is composability: once you carve out a window, all downstream operations — `subspan`, `chunkat`, `dma.copy`, `tma.copy` — apply to the window as if it were a standalone tensor. The pipeline stays unchanged; only the origin shifts:

```
expert_lhs = lhs.view(expert_M, K).from(expert_offset, 0);
dma.copy expert_lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => shared;
```

Listing 3.31: Arbitrary-offset windows: view and from

In an MoE stack, each expert's token batch starts at a dynamic row `expert_offset`. The `view` / `from` call rewires the operand at the top of the pipeline; everything below — DMA, staging, MMA loop — runs the same code for every expert.

When to use it. Prefer `chunkat` when the tensor divides evenly into tiles at compile time. Use `view` / `from` when the window's origin or extent is determined at runtime, or when the geometry does not align to the tile grid.

3.5.2 Strided Tiles: subspan, step, and at

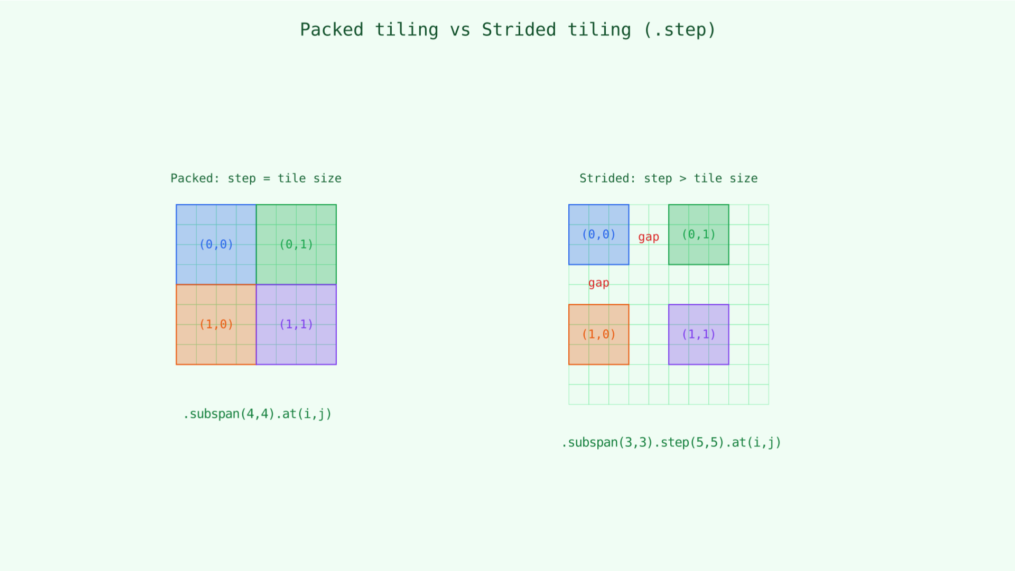
Strided tiles need an explicit step. This section shows how `subspan`, `step` and `at` combine.

`subspan(M, K).at(i, j)` selects the tile at logical tile indices `(i, j)` with extent `[M, K]`. By default, tiles are packed contiguously: tile `(1, 0)` starts immediately after tile `(0, 0)`. But some layouts space tiles apart — either because there is padding between rows, or because a stencil kernel needs overlapping windows.

Adding `.step(sM, sK)` overrides the stride: tiles are spaced `sM` rows and `sK` columns apart instead of being packed:

```
matrix.subspan(16, 16).step(32, 32).at(i, j);
```

Listing 3.32: Strided tiles: .subspan, .step, and .at


Packed tiling vs Strided tiling (.step)
Packed: step = tile size
(0,0)
(0,1)
(1,0)
(1,1)
.subspan(4,4).at(i,j)
Strided: step > tile size
(0,0)
gap
(0,1)
gap
(1,0)
(1,1)
.subspan(3,3).step(5,5).at(i,j)

Tile `(0,0)` starts at `(0,0)`, but tile `(1,0)` starts at row 32 (not row 16), leaving a 16-row gap. Omitting `.step` uses a step equal to the tile size — the packed case.

When you need this. Skipping padding or guard bands between rows. Overlapping stencil windows where the step is smaller than the tile extent (e.g., a `16 x 16` tile sliding by 8 rows). Matching an outer memory layout that is not dense tile-major — for instance, when tiles are interleaved with metadata.

3.5.3 Zero-Padding with zfill

Partial tiles at the edge of a shape need padding. This section shows how `.zfill` supplies it.

Zero-padding: `.zfill`

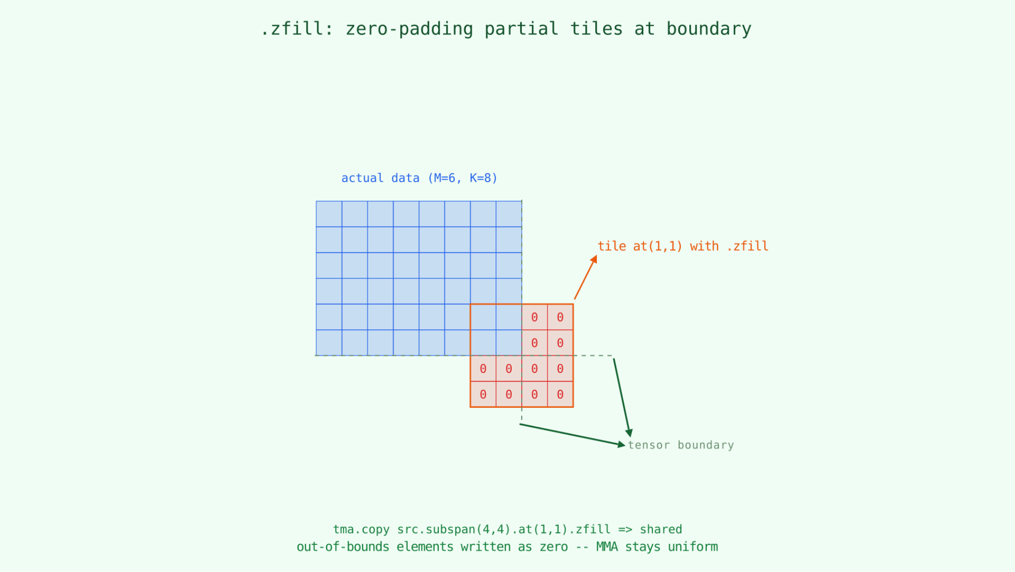
Every tiled kernel faces the same edge case: what happens when M or K is not a multiple of the tile size? The last tile along that axis is partial — some of its elements lie beyond the tensor boundary.

The brute-force fix is to add a special code path for the boundary tile. But this breaks the uniformity of the MMA loop and adds branches that the compiler cannot optimize away. `.zfill` solves this elegantly:

```
tma.copy.swiz<3> lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k).zfill
  => lhs_load_s;
```

Listing 3.33: Zero-padding: .zfill

`.zfill` applies to the source side of a copy: out-of-range elements are written as zero in the destination tile. For a GEMM, zeros contribute nothing to the accumulation (0 × anything = 0), so the MMA loop stays completely uniform — same code for interior tiles and boundary tiles — while remaining mathematically correct. No branches, no special cases, no performance penalty.


.zfill: zero-padding partial tiles at boundary
actual data (M=6, K=8)
tile at(1,1) with .zfill
0
tensor boundary
tma.copy src.subspan(4,4).at(1,1).zfill => shared
out-of-bounds elements written as zero -- MMA stays uniform

3.5.4 Layout Reinterpretation with span_as

A buffer can be reinterpreted under a different shape. This section shows `span_as` and the constraints on it.

Layout reinterpretation: `span_as`

Some kernels load data as a flat 1D strip but need to feed it to MMA as a 2D matrix. A naive approach would copy the data into a reshaped buffer — `span_as` avoids this by reinterpreting the existing buffer's shape in place:

```
flat_buffer.span_as([rows, cols])
```

Listing 3.34: Layout reinterpretation: span as

Element count is preserved; only the logical rank changes. No data is moved.

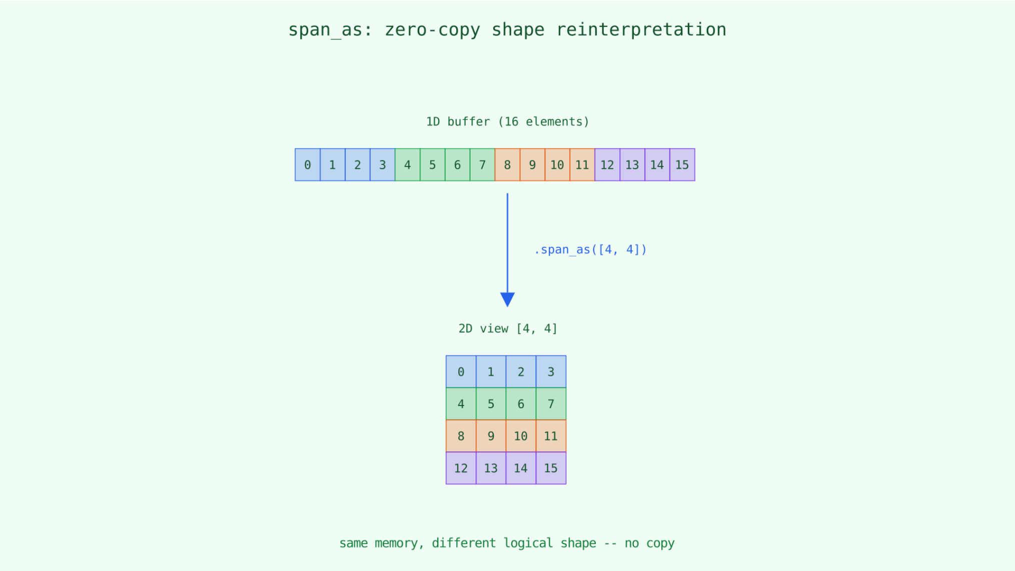
```
strip_load = dma.copy data.chunkat(tile) => shared;
tile_2d = strip_load.data.span_as([tile_m, tile_k]);
ma = mma.load tile_2d.chunkat(_, iv_warp);
```

Listing 3.35: Layout reinterpretation: span as

`rows * cols` must equal the span length of the underlying storage, or the compiler rejects the program — this invariant is checked at compile time, not at runtime.

3.5.5 Tile Selection: Summary and Syntax

This section collects the tile-selection syntax and the rules that govern it.


span_as: zero-copy shape reinterpretation
1D buffer (16 elements)
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
.span_as([4, 4])
2D view [4, 4]
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
same memory, different logical shape -- no copy

Concept	**Syntax**	**Role**
Software DMA (Ch. 2, 6)	`dma.copy` / `dma.copy.swiz<N>`	Thread-cooperative tile transfer;
Swizzle	`.swiz<N>` on copy + `mma.load.swiz<N>`	layout conflict-free SMEM layout; same effect for DMA and TMA
Arbitrary windows	`view(M,N).from(r,c)`	Ragged or runtime-positioned slices
Strided tiling	`.subspan().step().at()`	Non-packed layouts, overlapping stencils
Partial tiles	`.zfill`	Zero-fill out-of-bounds elements
Shape reinterpretation	`span_as([dims])`	Zero-copy reshape for staging buffers

In short

Syntax	Meaning
`tma.copy.swiz<N> src => dst`	TMA copy with swizzle mode `N` (0–3)
`dma.copy.swiz<N> src => dst`	DMA copy with swizzle mode `N` (0–3); same layout as TMA
`mma.load.swiz<N> src`	MMA operand load consistent with swizzle `N`
`.view(M, N).from(r, c)`	Arbitrary-offset `M x N` window
`.subspan(M, K).step(sM, sK).at(i, j)`	Strided tile selection
`.zfill`	Zero-fill out-of-bounds elements on copy source
`span_as(dims)`	Reinterpret linear storage as shaped tensor

Syntax at a glance Section 2.13 covers C++ interop: `__device__` functions, preprocessor guards and injected source text for the cases where a program needs plain C++.

3.6 Data Movement: Operations, Configuration, and Async Patterns

Data movement has a small grammar of operations, configurations and futures. This section describes that grammar and the asynchronous patterns built on it.

What You Have So Far

The skeleton of every Croqtile program is now in your hands: a `__co__` function with typed, shaped parameters and a return value; `parallel` and `.at()` for element-wise computation; `lhs.span` for deriving output shapes from inputs; and the host-side `make_spandata` / `.view()` API for creating and passing tensors.

Section 2.9 introduces `dma.copy` and `chunkat` to express block-level data movement.

Data Movement: The Tile Movement Engine

Section 1.7 used scalar indexing: pick one position, read inputs, compute, write output.

Croqtile makes that explicit. Instead of treating data movement as a hidden side effect of indexing, it gives you a small family of tile-movement statements:

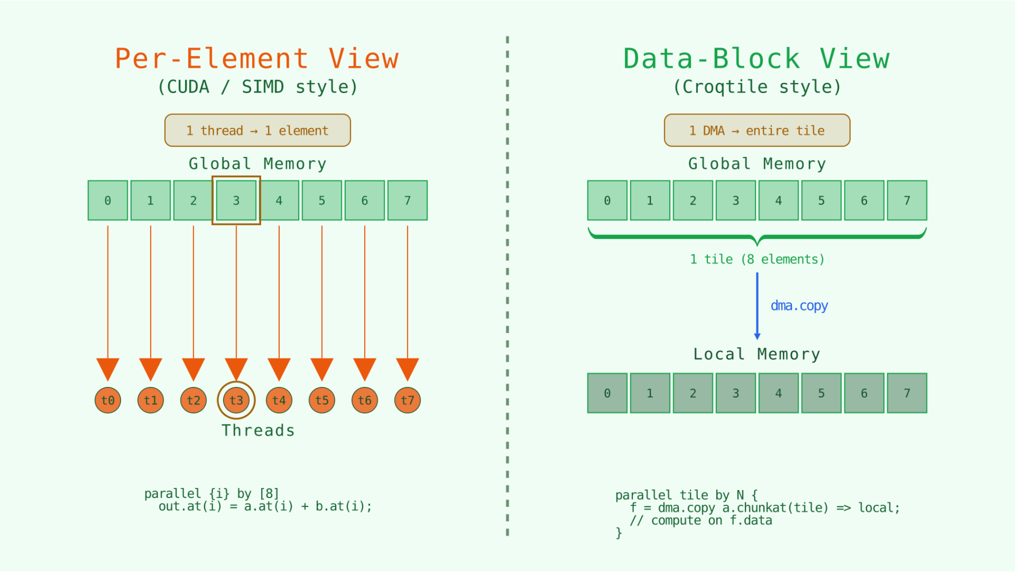
- `dma.copy` moves a tile as-is.
- `dma.transp<...>` permutes dimensions while moving it.
- `dma.pad<...>` enlarges a tile and fills the border.

This part of the guide introduces that model, shows the core syntax, and gives the rules you need to write DMA and TMA code that is both correct and likely to lower well.

Left: element-by-element thinking. Right: tile-by-tile movement into fast memory.

Other Operations: `transp` and `pad`

`copy` is only the simplest member of the family.


Per-Element View
(CUDA / SIMD style)
1 thread → 1 element
Global Memory
0
1
2
3
4
5
6
7
t0
t1
t2
t3
t4
t5
t6
t7
Threads
Data-Block View
(Croqtile style)
1 DMA → entire tile
Global Memory
0
1
2
3
4
5
6
7
1 tile (8 elements)
dma.copy
Local Memory
0
1
2
3
4
5
6
7

```
parallel {i} by [8]
  out.at(i) = a.at(i) + b.at(i);
```

```
parallel tile by N {
  f = dma.copy a.chunkat(tile) => local;
  // compute on f.data
}
```

DMA vs TMA

Both engines move shaped tiles. The difference is how they do it and what constraints they require.

	`dma`	`tma`
Common destination	`shared`	`shared`
Async support	Yes	Yes

Example TMA use:

```
f = tma.copy.async lhs.subspan(64, 64).at(block_m, iv_k) => shared;
wait f;
```

Listing 3.36: DMA vs TMA

The practical rule is simple:

- Use `dma` when tile sizes are dynamic, irregular, or you want the portable path.

The main TMA constraint

TMA tile shapes must be host-computable. Runtime tile indices are fine:

```
tma.copy lhs.subspan(64, 64).at(block_m, iv_k) => shared;
```

Listing 3.37: The main TMA constraint

But a kernel-runtime value cannot define the tile shape itself:

```
// Not valid for TMA if TILE_M is only known inside the kernel
tma.copy lhs.subspan(TILE_M, 64).at(block_m, iv_k) => shared;
```

Listing 3.38: The main TMA constraint

If the tile size truly depends on kernel-runtime state, use `dma.copy` instead.

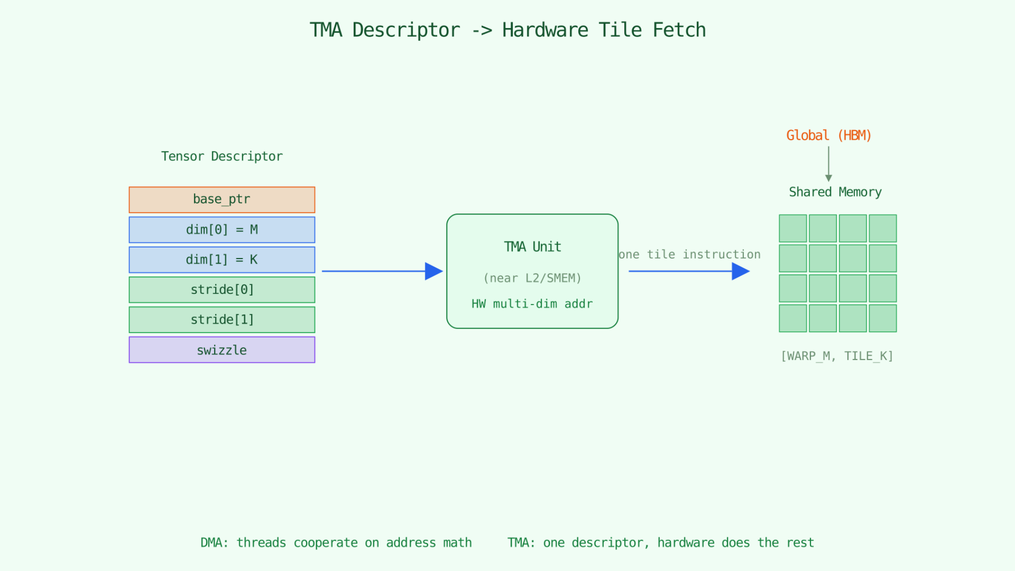
`tma.copy`: target tensor movement

The surface syntax mirrors `dma.copy`:

```
tma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s;
```

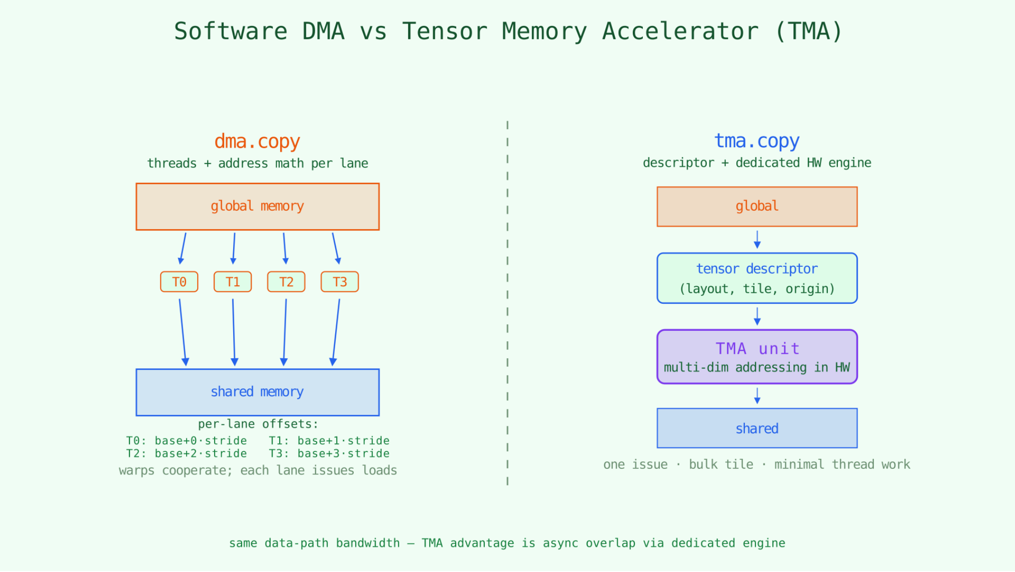
Listing 3.39: tma.copy: target tensor movement

Same source expression, same `=>` destination form. The difference is **who does the work**:


TMA Descriptor -> Hardware Tile Fetch
Tensor Descriptor
base_ptr
dim[0] = M
dim[1] = K
stride[0]
stride[1]
swizzle
TMA Unit
(near L2/SMEM)
HW multi-dim addr
one tile instruction
Global (HBM)
Shared Memory
[WARP_M, TILE_K]
DMA: threads cooperate on address math
TMA: one descriptor, hardware does the rest

- **DMA path.** Threads cooperate to cover the tile; each lane participates in address math and load issue.
- **TMA path.** One descriptor-based operation describes the tensor slice;

What this buys you. You still synchronize on the whole tile (with events or the pipeline discipline from Section 3.8.5), but you drop the per-thread load choreography. The compiler builds the tensor descriptor from your `__co__` signature and global layouts.


Software DMA vs Tensor Memory Accelerator (TMA)
dma.copy
threads + address math per lane
global memory
T0
T1
T2
T3
shared memory
per-lane offsets:
T0: base+0·stride T1: base+1·stride
T2: base+2·stride T3: base+3·stride
warps cooperate; each lane issues loads
tma.copy
descriptor + dedicated HW engine
global
tensor descriptor
(layout, tile, origin)
TMA unit
multi-dim addressing in HW
shared
one issue · bulk tile · minimal thread work
same data-path bandwidth — TMA advantage is async overlap via dedicated engine

Async Operation: the DMA Statement

Except for parallel execution, Croqtile allows one fixed form of async operation: the DMA statement.

Conceptually, a DMA statement is executed asynchronously with the SPMD code. It works quite similar to CPU async thread, except its behavior is limited by the DMA configuration. (CPU allows to program the async thread as will)

The below code showcases one basic DMA statement.

```
global f32 [10] g_data;
local f32 [10] l_data;
f = dma.copy.async g_data => l_data;
// ... async operations
wait f;          // explicit wait
```

Listing 3.40: Async Operation: the DMA Statement

You might observe the assignment of ’f’. This is the handler for the asynchronous DMA entity. In Croqtile, you refer to it as the ’future’ of the DMA operation. Programmers have the option to code the explicit synchronization statement ’wait’ to pause the current thread until the ’future’ arrives. Otherwise, the thread continues execution in parallel with the DMA operation.

At times, programmers may find it tedious to explicitly define temporal data. Croqtile offers an even simpler syntax:

```
global f32 [10] data;
f = dma.copy data => local;
... f.data;  // retrieve the 'local' data from the future
```

Listing 3.41: Async Operation: the DMA Statement

Here, you do not need to specify the exact target location where the DMA transfers data. Instead, you only specify the destination memory type. This is advantageous in many scenarios. And to retrieve the transferred data, you simply invoke the ’data’ member function of ’future’. Besides, the dma operation is not marked with ’.async’, so the wait operation on ’f’ is not needed and cannot be performed.

DMA operations entail intricate details that demand careful programming. Programmers should refer to the DMA manual to make informed decisions for their code. Nonetheless, Croqtile compiler provides plenty of static and runtime checks to assist programmers in avoiding potential errors in this aspect.

Data Movement Statement

Croqtile aims to address multiple levels of data movement. Croqtile adopts this terminology and abstracts data movement as **DMA Operations**.

Operation Type

Croqtile maps these functionalities at the software level as different **DMA Operation Types**. The supported operations include:

- `dma.copy`: Copies flat memory directly.
- `dma.pad`: Pads shaped data while transferring the data.

- `dma.transp`: Transposes shaped data while transferring the data.

The configurations, such as padding and transposing details, are programmed as parameters of the DMA operation. For example:

```
global f32 [32, 16, 9] input;
dma.transp<0, 2, 1> input => shared;  // Result shape [32, 9, 16]
dma.pad<{1, 0, 3}, {0, 1, 2}, {0, 0, 0}, 0.1f> input => shared; // Result shape
    [33, 17, 14]
```

Listing 3.42: Operation Type

Here, the **DMA configuration**s are enclosed by `<>`. The configuration varies according to different operations. The detailed configuration syntax and limitations for a specific platform, such as , are listed below as an example:

Programmers should note that support for DMA types other than `dma.copy` varies by platform. For instance, on the platform, it could be either a mapping of TMA or an orchestration of load instructions from multiple threads.

The data movement is essential for *tileflow programs*. In this section, you will learn more advanced syntax of *DMA statements* to support different scenarios.

Full Non-Blocking DMA Mode (Chain Mode in Croqtile)

In Croqtile, it's possible to perform a full non-blocking DMA by chaining multiple asynchronous DMA operations and using event-based notifications with after. This enables complete non-blocking execution, where one DMA operation is triggered only after the completion of a prior one. Here's an example of such a setup:

```
out_store = dma.copy.async l2_out => output.chunkat(m_tile, n_tile) after
    out_store_s;
```

Listing 3.43: Full Non-Blocking DMA Mode (Chain Mode in Croqtile)

In this example:

- `out_store` is the asynchronous DMA operation that transfers data from l2_out to output.chunkat(m_tile, n_tile).
- `out_store_s` is another DMA operation or event that must complete before out_store can proceed.
- The after out_store_s syntax specifies that `out_store` should only start after the completion of `out_store_s`, which ensures that there is no blocking in the main thread.

In this case, neither `out_store` nor `out_store_s` will block the main program flow. The program continues executing while these DMA operations are handled in the background. The key difference here is that the completion of `out_store_s` triggers the start of `out_store`, creating an event-driven dependency between the two DMA operations. This model enables highly efficient and non-blocking memory transfers.

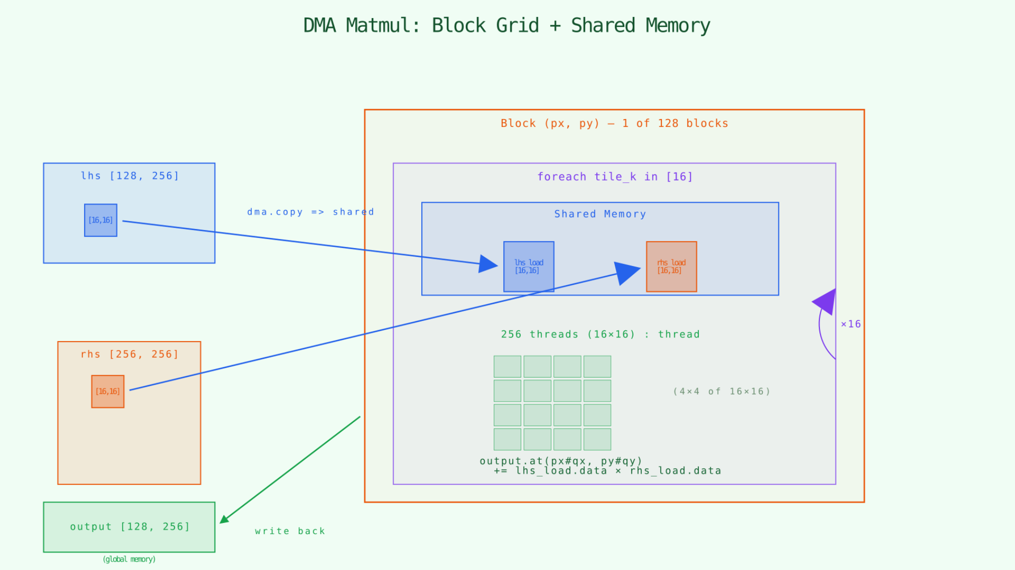
3.6.1 Worked Example: DMA Matmul

A tiled matmul with explicit copies puts the movement statements in context. This section reads the full kernel.

DMA Matmul: Tiles in shared storage

```
__co__ s32 [128, 256] matmul(s32 [128, 256] lhs, s32 [256, 256] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {px, py} by [8, 16] : block {
    foreach {tile_k} in [16] {
      lhs_load = dma.copy lhs.chunkat(px, tile_k) => shared;
      rhs_load = dma.copy rhs.chunkat(tile_k, py) => shared;
      parallel {qx, qy} by [16, 16] : thread {
        foreach k in [256 / #tile_k]
          output.at(px#qx, py#qy) += lhs_load.data.at(qx, k) * rhs_load.data.at(
    k, qy);
      }
    }
  }
  return output;
}
```

Listing 3.44: DMA Matmul: Tiles in shared storage


DMA Matmul: Block Grid + Shared Memory
Block (px, py) — 1 of 128 blocks
lhs [128, 256]
[16,16]
dma.copy => shared
foreach tile_k in [16]
Shared Memory
lhs_load
[16,16]
rhs_load
[16,16]
×16
256 threads (16×16) : thread
rhs [256, 256]
[16,16]
(4×4 of 16×16)
output.at(px#qx, py#qy)
+= lhs_load.data × rhs_load.data
output [128, 256]
(global memory)
write back

What changed from the scalar version:

1. The outer `parallel` now uses `: block` with brace-form indices `{px, py}`. An 8 × 16 grid of blocks covers the output.

1. A `foreach` over `tile_k` walks the K dimension in 16 steps. Each step copies one strip of `lhs` and one strip of `rhs` into `shared` storage with `dma.copy ... => shared`.

1. An inner `parallel {qx, qy} by [16, 16] : thread` creates 256 threads within each block. Each thread owns one output element within the block's tile.

1. The arithmetic reads from `lhs_load.data` and `rhs_load.data` — the shared-memory copies — instead of global `lhs` and `rhs`.

The composed indices work in two layers: `px#qx` composes block index `px` with thread index `qx` to form the global row; `py#qy` does the same for columns.

Dimension arithmetic. With `[8, 16]` blocks, each block owns `128/8 = 16` rows and `256/16 = 16` columns. The inner `[16, 16]` threads subdivide that into one element per thread. Along K, 16 tiles of `256/16 = 16` elements each cover the full contraction.

3.6.2 Multi-Buffering: dma.any, swap, and rotate

Overlapping copies with computation needs more than one buffer. This section covers `dma.any`, `swap` and `rotate`.

Double buffering with `swap`

Give the K-loop **two** logical buffers. While the math drains buffer 0, DMA fills buffer 1 with the next tile. After the math step, **swap** the handles: what was "next" becomes "current," and the freed slot is ready for the following load.

Croqtile spells this with `dma.copy.async` (non-blocking copy), `dma.any` (a placeholder future), `swap` (exchange future handles), and a three-phase loop:

```
__co__ auto matmul(s32 [M, K] lhs, s32 [K, N] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {px, py} by [8, 16] : block
    parallel {qx, qy} by [16, 16] : thread {
    with tile_k in 16 {
      // Prologue: start loading tile 0
      lf0 = dma.copy.async lhs.chunkat(px, tile_k) => shared;
      rf0 = dma.copy.async rhs.chunkat(tile_k, py) => shared;
      // Placeholder futures for buffer 1
      lf1 = dma.any;
      rf1 = dma.any;
      // Steady state: load next tile while computing on current
      foreach tile_k(1:) {
        lf1 = dma.copy.async lhs.chunkat(px, tile_k) => shared;
        rf1 = dma.copy.async rhs.chunkat(tile_k, py) => shared;
        wait lf0, rf0;
        foreach k in [256 / #tile_k]
          output.at(px#qx, py#qy) += lf0.data.at(qx, k) * rf0.data.at(k, qy);
        swap(lf0, lf1);
        swap(rf0, rf1);
      }
      // Epilogue: compute on the last loaded tile
      foreach k in [256 / #tile_k]
        output.at(px#qx, py#qy) += lf0.data.at(qx, k) * rf0.data.at(k, qy);
    }
  }
  return output;
}
```

Listing 3.45: Double buffering with swap

The three phases

Prologue. Issue loads for tile 0 into `lf0`/`rf0`. No compute yet — the first tile must land before anything can multiply it.

Steady state. For each subsequent tile: start loads into `lf1`/`rf1`, compute on `lf0`/`rf0` from the previous iteration, then `swap` so names track the active buffers. New copies land in `lf1`/`rf1` **before** the compute reads `lf0`/`rf0`, so you never read a buffer being overwritten.

Epilogue. After the last swap, `lf0`/`rf0` hold the final tile; one more compute pass drains them.

`swap`: names, not bytes

`swap(lf0, lf1)` exchanges **future handles** — the Croqtile-level names that refer to buffers. Only the names rotate. For triple buffering, `rotate(f0, f1, f2)` cycles three handles in one step.

`dma.any`: placeholder futures

`dma.any` creates a future that does not yet represent a transfer. It gives the type system something to `swap` against on the first steady-state iteration. Before any use of `lf1.data`, a real `dma.copy` has been assigned.

Multi-Buffering Optimization with Croqtile

To aggressively optimize the high-performance computation kernels, one effective method is to overlap computation with DMA (Direct Memory Access) data movement. This approach, known as **multi-buffering optimization**, typically requires programmers to manage multiple buffers.

In Croqtile, this method requires two key programming primitives: the 'dummy' future **dma.any**, and the **swap/rotate** function. Let's illustrate by an example:

```
__co__ s32 [6, 17, 128] ele_add(s32 [6, 17, 128] lhs,
                                s32 [6, 17, 128] rhs) {
  s32[lhs.span] output;
  parallel p by 6 {
    with index = {x, y} in [17, 4] {
      local s32 [rhs.span / {#p, #x, #y}] l_out;
      foreach x { // first bunch
        lfA = dma.copy.async lhs.chunkat(p, x, y) => local;
        rfA = dma.copy.async rhs.chunkat(p, x, y) => local;
        lfB = dma.any;  // dummy DMA used for multi-buffering
        rfB = dma.any;
        foreach y(1:) {
          lfB = dma.copy.async lhs.chunkat(p, x, y) => local;
          rfB = dma.copy.async rhs.chunkat(p, x, y) => local;
          wait lfA, rfA;  // wait another bunch
          call kernel(lfA.data, rfA.data, l_out, |lfB.span|);
          dma.copy l_out => output.chunkat(p, x, y - 1);
          swap(lfA, lfB); // exchange futures
          swap(rfA, rfB);
        }
        lf = select(#y % 2, lfB, lfA);
        rf = select(#y % 2, rfB, rfA);
        wait lf, rf;  // handle the last bunch
        call kernel(lf.data, rf.data, l_out, |lfB.span|);
        dma.copy l_out => output.chunkat(p, x, y(-1));
      }
    }
  }
  return output;
}
```

Listing 3.46: Multi-Buffering Optimization with Croqtile

In croqtile function `ele_add`, there are two input parameters: 'lhs' and 'rhs', both double-buffered in local storage. You refer to these buffers as 'A' and 'B'. While buffer A is used for computation, buffer B is filled with data via a DMA operation. Once the kernel function completes computation on buffer A, the roles of A and B are swapped: A becomes the buffer for loading the next data chunk, and B becomes the buffer for computation. This process iterates until all data chunks are consumed.

The implementation divides data movements and computations into three stages, including:

- the **prologue**(line 8-11). It (pre-)load the first chunk of data. In this example, buffer A is pre-loaded with data.
- the **body**(line 12-20). It processes the data pre-loaded in last iteration or in prologue, and pre-load the data for next iteration.
- and the **epilogue**(line 21-25). It processes the last data chunk only.

In the code, futures 'lfB' and 'rfB' are declared as dummies. These dummy futures serve as placeholders and are replaced by DMA statements at lines 13-14. The reason for declaring dummy futures is that the futures ('lfB' and 'rfB') of the DMA invoked in the body stage are used in the epilogue stage (lines 21-22). However, from a lexical scope perspective, defining futures inside the foreach-block (lines 12-20) would not extend their lifetime to their last uses. Therefore, it is necessary to declare 'lfB' and 'rfB' early.

The **swap** statements at lines 18-19 exchange futures to ensure the **wait** statement at line 15 functions correctly. Note that only futures with identical DMA operations can be swapped. Additionally, the iteration range of the *foreach* statement at line 12 is adjusted. `foreach y(1:)` means the loop over the *bounded variable* 'y' starts from the second value up to the upper bound. In line 17, the *chunkat* expression is also tuned. `y - 1` results in the current value of 'y' minus 1, setting up the body stage to work properly with the prologue stage. This adjustment is necessary because the iteration count must be reduced by 1, and the pre-load operation should fetch the 'next' chunk while computation is conducted on the 'current' chunk of data. Alternatively, you can use `foreach y(:-1)` and adjust the chunkat expression of the pre-load statements at lines 13-14 to achieve the same effect.

In line 25, the expression `y(-1)` retrieves the 'last value' of 'y' within its bound.

Note that the **select** expressions in line 21-22 handle both odd and even upper-bound cases for the bounded variable 'y'. In either case, it obtains the future of the last chunk.

If the programmer want to enable more than two buffers, **rotate** statements is required to replace **swap**.

3.6.3 Worked Example: A Pipelined Kernel

A pipelined kernel combines tile selection, movement and contraction. This section presents one complete program.

Example: TMA in a pipelined matmul

The pipeline skeleton from Section 3.8.5 is unchanged: ring of stages, `wait` / `trigger` on events, MMA commit, consumer drains tiles while producer fills the next slot. Here the producer swaps `dma.copy` for `tma.copy.swiz<3>` and the consumer swaps `mma.load` for `mma.load.swiz<3>`:

```
__co__ void matmul(global f16 [M, K] lhs, global f16 [N, K] rhs, global f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
    shared event full[STAGES], empty[STAGES];
    shared f16 [WARP_M, TILE_K] lhs_load_s[STAGES];
    shared f16 [WARP_N, TILE_K] rhs_load_s[STAGES];
    shared f16 [WARP_M, WARP_N] output_s;
    parallel p1 by 2 : group-4 {
      inthreads.async (p1 == 0) {
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          stage = iv_k % STAGES;
          wait empty[stage];
          tma.copy.swiz<3> lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k)
            => lhs_load_s[stage];
          tma.copy.swiz<3> rhs.subspan(WARP_N, TILE_K).at(block_n, iv_k)
            => rhs_load_s[stage];
          trigger full[stage];
        }
      }
      inthreads.async (p1 == 1) {
        mc = mma.fill.f16 0.0f;
        foreach {s} in [STAGES] { trigger empty[s]; }
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          stage = iv_k % STAGES;
          wait full[stage];
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            ma = mma.load.swiz<3> lhs_load_s[stage].chunkat(_, iv_warp);
            mb = mma.load.swiz<3> rhs_load_s[stage].chunkat(_, iv_warp);
            mma.row.row mc, ma, mb;
          }
          mma.commit;
          trigger empty[stage];
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m, block_n);

      }
    }
  }
}
```

Listing 3.47: Example: TMA in a pipelined matmul

Compared to the Section 3.8.5 `dma.copy` version, only the ingress and operand load lines change; events, staging indices, and commit stay the same. — Choose TMA or DMA for stores according to your target.

3.7 Contraction and Layout Contracts

Contraction in Croqtile is a four-step contract between layout and accumulation. This section describes that contract and the variants that share it.

Partial Tiles, Guarding, and `.zfill`

Real kernels often end with a partial tile. The important distinction is whether the **source tile shape** and **destination tile shape** are the same.

For example:

```
dma.copy.swiz<32>.zfill
  lhs.view(ROWS_M, TILE_K).from(base_m, base_k)
    => lhs_load_s;
```

Listing 3.48: Partial Tiles, Guarding, and .zfill

Here `lhs_load_s` might have shape `[group_M, TILE_K]`, while the source view has shape `[ROWS_M, TILE_K]` and `ROWS_M` may be smaller than `group_M` on the last tile. That is a real shape mismatch, so `.zfill` means: copy the valid source rows and write zero into the remaining rows of the destination tile.

That is the right use for `.zfill`.

What happens when the shapes are the same:

- If the source view and destination tile have the same shape, the compiler already guards runtime tails automatically.
- In that case it generates masking so out-of-bounds elements are not read or written.
- This is enough for cases like tiling a `[100]` tensor into `[64]` tiles: the last tile simply has 36 valid elements.

So `.zfill` is **not** the general mechanism for "any partial tile." It is specifically for cases where the destination tile is larger than the valid source region and the extra part of the destination must be filled with zeros.

Why `.zfill` matters:

- If the consumer reads only the valid region, the compiler's automatic masking is usually enough.
- If the consumer reads the whole destination tile, especially an MMA fragment, the invalid region must hold a known value.
- `.zfill` makes that extra region zero.

Use `.zfill` when the valid source region is smaller than the destination tile and the consumer will treat the destination as a full dense tile. This is the common case for GEMM-style pipelines with dynamic row counts or padded edge tiles.

If you add `.zfill` when the source and destination shapes are already the same, the compiler can warn that the `.zfill` is redundant. Conversely, when the shapes differ, the compiler can also warn that `.zfill` is needed to avoid using undefined values in the uncovered region.

`.swiz<N>`: Shared-Memory Layout for the Consumer

Typical GEMM staging code looks like this:

```
dma.copy.swiz<128> lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s;
ma = mma.load.swiz<128> lhs_load_s.chunkat(_, iv_warp);
```

Listing 3.49: .swiz N : Shared-Memory Layout for the Consumer

The load-side and compute-side swizzles need to agree. If you stage a tile with `.swiz<128>`, the consumer should usually load it with the matching swizzle.

This part of the guide only introduces the idea. The MMA chapters go deeper into which swizzle values pair well with which matrix layouts.

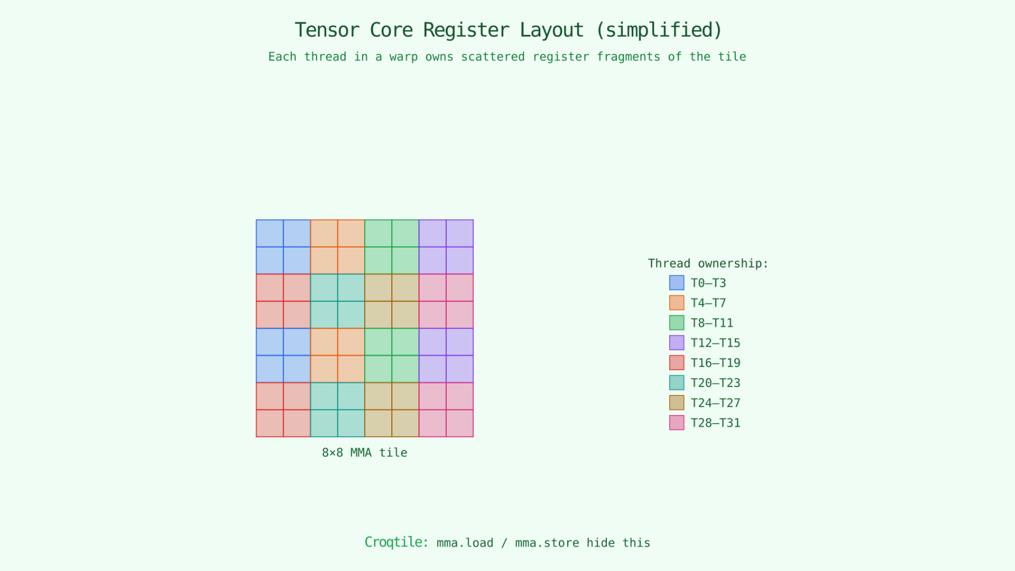
Tensor Contraction: the `mma` Syntax

Section 3.4's tiled matmul computes the inner contraction the way a CPU would: a `foreach k` loop where each iteration multiplies two scalars and accumulates into one output element. That works.

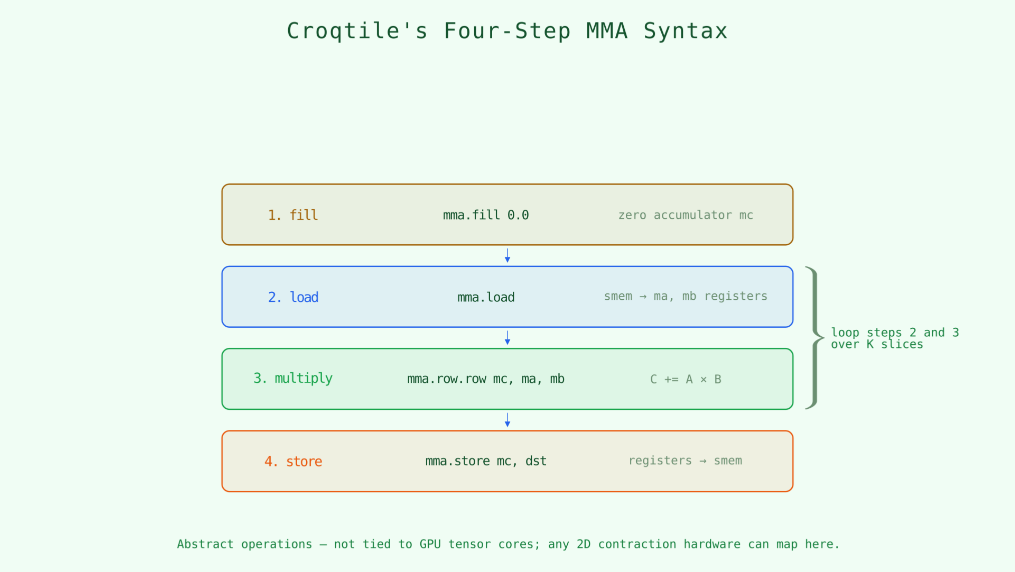
The job is **2D tensor contraction** — the operation C += A × B where A, B, and C are small, fixed-shape matrix tiles. It is the inner kernel of every GEMM, every convolution expressed as an im2col matmul, every attention head's QK^T.

The same mathematical operation — C += A × B on tile-shaped operands

What makes this hard for programmers is not the math but the . Get the layout wrong — say, load a column-major tile into a row-major fragment — and the result is silently incorrect.


Tensor Core Register Layout (simplified)
Each thread in a warp owns scattered register fragments of the tile
8×8 MMA tile
Thread ownership:
T0–T3
T4–T7
T8–T11
T12–T15
T16–T19
T20–T23
T24–T27
T28–T31
Croqtile: mma.load / mma.store hide this

Croqtile's design sidesteps this complexity entirely.


Croqtile's Four-Step MMA Syntax
1. fill
mma.fill 0.0
zero accumulator mc
2. load
mma.load
smem → ma, mb registers
3. multiply
mma.row.row mc, ma, mb
C += A × B
4. store
mma.store mc, dst
registers → smem
loop steps 2 and 3 over K slices
Abstract operations — not tied to GPU tensor cores; any 2D contraction hardware can map here.

The four-step MMA syntax is an abstract interface

Scaling the cooperation scope

The four-step syntax stays the same regardless of how many threads cooperate on a single contraction. What changes is the **cooperation scope** — And that progression is the story of this section.

Beyond dense FP16: what else the four steps express

The examples above use `mma.row.row` on dense FP16 tiles. The same four-step pattern extends to workloads the basic form cannot reach.

But it needs a **metadata operand** `me` that encodes which elements are nonzero:

```
mma.row.row.sp mc, ma, mb, me;
```

Listing 3.50: Beyond dense FP16: what else the four steps express

The `.sp` suffix adds the metadata operand; everything else is the same fill-load-multiply-store rhythm. Any layout combination works: `mma.row.col.sp`, etc. You load `me` from a separate metadata tensor alongside A and B.

Quantized operands with per-tile scaling. FP8 operands (`f8_e4m3`, `f8_e5m2`) need per-tile dequantization so the accumulator stays numerically accurate. Croqtile fuses the scaling into the contraction:

```
mma.row.row.scale mc, ma, mb, sc_a, sc_b;
```

Listing 3.51: Beyond dense FP16: what else the four steps express

Each result element is scaled by `sc_a` and `sc_b` after the contraction — no separate dequant kernel needed. Alternatively, scaling can be a **standalone statement** when the scale source differs from the standard fused path:

```
mma.row.row mc, ma, mb;
mma.scale mc, sc_a, sc_b;
```

Listing 3.52: Beyond dense FP16: what else the four steps express

The standalone `mma.scale` appears in some MoE and mixed-precision kernels.

The `<N>` must agree between `tma.copy.swiz<N>` and `mma.load.swiz<N>` — a mismatch reads garbage. For output, `mma.store.transp mc, dst` writes the accumulator with rows and columns swapped, useful when the next stage expects column-major data.

The table below collects every variant for reference.

Swizzle and layout conflicts

Swizzle applies a fixed XOR-style remapping to column indices within each row, spreading accesses across banks. Croqtile exposes it on **both DMA and TMA**, with the same syntax and the same effect:

```
dma.copy.swiz<3> src => dst;       // software DMA with swizzle
tma.copy.swiz<3> src => dst;
```

Listing 3.53: Swizzle and layout conflicts

The MMA read path must use the same `swiz<N>` so addresses match the staged layout:

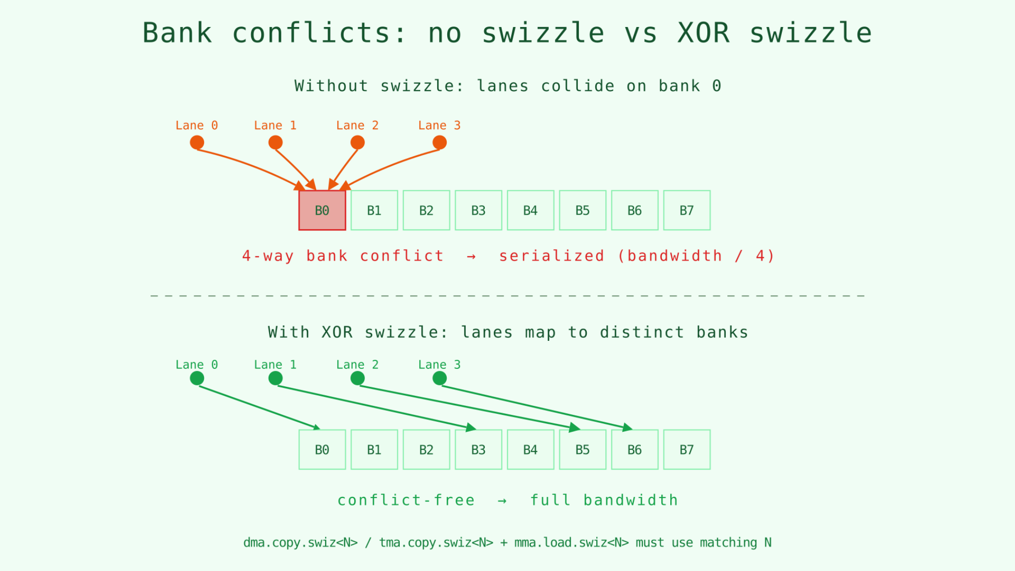
```
ma = mma.load.swiz<3> lhs_load_s.chunkat(_, iv_warp);
```

Listing 3.54: Swizzle and layout conflicts

Swizzle is not a TMA-specific feature. In Croqtile, `dma.copy.swiz<N>` and `tma.copy.swiz<N>` produce identical shared-memory layouts. The difference is only in how the data gets there; the layout that arrives is the same in both cases.

Swizzle levels. The template argument sets the granularity: `swiz<0>` is identity, then 64B, 128B, and 256B XOR patterns for `<1>`, `<2>`, and `<3>`. Larger granularities defeat wider conflict patterns but require tile extents that line up with that granularity.

Matching rule. The `<N>` on the copy must match `mma.load.swiz<N>`. If you load with plain `mma.load` from `swiz<3>` data, addresses disagree and you read garbage. The compiler does not enforce the pairing — it is a correctness invariant you maintain. (As introduced in Section 3.7, `mma.load.swiz<N>` is part of the MMA load family.)


Bank conflicts: no swizzle vs XOR swizzle
Without swizzle: lanes collide on bank 0
Lane 0
Lane 1
Lane 2
Lane 3
B0
B1
B2
B3
B4
B5
B6
B7
4-way bank conflict → serialized (bandwidth / 4)
With XOR swizzle: lanes map to distinct banks
Lane 0
Lane 1
Lane 2
Lane 3
B0
B1
B2
B3
B4
B5
B6
B7
conflict-free → full bandwidth
dma.copy.swiz<N> / tma.copy.swiz<N> + mma.load.swiz<N> must use matching N

3.7.1 Worked Example: One Cooperative Group

This worked kernel shows one cooperative group carrying out the four steps on a single tile.

One group, one tile: the simplest MMA matmul

In Croqtile, that corresponds to : `group`.

```
#define TILE_M 16
#define TILE_N 16
#define TILE_K 16
#define WARP_M 16
#define WARP_N 16
#define WARP_K 16
__co__ void matmul_wmma(global f16 [M, K] lhs, global f16 [N, K] rhs, global f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, TILE_M), cdiv(N, TILE_N)] : block {
    shared f16 [TILE_M, TILE_N] output_s;
    parallel {warp_m, warp_n} by [cdiv(TILE_M, MMA_M), cdiv(TILE_N, MMA_N)] : group {
      mc = mma.fill 0.0;
      foreach {iv_k} in [cdiv(K, TILE_K)] {
        lhs_load_s = dma.copy lhs.subspan(TILE_M, TILE_K).at(block_m, iv_k) =>
    shared;
        rhs_load_s = dma.copy rhs.subspan(TILE_N, TILE_K).at(block_n, iv_k) =>
    shared;
        foreach iv_warp_k in [cdiv(TILE_K, MMA_K)] {
          ma = mma.load lhs_load_s.chunkat(warp_m, iv_warp_k);
          mb = mma.load rhs_load_s.chunkat(warp_n, iv_warp_k);
          mma.row.row mc, ma, mb;
        }
      }
      mma.store mc, output_s.subspan(MMA_M, MMA_N).at(warp_m, warp_n);
    }
    dma.copy output_s => output.subspan(TILE_M, TILE_N).at(block_m, block_n);
  }
}
```

Listing 3.55: One group, one tile: the simplest MMA matmul

`__co__` void and in-place output. The kernel returns nothing;

Block grid. `cdiv(M, TILE_M)` is ceiling division — how many tiles along M, including partial tiles. `block_m` and `block_n` pick which output tile this block owns.

With all dimensions 16, extents are 1×1

Section 3.6 goes deeper on `subspan` versus `chunkat`.

Quick ‘subspan‘ syntax reminder

`subspan(h, w)` declares a fixed-size rectangular view shape, and `.at(i, j)` selects which view instance you want from the parent tensor. In other words, `lhs.subspan(TILE_M, TILE_K).at(block_m, iv_k)` means "take the `[TILE_M, TILE_K]` tile at `(block_m, iv_k)`." Use `subspan` when you want an explicit tile shape; use `chunkat` when you want to slice by equal chunks of an existing dimension.

The four-step syntax reads linearly and the tile geometry is obvious.

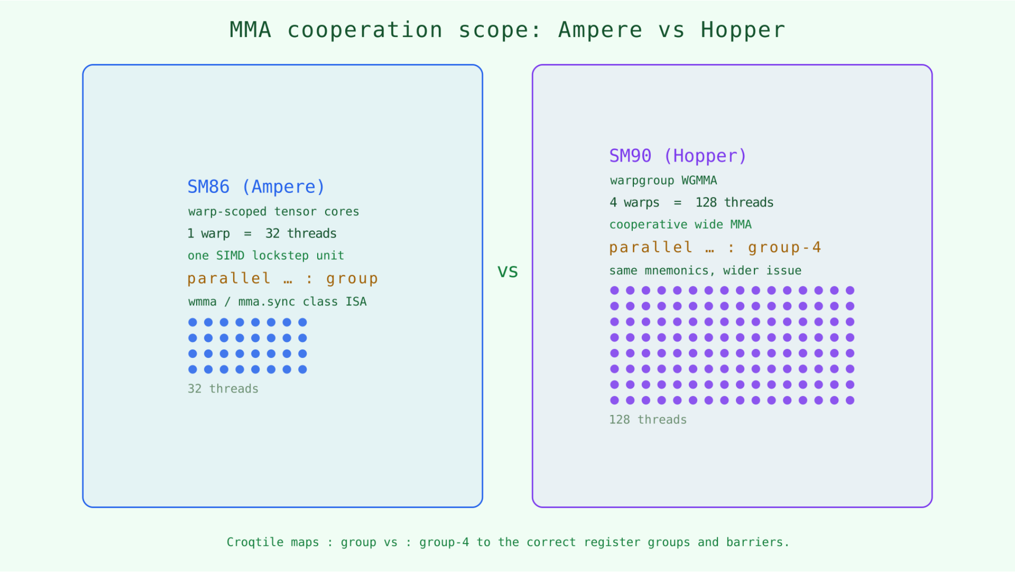
3.7.2 Worked Example: Widening the Cooperation Scope

Widening the cooperation scope changes the kernel in a few lines. This section compares the two forms side by side.

Widening the scope: cooperative group and mma

But the four-step syntax does not change. The only thing that changes in Croqtile is the space specifier: `: group-4` instead of `: group`.

```
#define TILE_M 128
#define TILE_N 128
#define TILE_K 128
#define WARP_M 64
#define WARP_N 64
#define WARP_K 16
__co__ void matmul_wgmma(global f16 [M, K] lhs, global f16 [N, K] rhs, global
    f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
```


MMA cooperation scope: Ampere vs Hopper
SM86 (Ampere)
warp-scoped tensor cores
1 warp = 32 threads
one SIMD lockstep unit
parallel … : group
wmma / mma.sync class ISA
32 threads
VS
SM90 (Hopper)
warpgroup WGMMA
4 warps = 128 threads
cooperative wide MMA
parallel … : group-4
same mnemonics, wider issue
128 threads
Croqtile maps : group vs : group-4 to the correct register groups and barriers.

```
    shared f16 [WARP_M, TILE_K] lhs_load_s;
    shared f16 [WARP_N, TILE_K] rhs_load_s;
    mc = mma.fill.f16 0.0f;
    foreach {iv_k} in [cdiv(K, TILE_K)] {
      dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s;
      dma.copy rhs.chunkat(block_n, iv_k) => rhs_load_s;
      foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
        parallel p by 1 : group-4 {
          ma = mma.load lhs_load_s.chunkat(_, iv_warp);
          mb = mma.load rhs_load_s.chunkat(_, iv_warp);
          mma.row.row mc, ma, mb;
        }
      }
    }
    shared f16 [WARP_M, WARP_N] output_s;
    mma.store mc, output_s;
    dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m, block_n);
  }
}
```

Listing 3.56: Widening the scope: cooperative group and mma

Fill, load, multiply, store — the rhythm is identical. The differences are mechanical:

`.f16`, `.f32`, etc. FP16 operands with FP32 accumulation is a common pattern for long K dimensions, avoiding numerical overflow in partial sums. The `0.0f` means the initial value is of type `f32` (there is no literal `f16` in the Croqtile). Casting will be performed to `f16` in the fill operation. If `.f16` is not specified, the initial value will be inferred from the type of the `ma` and `mb` operands.

`chunkat(_, iv_group)`. `_` means "do not tile that dimension" — Only K is subdivided per MMA slice.

That is the whole point of the abstraction: the same four operations, the same layout contract, the same `chunkat` / `subspan` expressions. You think about *what* contraction to perform; the cooperation width is a deployment detail.

3.7.3 Worked Example: Two Cooperative Groups

Two cooperative groups can share operands. This section shows the tiling and the resulting program.

Tiling further: two cooperative group sharing operands

Section 3.4 introduced `parallel p1 by 2 :  group-4` — With MMA, both groups can share the same B tile while loading different rows of A.

```
#define TILE_M 128
#define TILE_N 128
#define TILE_K 128
#define WARP_M 64
#define WARP_N 64
#define WARP_K 16
__co__ void matmul_wgmma_2group(global f16 [M, K] lhs, global f16 [N, K] rhs, global f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, TILE_M), cdiv(N, WARP_N)] : block {
    shared f16 [TILE_M, TILE_K] lhs_load_s;
    shared f16 [WARP_N, TILE_K] rhs_load_s;
    shared f16 [TILE_M, WARP_N] output_s;
    mc = mma.fill.f32 0.0f;
    foreach {iv_k} in [cdiv(K, TILE_K)] {
      dma.copy lhs.subspan(TILE_M, TILE_K).at(block_m, iv_k) => lhs_load_s;
      dma.copy rhs.chunkat(block_n, iv_k) => rhs_load_s;
      parallel p1 by 2 : group-4 {
        foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
          ma = mma.load lhs_load_s.subspan(WARP_M, TILE_K).at(p1, 0).chunkat(_, iv_warp);
          mb = mma.load rhs_load_s.chunkat(_, iv_warp);
          mma.row.row mc, ma, mb;
        }
      }
    }
    parallel p1 by 2 : group-4
      mma.store mc, output_s.subspan(WARP_M, WARP_N).at(p1, 0);
    dma.copy output_s => output.subspan(TILE_M, WARP_N).at(block_m, block_n);
  }
}
```

Listing 3.57: Tiling further: two cooperative group sharing operands

Splitting M with `p1`. With `TILE_M = 128` and `group_M = 64`, the block spans 128 rows; `p1` selects the upper or lower 64-row strip. Both use the same `rhs_load_s`.

The four-step syntax stays invariant; only the parallel decomposition of the tile changes.

Aspect			
Operand sharing	N/A	N/A	B tile shared, A rows split by `p1`
Threads	32	128	256
Accumulator	`mma.fill 0.0`	`mma.fill.f16 0.0f`	`mma.fill.f32 0.0f`

3.7.4 MMA Syntax Summary

The contraction syntax is gathered here for quick reference.

Syntax	**Meaning**
`mc = mma.fill 0.0`	Initialize accumulator tile to zero
`mma.fill.f16 0.0f` / `mma.fill.f32 0.0f`	Accumulator with explicit precision
`mma.load.swiz<N> src`	Load with swizzle mode (see *Ch 7*)
`mma.row.row mc, ma, mb`	C += A × B (both row-major)
`mma.row.col mc, ma, mb`	C += A × B (A row-major, B col-major)
`mma.row.row.sp mc, ma, mb, me`	Sparse MMA with metadata operand
`mma.row.row.scale mc, ma, mb, sc_a, sc_b`	Fused MMA + per-tile dequantization
`mma.scale mc, sc_a, sc_b`	Standalone post-MMA scaling
`mma.store.transp mc, dst`	Write accumulator transposed
`cdiv(a, b)`	Ceiling division
`__co__ void fn(...)`	Kernel that writes results in-place

Syntax at a glance

3.7.5 MMA Design Summary

This section summarises what the four-step design achieves and where its limits are.

Topic	**Takeaway**
2D tensor contraction	C += A × B on tile-shaped operands — the universal inner kernel
Four-step abstraction	**fill** → **load** → **multiply** → **store**; the compiler handles fragment layouts for each target
Scaling cooperation	Syntax stays invariant
Layout contract	`mma.row.row`, `mma.row.col`, etc.
Sparse and quantized	`.sp` adds a metadata operand; `.scale` fuses per-tile dequantization
Swizzle and pipeline	`mma.load.swiz<N>` matches swizzled shared layouts; `mma.commit` fences pipeline stages

In short The contraction is fast, but loads and compute still take turns — Section 3.8 introduces **role specialization** so different thread groups can play different roles: one group loading data while another computes, overlapping memory and math in time.

3.8 Coordination, Pipelines, and Persistent Scheduling

Pipelines and persistent kernels are built from concurrent regions and events. This section explains the coordination primitives and the protocols they implement.

Control Flow: Divergence, Concurrency, and Coordination

Every programming language that targets a parallel machine must answer one question: **what happens when different threads need to do different things?**

On a CPU, the answer is trivial — each core has its own program counter, so each thread runs whatever code it likes. — It must **serialize** the divergent paths and mask inactive threads.

All threads evaluate the condition; threads where it is false are masked (deactivated); This is **predicated execution** — simple, universal, and sometimes ruinously expensive.

But predicated execution is not enough. This is not a data-dependent branch — it is two **structurally different programs** that happen to share an address space. Trying to express this with `if` means one program pauses while the other runs. There is no overlap, no pipeline.

Croqtile introduces two additional control-flow primitives to fill this gap:

- **`inthreads.async` — structured concurrent regions**: compile-time partitioning of threads into groups that run **different programs simultaneously**. The compiler generates separate instruction streams;
- **`shared event` / `wait` / `trigger` — inter-region signaling**: lightweight synchronization tokens that let concurrent regions communicate safely.

Structured concurrent regions with `inthreads.async`

`inthreads.async` solves a fundamentally different problem than `if`. Instead of asking "should this thread execute this code?" at runtime, it says "this group of threads runs this program, that group runs that program" at **compile time**.

```
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) {
  }
  inthreads.async (p1 == 1) {
  }
}
```

Listing 3.58: Structured concurrent regions with inthreads.async

The distinction from `if` is structural, not just performance:

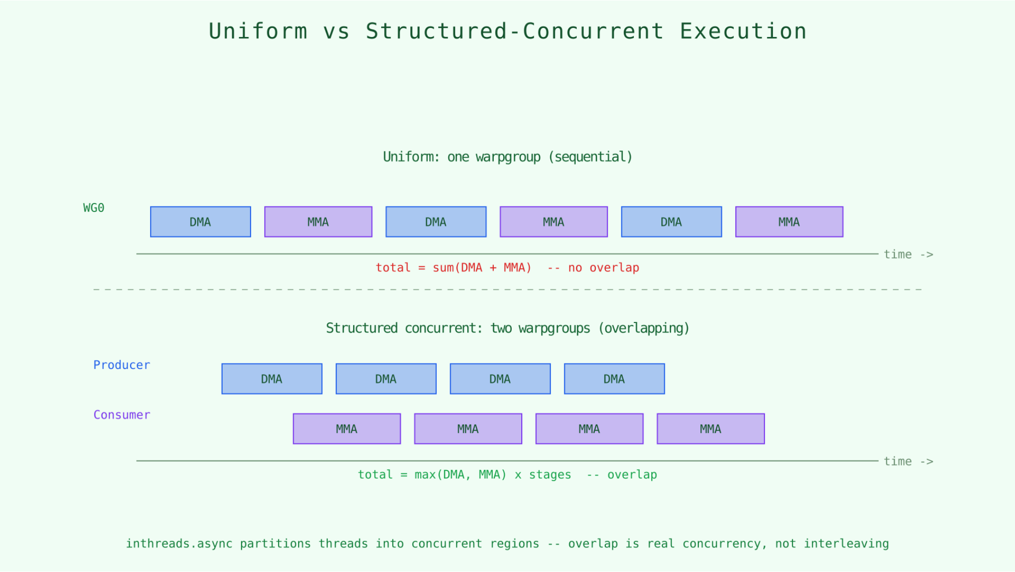
	`if` (predicated execution)	**`inthreads.async` (structured concurrency)**
Resolution	Runtime — every thread evaluates the condition	Compile time — thread assignment is fixed
Instruction streams	One program; divergent threads masked	Separate programs per region
PL analogy	`if`/`else` in any language	`async`/`spawn` in structured concurrency (Trio, Go goroutines, Cilk)

Why "structured"? The regions are lexically scoped — the compiler knows at parse time which threads belong to which region. There is no dynamic spawn, no unbounded concurrency. Each `inthreads.async` block is a static partition.

The `.async` modifier. Without `.async`, `inthreads` would execute regions sequentially — thread subsets take turns. This is analogous to the `async` keyword in structured concurrency frameworks — it marks a region as independently schedulable.

The figure below shows the effect. — The producer's DMA and the consumer's MMA overlap in time:

Each waits for the other. Bottom: `inthreads.async` partitions into two concurrent programs — DMA and MMA overlap, roughly halving wall-clock time.


Uniform vs Structured-Concurrent Execution
Uniform: one warpgroup (sequential)
WG0
DMA
MMA
DMA
MMA
DMA
MMA
time ->
total = sum(DMA + MMA) -- no overlap
Structured concurrent: two warpgroups (overlapping)
Producer
DMA
DMA
DMA
DMA
Consumer
MMA
MMA
MMA
MMA
time ->
total = max(DMA, MMA) x stages -- overlap
inthreads.async partitions threads into concurrent regions -- overlap is real concurrency, not interleaving

Inter-region signaling with events

When `inthreads.async` creates concurrent regions, those regions need a way to communicate. Croqtile provides **events**

```
shared event full;
shared event empty;
```

Listing 3.59: Inter-region signaling with events

Events have two operations:

- `trigger name` — signal that a condition is met (e.g., "data is ready")
- `wait name` — block until the corresponding `trigger` fires

The producer calls `trigger full` after writing a tile to signal "data ready." The consumer calls `wait full` before reading, blocking until the signal arrives. Symmetrically, the consumer triggers `empty` after finishing its read (the buffer can be reused), and the producer waits on `empty` before writing the next tile.

This is a **credit-based bounded buffer** protocol — `full` is the "data available" credit; `empty` is the "buffer free" credit.

Synchronization in Practice: Pipelines, Buffers, and Events

Section 3.8 introduced three control-flow primitives: `if` for predicated execution, `inthreads.async` for structured concurrent regions, and **events** (`shared event` / `wait` / `trigger`) for inter-region signaling. This part of the guide puts them to work. You will see two progressively complex kernel patterns, each demonstrating how the primitives compose to solve real synchronization problems.

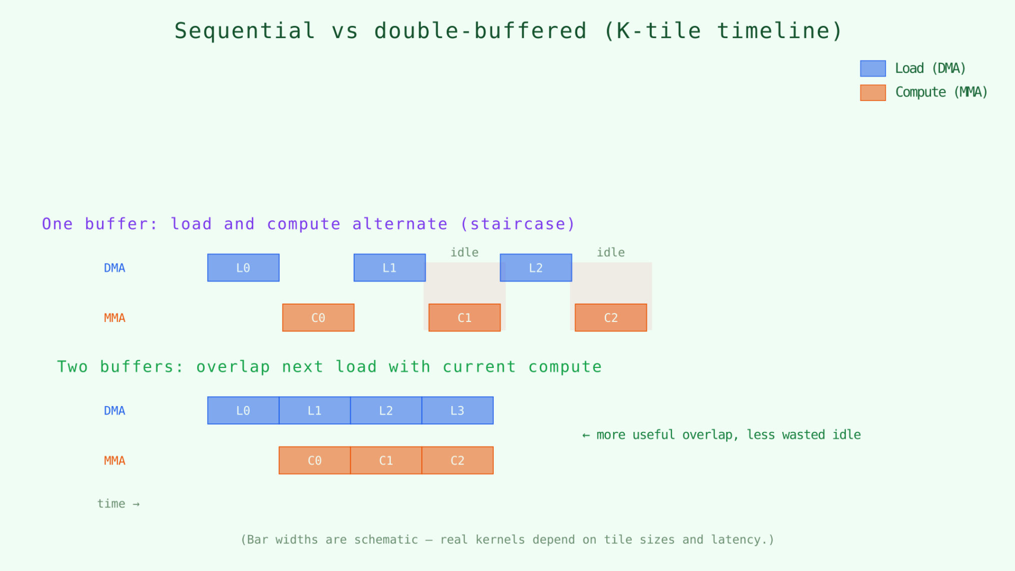
The first pattern — **double buffering with `swap`** — uses a single thread group that interleaves loading and computing within one program. No `inthreads.async`, no events: just two buffer handles and a rotation. The second pattern — **the full 1P1C event pipeline** — splits loading and computing into separate concurrent programs with `inthreads.async` and coordinates them with event arrays.

Both patterns solve the same underlying problem: **you cannot read a buffer while someone is writing to it**. They differ in how they structure the solution.

Why you cannot "just overlap" without events

The `swap` pattern works because one thread group controls both loading and computing — it knows the order. They cannot share a `swap` schedule; they need a signaling mechanism.

The picture below contrasts a strict load-then-compute staircase with double-buffered overlap: same logical work, less idle time.


Sequential vs double-buffered (K-tile timeline)
Load (DMA)
Compute (MMA)
One buffer: load and compute alternate (staircase)
DMA
L0
L1
idle
L2
idle
MMA
C0
C1
C2
Two buffers: overlap next load with current compute
DMA
L0
L1
L2
L3
MMA
C0
C1
C2
← more useful overlap, less wasted idle
time →
(Bar widths are schematic — real kernels depend on tile sizes and latency.)

Make it Asynchronous

Similar to the `parallel-by` statements, it is possible to make the threads with/without executing the divergent code be asynchronous. The following code showcases a example that conduct MPMD execution in the SPMD code:

```
__co__  void foo() {
  parallel p by 6 {
    inthreads.async (p <= 2) {
      // divergent code for thread 0, 1, 2 (thread-mask: 000111)
    }
    inthreads.async (p > 2) {
      // divergent code for thread 3, 4, 5 (thread-mask: 111000)
    }
    sync.shared; // sync point
                 // all ’inthreads’ paths must get executed
  }
}
```

Listing 3.60: Make it Asynchronous

In this example, it divides the parallel threads evenly into two sub-groups, without forcing synchronization after each `inthreads` block. In this way, threads in different groups are executed in parallel, mimicking a MPMD executing. However, the `sync.shared` statement established a **synchronization point** for all threads. That makes sure code for both paths get executed after this *synchronization point*.

Note that, only the outer `inthreads` can be declared as asynchronous. Programmers will get an compile-time error when using `inthreads.async` for any inner `inthreads` block.

Event Instances

In Croqtile, events are associated with the storage specifier, either `global`, `shared`, and `local`. Such storage specifier specifies the event instances.

```
__co__ void foo() {
  global event ge;      // single instance in the tileflow program
  parallel p by 2 {     // shared level
    shared event se;    // two instances, 1 for each shared instance
    parallel q by 6 {   // local level
      local event le;   // 12 instances in total, 6 for each shared instance
    }
  }
}
```

Listing 3.61: Event Instances

The comments in the above code explain the event instances. The mechanism is same to the spanned data declarations, where storage specifier is also required. From implementation perspective, event can be implement as the leveled boolean data within the specified storage. Therefore, it also takes the storage of this level. However, in concept, the event belongs to a specific level. Therefore, its instance count coresponds to its parallel level.

DeadLocks

By default, the event is lock-free for high-performance synchoronization between different asynchronous blocks. That implies it is risky when multiple threads try to update the event, especially for `wait`.

For example,

```
parallel p by 1 {
  shared event e;
  trigger e;    // one thread set e;
  parallel q by 2 {
    wait e;     // two threads can reset e;
  }
}
```

Listing 3.62: DeadLocks

In this code, two threads wait a single event `e`. However, since `wait` resets the event `e`, it is possible that `thread-0` resets the event before `thread-1` find the `e` has been triggered. As a result, `thread-1` is never unblocked and deadlocking happens.

```
qc = 2;
parallel p by 1 {
  shared event e[qc];
  trigger e;    // one thread set e[0] and e[1];
  parallel q by qc {
    wait e[q]; // no dead lock
  }
}
```

Listing 3.63: DeadLocks

In this code, you utilize an event array to guarantee each event is waited and reset by a single thread.

3.8.1 Canonical Pattern: One Producer and One Consumer

The smallest useful pipeline has one producer and one consumer. This section establishes that pattern before adding stages.

The canonical pattern: 1 producer + 1 consumer

The most common use of `inthreads.async` is the **1P1C** (one producer, one consumer) split for matmul:

```
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) {
  }
  inthreads.async (p1 == 1) {
  }
}
```

Listing 3.64: The canonical pattern: 1 producer 1 consumer

`parallel p1 by 2 : group-4`

`inthreads.async (p1 == 0)`

`inthreads.async (p1 == 1)` — The two blocks are separate programs sharing an address space.

But sharing an address space is exactly what makes this dangerous. Without coordination, the consumer might read a buffer before the producer has finished writing it. This is where events come in.

3.8.2 Worked Example: A 1P1C Kernel Skeleton

A 1P1C kernel skeleton shows the producer and consumer regions in code. This section reads it in full.

A 1P1C matmul skeleton

Event-based synchronization is left out of this skeleton; Section 3.8.5 adds the full pipeline protocol. Focus on the program structure:

```
__co__ void matmul(global f16 [M, K] lhs, global f16 [N, K] rhs, global f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
    shared f16 [WARP_M, TILE_K] lhs_load_s;
    shared f16 [WARP_N, TILE_K] rhs_load_s;
    shared f16 [WARP_M, WARP_N] output_s;
    parallel p1 by 2 : group-4 {
      inthreads.async (p1 == 0) {
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s;
          dma.copy rhs.chunkat(block_n, iv_k) => rhs_load_s;
        }
      }
      inthreads.async (p1 == 1) {
        mc = mma.fill.f16 0.0f;
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            ma = mma.load lhs_load_s.chunkat(_, iv_warp);
            mb = mma.load rhs_load_s.chunkat(_, iv_warp);
            mma.row.row mc, ma, mb;
          }
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m, block_n);
      }
    }
  }
}
```

Listing 3.65: A 1P1C matmul skeleton

Producer `foreach` — Walks K with `cdiv(K, TILE_K)` steps;
Consumer `mma` path
Missing coordination — Both sides loop over K independently. The consumer assumes each K-slab is ready when it reads. Making that assumption correct requires events (Section 3.8.5).

3.8.3 Persistent Scheduling

A persistent kernel decides its own work rather than being launched once per tile. This section covers the guard and the scheduling loop.

In Chapters 3–4, the grid grew with the problem: roughly one block per output tile. For large matrices that means large launch counts, and the last wave of blocks often leaves SMs partially idle — **tail underutilization**.

A **persistent kernel** fixes the launch size (often near the SM count) and lets each block iterate over multiple tiles:

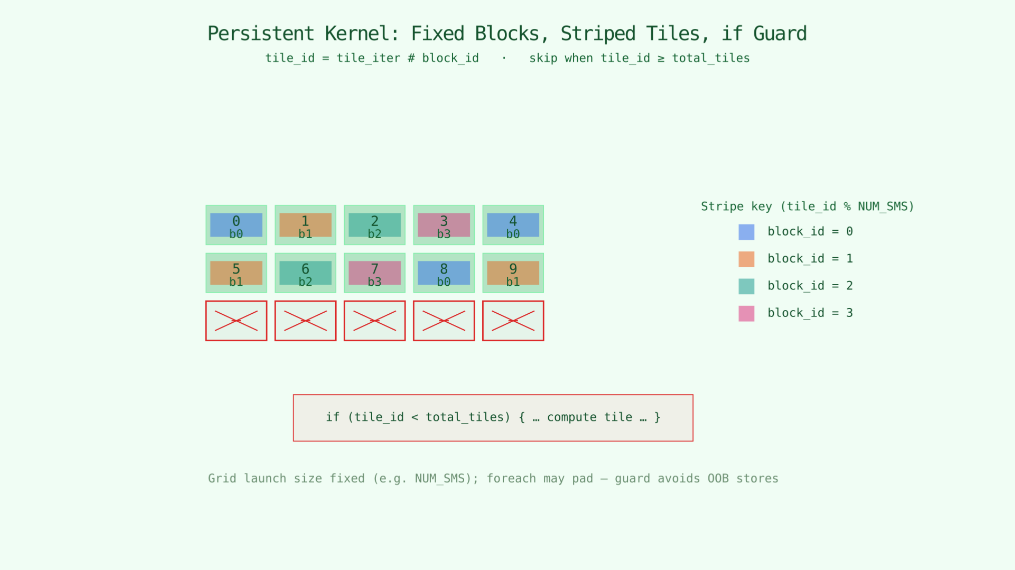
```
__co__ void matmul(global f16 [M, K] lhs, global f16 [N, K] rhs, global f16 [M, N] output) {
  int total_tiles = cdiv(M, WARP_M) * cdiv(N, WARP_N);
  parallel block_id by NUM_GROUPS : block {
    shared f16 [WARP_M, TILE_K] lhs_load_s;
    shared f16 [WARP_N, TILE_K] rhs_load_s;
    shared f16 [WARP_M, WARP_N] output_s;
    foreach {tile_iter} in [cdiv(total_tiles, NUM_GROUPS)] {
      tile_id = tile_iter # block_id;
      if (tile_id < total_tiles) {
        block_m = tile_id / cdiv(N, WARP_N);
        block_n = tile_id % cdiv(N, WARP_N);
        mc = mma.fill.f16 0.0f;
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s;
          dma.copy rhs.subspan(WARP_N, TILE_K).at(block_n, iv_k) => rhs_load_s;
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            parallel p by 1 : group-4 {
              ma = mma.load lhs_load_s.chunkat(_, iv_warp);
              mb = mma.load rhs_load_s.chunkat(_, iv_warp);
              mma.row.row mc, ma, mb;
            }
          }
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m, block_n);

      }
    }
  }
}
```

Listing 3.66: Persistent scheduling and the if guard

`parallel block_id by NUM_GROUPS : block` — Fixed worker count.

`tile_id = tile_iter # block_id` — Composes iteration with block index to stripe across tiles.

`if (tile_id < total_tiles)` — The `if` guard: a runtime predicate that skips the body for padding iterations. This is exactly the use case `if` is designed for — a data-dependent decision, not a structural partition.


Persistent Kernel: Fixed Blocks, Striped Tiles, if Guard
tile_id = tile_iter # block_id · skip when tile_id ≥ total_tiles
0 b0
1 b1
2 b2
3 b3
4 b0
5 b1
6 b2
7 b3
8 b0
9 b1
Stripe key (tile_id % NUM_SMS)
block_id = 0
block_id = 1
block_id = 2
block_id = 3
if (tile_id < total_tiles) { … compute tile … }
Grid launch size fixed (e.g. NUM_SMS); foreach may pad — guard avoids OOB stores

Data-dependent vs persistent grids

Aspect	One block per tile	Persistent (`NUM_GROUPS` blocks)
Grid size	Grows with problem	Fixed
Tail utilization	Last wave may leave SMs idle	All SMs stay busy
Extra constructs	Minimal	`total_tiles`, `tile_iter # block_id`, `if`
Complexity	Lower	Higher

3.8.4 Host-Level Concurrency and Stream Binding

Concurrency can also be expressed on the host side. This section covers `parallel.async` *and stream binding.*

`parallel.async`: host-level concurrency

Everything above runs inside a kernel. Sometimes you need concurrency at the **host level**: launch a grid without blocking the CPU.

```
parallel.async {px, py} by [grid_m, grid_n] : block {
  // kernel body
}
```

Listing 3.67: parallel.async: host-level concurrency

`parallel.async` returns control to the host immediately — the kernel is enqueued but the host does not wait for completion. This is the Croqtile equivalent of `launch_kernel` with a non-default stream.

This is **host orchestration**, orthogonal to in-kernel control flow. It does not replace `inthreads.async` for thread partitioning or `if` for runtime predicates — it decides *when* and *where* a grid runs relative to other grids.

Asynchronous program launch and Stream Binding

By default, the host waits for a `parallel-by` kernel to complete before continuing. To launch kernels asynchronously, use `parallel.async`.

```
__co__ void foo(stream s, f32[M] a) {
  parallel.async<s> p by M : block {
    a.at(p) = 1.0f;
  }
  // host continues immediately (no synchronization)
}
```

Listing 3.68: Asynchronous program launch and Stream Binding

Section 3.8.4 covers this in detail.

Synchronization and Asynchronous `parallel-by`

Similar to `inthreads`, `parallel-by` can also be made asynchronous by appending `.async`:

```
parallel.async p by 6 : block {
}
```

Listing 3.69: Synchronization and Asynchronous parallel-by

Without `.async`, the host waits for the kernel to finish before continuing:

```
parallel p by 6 : block {
}
// host continues only after the kernel finishes
```

Listing 3.70: Synchronization and Asynchronous parallel-by

With `.async`, the host code continues immediately after launching the kernel. This is essential for overlapping computation with data transfers or launching independent kernels concurrently.

In Croqtile, you can bind a `parallel-by` block to a specific stream using angle-bracket syntax:

```
__co__ void foo(stream s0, stream s1, f32[M] a, f32[N] b) {
  parallel.async<s0> p by M : block {
    a.at(p) = 1.0f;
  }
  parallel.async<s1> q by N : block {
    b.at(q) = 2.0f;
  }
}
```

Listing 3.71: Stream Binding

In this example:

- **`stream s0, stream s1`**: Stream parameters passed from the host.
- **`parallel.async<s0>`**: Launches the kernel asynchronously on stream `s0`.
- **`parallel.async<s1>`**: Launches a second kernel on stream `s1`.

Stream Binding without `.async`

Stream binding also works with synchronous `parallel-by`. In this case, the host launches the kernel on the specified stream and then waits for that stream to complete:

```
__co__ void bar(stream _s, f32[M] a) {
  parallel<_s> p by M : block {
    a.at(p) = 1.0f;
  }
  // host synchronizes on stream _s before continuing
}
```

Listing 3.72: Stream Binding without .async

Rules

- Stream binding is only allowed on **block-level** parallel (`: block`).
- The identifier inside `<...>` must be a `stream`-typed variable.
- Without stream binding, synchronous `parallel-by` uses device-wide

  synchronization, and asynchronous `parallel-by` uses the default stream.

3.8.5 Worked Example: A Full Event Pipeline

A full event pipeline combines the coordination primitives described above. This section walks through its stages and credit flow.

The full 1P1C event pipeline

This kernel combines `inthreads.async` (from Section 3.8) with event arrays to build a complete multi-stage pipeline. The producer and consumer run as separate concurrent programs, coordinated entirely through `wait` / `trigger`:

```
#define TILE_M 128
#define TILE_N 128
#define TILE_K 128
#define WARP_M 64
#define WARP_N 64
#define WARP_K 16
#define STAGES 2
__co__ void matmul(global f16 [M, K] lhs, global f16 [N, K] rhs, global f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
    shared event full[STAGES], empty[STAGES];
    shared f16 [WARP_M, TILE_K] lhs_load_s[STAGES];
    shared f16 [WARP_N, TILE_K] rhs_load_s[STAGES];
    shared f16 [WARP_M, WARP_N] output_s;
    parallel p1 by 2 : group-4 {
      inthreads.async (p1 == 0) {
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          stage = iv_k % STAGES;
          wait empty[stage];
          dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k)
            => lhs_load_s[stage];
          dma.copy rhs.chunkat(block_n, iv_k)
            => rhs_load_s[stage];
          trigger full[stage];
        }
      }
      inthreads.async (p1 == 1) {
        mc = mma.fill.f16 0.0f;
        foreach {s} in [STAGES]
          trigger empty[s];
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          stage = iv_k % STAGES;
          wait full[stage];
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            ma = mma.load lhs_load_s[stage].chunkat(_, iv_warp);
            mb = mma.load rhs_load_s[stage].chunkat(_, iv_warp);
            mma.row.row mc, ma, mb;
          }
          mma.commit;
          trigger empty[stage];
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m, block_n);

      }
    }
  }
}
```

Listing 3.73: The full 1P1C event pipeline

Walking through the kernel

Ring index. `stage = iv_k % STAGES` maps the unbounded K iteration to a fixed number of physical buffer slots — double buffering generalized to N buffers.

Producer path. For each `iv_k`, `wait empty[stage]` acquires a free slot. The `dma.copy` lines fill `lhs_load_s` / `rhs_load_s` at that stage. Then `trigger full[stage]` hands the slot to the consumer.

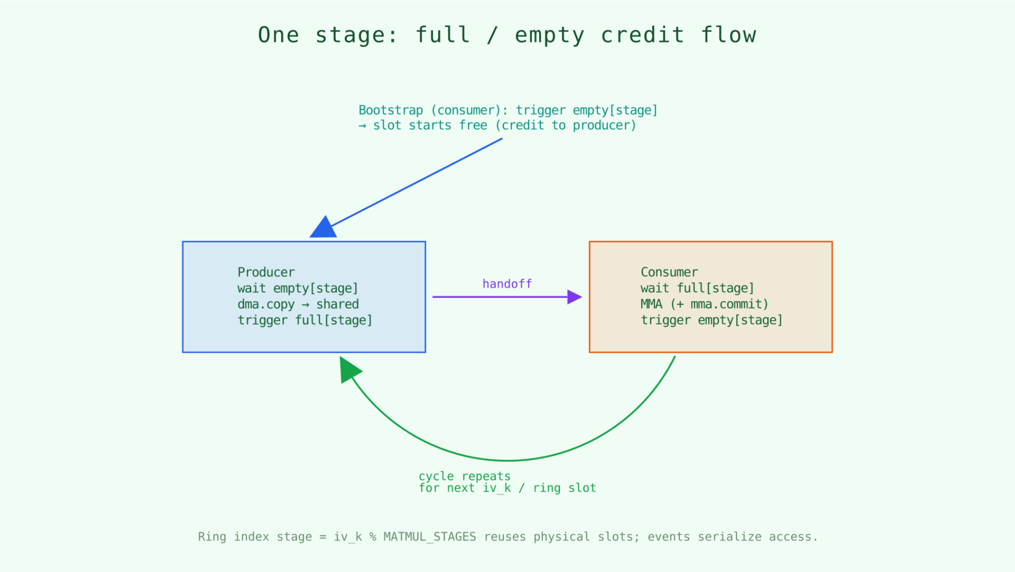
Consumer bootstrap. The loop `foreach {s} in [STAGES] { trigger empty[s]; }` runs **before** the K-loop so every stage starts with an `empty` credit. Without this, the producer blocks forever on its first `wait empty` — a deadlock.

Consumer path. Each `iv_k`: `wait full[stage]` blocks until the producer has filled that slot, then MMA over the tile, `mma.commit`, and `trigger empty[stage]` to release the slot for reuse.

`mma.commit` is the fence that completes one K-slab's contribution to `mc` before that stage's shared buffer may be reused. Omitting it risks reading stale data — the MMA might still be consuming operands when the producer overwrites the buffer.

Credit flow for one stage

The diagram matches the code: bootstrap grants empty credits; the producer waits on `empty`, fills, signals `full`; the consumer waits on `full`, computes, signals `empty`. When `iv_k` wraps modulo `STAGES`, the same physical stage re-enters the cycle.


One stage: full / empty credit flow
Bootstrap (consumer): trigger empty[stage]
→ slot starts free (credit to producer)
Producer
wait empty[stage]
dma.copy → shared
trigger full[stage]
handoff
Consumer
wait full[stage]
MMA (+ mma.commit)
trigger empty[stage]
cycle repeats
for next iv_k / ring slot
Ring index stage = iv_k % MATMUL_STAGES reuses physical slots; events serialize access.

3.8.6 Coordination Pattern Summary

The coordination patterns and their syntax are collected here.

In short The 1P1C skeleton above is incomplete: without `wait` / `trigger`, the consumer can read before the producer has finished writing. Section 3.8.5 adds the buffering and event protocols that make the pipeline safe, and walks through the resulting kernel stage by stage.

In short Both patterns achieve the same goal — overlapping memory and compute — but at different levels of complexity. Start with `swap` for simpler kernels;

The same synchronization patterns from this part of the guide, with richer data movement primitives underneath.

Concept	**Primitive**	**When to use**
Predicated execution	`if`	Data-dependent decisions (bounds, conditions)
Structured concurrency	`inthreads.async`	Compile-time thread partitioning (producer/consumer, heterogeneous roles)
Inter-region signaling	`shared event` / `wait` / `trigger`	Coordination between concurrent regions
Host concurrency	`parallel.async` / `stream s`	Multi-kernel overlap, non-blocking launch
Persistent scheduling	`if` + `foreach` + `#`	Fixed grid size, tile striping with padding guard

Pattern	**Primitives used**	**When to use**
`swap` double buffering	`dma.any`, `swap`	Single thread group interleaving load and compute

3.9 C++ Interoperability and the Preprocessor

Croqtile does not replace C++; it sits inside it. This section covers the boundary between the two, including calls, injected source and template instantiation.

C++ Interop: Device Functions, Inline Code, and the Preprocessor

Every high-level language needs an escape hatch to the layer below. Python has `ctypes` and C extensions. Rust has `unsafe` blocks and `extern "C"`. Java has JNI.

Croqtile's interoperability story has three parts:

1. **`__device__` functions** — The compiler passes them through unchanged.
2. `__cpp__`
3. **The preprocessor** — `#define`, `#if`, `#ifdef`, `#error` for compile-time configuration, the same role `#define` plays in C/C++ codebases.

The preceding sections have built a stack of abstractions: tiles, parallelism, MMA, pipelines, events, TMA. That is the Croqtile you want to live in. This part of the guide is about the times you need to step outside.

`__device__` vs ordinary C++ inline

Both `__device__` functions and ordinary C++ functions are passed through unchanged, but they land in different compilation paths — and the distinction matters.

A `__device__` function compiles to . The `__co__` body calls it with the `call` keyword, and the generated `__global__` kernel invokes it directly on the device.

An ordinary C++ function (no `__device__` or `__co__` annotation) compiles to **CPU code**. It runs on the host — setting up buffers, launching kernels, printing results. You cannot call a host function from inside a `__co__` body;

The boundary is strict: `__device__` lives on the device side of the `c++` split, host C++ lives on the host side. The `__co__` function is the bridge — the compiler transforms it into a `__global__` kernel (device side) and generates a host-side wrapper that launches it.

Difference with C++ pre-processing

One important notices about croqtile preprocessing is that it is triggered much earlier than c++ preprocessing. The workflow of croqtile compilation is as shown below:

```
chore-preprocessing -> croqtile compilation -> c/c++ preprocessing -> c/c++ compilation
```

Listing 3.74: Difference with C pre-processing

The primary target of croqtile preprocessing is to make host/device macros work as a whole, but such a workflow makes it possible sometimes different. From an implementation perspective, Croqtile pre-processor only substitute/conditionally-compile code inside the tileflow function, while leaving other pre-processing to the C++ preprocessor. That could restrict Croqtile pre-processing in a limited scope.

Pre-defined Macros

To mimic a target native compilation, croqtile preprocess also takes the builtin macros from the target. Consequently, these macros can be utilized inside tileflow functions as well as the host code.

Trigger C++ Template Instantiation

It is possible for Croqtile code to make function call to a template function. For example:

```
template<int M, int N, int K>
__device__ void matmul_kernel(int *lhs, int* rhs, int* output) {}
__co__ auto matmul(s32 [96, 72] lhs, s32 [72, 24] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel p by 6 {
    shared s32 [output.span / #p] buffer;   // shape: [16, 4]
    lhs_load = dma.copy lhs.chunkat(p, _) => shared;
    rhs_load = dma.copy rhs.chunkat(_, p) => shared;
    call matmul_kernel<buffer.span(0), buffer.span(1), 72>(lhs_load.data, rhs_load.data, buffer);
  }
  return output;
}
```

Listing 3.75: Trigger C Template Instantiation

In this case, it defines device function template named `matmul_kernel`, which takes three template parameters. As the caller specified the template function arguments, it triggers the instantiation of the function template, which results in a template function `matmul<16, 4, 72>` for the call.

So in Croqtile, calling a template function is similar to those in C++. However, **the template argument passed must be able to be inferred as compile time constant value** by Croqtile compiler. Therefore, any runtime values can result in error.

```
__co__ void foo(int M) {
  parallel p by 1 {
    call bar<M>();  // error: 'M' is a runtime value
  }
}
```

Listing 3.76: Trigger C Template Instantiation

Croqtile compiler could inference values as much as possible at compile time. If the template argument can not be inferred, the compilation will abort and error will be emitted.

More Questions

Like C++ functions, Croqtile functions can be defined anyway in the source code. One common raised question is about the **function signature**, as long as programmers may want to link the module with Croqtile function definition.

The answer is that Croqtile function follows the C++ calling convention, which makes it standard C++ function with C++ name mangling. Note that `extern "C"` is not possible for a Croqtile function, since it utilize C++ template-based object as its parameter type (spanned data).

However, there are some features yet to support:

- Croqtile function declaration without definition. This allows calling Croqtile function from another module.
- Croqtile function with `inline`. This allows the Croqtile function be used in headers.

Factor Target Specific Details

In some target like *factor*, it requires the device function to follow C-linkage, where the device functions must be prefixed with `extern "C"`. Since template functions can not be applies to functions with C-linkage, Croqtile has some wrapper tricks to enable it. For example:

```
__cok__ {
extern "C" void bar(int v) {};  // v is a compile-time constant
}
__co__ void foo() {
  parallel p by 1 { call bar(3); }
}
```

Listing 3.77: Factor Target Specific Details

Programmers can turn `bar` into a function template, like following:

```
__cok__ {
template <int v> void bar() {};
// compiler generate a explicit function template specialization:
// template<> void bar<3>() {};
}
__co__ void foo() {
  parallel p by 1 { call bar<3>(); }
}
```

Listing 3.78: Factor Target Specific Details

As the comment depicts, Croqtile compiler may generate a specialization version of function template `bar<3>`, which enables the function template as well. But note that, the function template no longer requires `extern "C" decoration` as those for normal *factor* device functions.

As the implementation is different with other targets, you may utilize `-kt` to enable such functionality for *croqtile-factor* compilation.

3.9.1 Worked Example: Calling a Device Helper

A device helper called from a `__co__` function shows the argument conventions in practice.

`__device__` functions: target alongside Croqtile

A custom reduction, a sorting network, a special math function — Croqtile does not need to manage these;

A `__co__` function can call a `__device__` function using the `call` keyword:

```
template <int K>
__device__ void warp_topk(f32* vals, s32* idxs);
template <typename T>
__device__ __forceinline__ T SHFL_XOR(T var, int lane_mask, int width) {
  return __shfl_xor_sync(uint32_t(-1), var, lane_mask, width);
}
__co__ void moe_topk(f32 [N_TOKENS, N_EXPERTS] scores,
                     s32 [N_TOKENS, K]& topk_ids,
                     f32 [N_TOKENS, K]& topk_scores,
                     int N_BLOCK) {
  parallel n by N_BLOCK : block {
    foreach m in [ |scores.span| / N_THREAD / N_BLOCK ] {
      shared_scores = dma.copy scores.chunkat(n#m, _) => shared;
      parallel gid by [N_THREAD / 32] : group
        parallel tid by 32 : thread {
          score = shared_scores.data.span_as(|shared_scores.data.span|).at(gid # tid);
          local s32 [K] frag_idx{-1};
          local f32 [K] frag_val{-1.0f};
          frag_idx.at(0) = tid;
          frag_val.at(0) = score;
          call warp_topk<8>(frag_val, frag_idx);
          inthreads (tid == 0)
            foreach k in K {
              topk_ids.at(n#m, gid#k) = frag_idx.at(k);
              topk_scores.at(n#m, gid#k) = frag_val.at(k);
            }
        }
    }
  }
}
```

Listing 3.79: device functions: target alongside Croqtile

`__device__ void group_topk<K>` — It operates on raw pointers (`f32*`, `s32*`), not Croqtile spans.

`__device__ T SHFL_XOR` — Croqtile passes them through.

`call group_topk<8>(...)` — The `call` keyword invokes a `__device__` function from within a `__co__` body. Arguments are passed by pointer; the compiler handles the address translation between Croqtile spans and raw pointers.

`inthreads (tid == 0)` — Note this uses `inthreads` without `.async` — a sequential thread filter, not a concurrent region.

This pattern — Croqtile handles the parallelism, tiling, and memory orchestration; — Is common in production kernels like MoE (mixture-of-experts) top-K, custom reductions, and specialized math.

3.9.2 Verbatim C++ Injection

Verbatim C++ injection lets a program carry target-specific snippets. This section shows the forms it takes.

storage hints: `setmaxnreg`

```
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) {
    __cpp__(R"(asm volatile("setmaxnreg.dec.sync.aligned.u32 40;");)");
    foreach {iv_k} in [cdiv(K, MATMUL_TILE_K)] {
      // ... TMA loads ...
    }
  }
  inthreads.async (p1 == 1) {
    __cpp__(R"(asm volatile("setmaxnreg.inc.sync.aligned.u32 216;");)");
    mc = mma.fill.f16 0.0f;
  }
}
```

Listing 3.80: storage hints: setmaxnreg

Placement — Put the hint at the top of each `inthreads.async` branch, before the heavy loop.

3.9.3 Interoperability Summary

The interoperability mechanisms and their rules are collected here.

Syntax	What it does	Runs on
`#define NAME value`	Object-like macro; shared across `__co__` and host regions	Preprocessor
`#if` / `#ifdef` / `#error`	Conditional compilation and compile-time assertions	Preprocessor
`-DNAME=value`	Define macros from the command line for tile sweeps	Preprocessor

In short Section 1.9 turns to the other side of the workflow: what to do when the kernel compiles but the output is wrong — debugging, verbose modes and systematic narrowing.

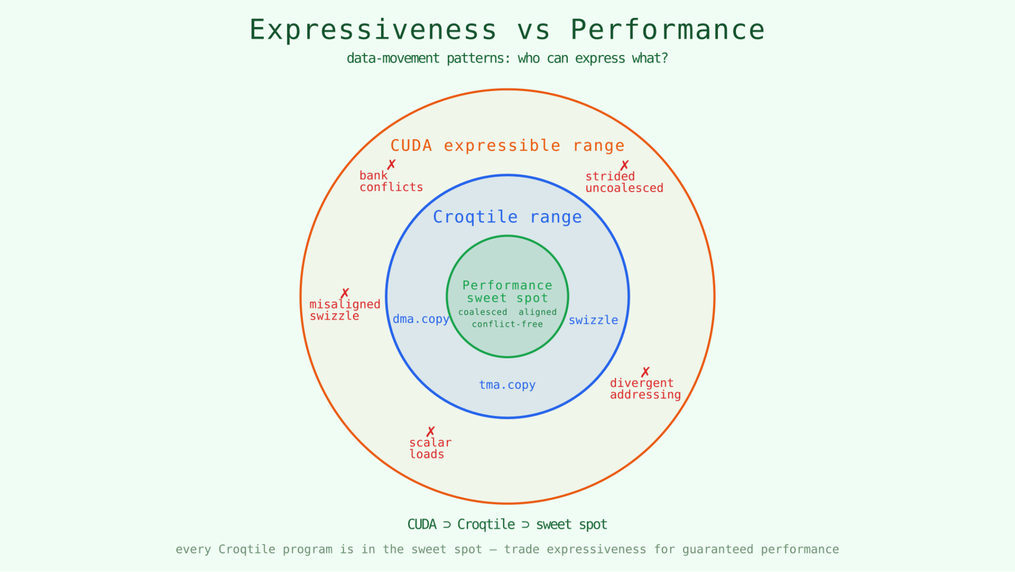
3.10 Tileflow Patterns and Design Rationales

Tileflow patterns and the design decisions behind the language are easiest to see in complete programs. This section presents two worked examples and the reasoning they illustrate.

Why the restricted interface works: expressiveness vs performance

This flexibility is a double-edged sword.

Croqtile takes the opposite approach: it restricts the expressible patterns to those that are **guaranteed to be in the performance sweet spot**. There is no way to accidentally write a strided, uncoalesced global load or a layout conflict shared-memory layout — the syntax simply does not allow it.


Expressiveness vs Performance
data-movement patterns: who can express what?
CUDA expressible range
✗ bank conflicts
✗ strided uncoalesced
Croqtile range
✗ misaligned swizzle
Performance sweet spot
coalesced aligned
conflict-free
dma.copy
swizzle
tma.copy
✗ divergent addressing
✗ scalar loads
CUDA ⊃ Croqtile ⊃ sweet spot
every Croqtile program is in the sweet spot – trade expressiveness for guaranteed performance

Croqtile's **`dma.copy`** follows the same principle — The trade is explicit: you give up the ability to write arbitrary data movement, and in return, every data movement you write is fast.

Easy to Use

You work on tensors and tiles, not raw buffers and pointers.

Compile-Time Safety

In development builds, 1,319 runtime assertions guard every transfer and memory access. Together they eliminate entire classes of bugs that would otherwise need **`memcheck`** and hours of printf debugging.

Dynamic Shapes

First-class symbolic dimensions let you write a kernel once and run it on any shape without recompilation. The compiler derives packed-K, metadata columns, grid dimensions, and shared-memory size automatically. Static and runtime memory are unified — no template metaprogramming, no boilerplate.

Born for AI Tuning

Compact, structured syntax keeps entire kernels inside AI context windows (30–60 lines vs. Structured error messages and well-documented CLI arguments let autonomous agents compile, profile, and iterate without human intervention.

Tiling Optimisation in Croqtile

This section will extend your discussion on tiling optimization in Croqtile by examining some practical examples.

3.10.1 Worked Example: Tiled matmul

A tiled matmul shows how bounded variables, `chunkat` *and movement statements compose into one program.*

Example 1: `matmul`

The first example is matrix multiplication on a device program. This example demonstrates how Croqtile's tiling and memory management mechanisms work together to optimize matrix operations, including how DMA (Direct Memory Access) is leveraged for efficient data movement.

Example Code: Matrix Multiplication with Tiling

```
__co__ s32 [512, 1024] matmul(s32 [512, 1024] lhs, s32 [1024, 1024] rhs) {
  // Device program
  s32[lhs.span(0), rhs.span(1)] output; // Use same shape as lhs
  parallel p by 1 {
    with index = {m_tile, k_tile, n_tile} in [1, 1, 1] {
      foreach m_tile, n_tile {
        shared s32[512, 1024] l2_out;
        foreach k_tile {
          // Initiate DMA operations
          lhs_load = dma.copy.async lhs.chunkat(m_tile, k_tile) => shared;
          rhs_load = dma.copy.async rhs.chunkat(k_tile, n_tile) => shared;
          parallel q by 1 {
            with index = {m_tile_s, k_tile_s, n_tile_s} in [4, 4, 8] {
              // Local buffers for tiling
              local s32 [128, 256] lhs_load_s_buffer_0 {0}, lhs_load_s_buffer_1 {0};
              local s32 [256, 128] rhs_load_s_buffer_0 {0}, rhs_load_s_buffer_1 {0};
              foreach index {
                // Add "after" primitive to chain up paired DMA operations
                lhs_load_s = dma.copy.async lhs_load.data.chunkat(m_tile_s, k_tile_s) => select(k_tile_s % 2, lhs_load_s_buffer_0, lhs_load_s_buffer_1) after lhs_load;
                rhs_load_s = dma.copy.async rhs_load.data.chunkat(k_tile_s, n_tile_s) => select(k_tile_s % 2, rhs_load_s_buffer_0, rhs_load_s_buffer_1) after rhs_load;
                out_load_s = dma.copy.async l2_out.chunkat(m_tile_s, n_tile_s) => local;
                wait out_load_s;
                // Perform the matrix multiplication (dot product) operation
                call dot_general_fp32(lhs_load_s.data, rhs_load_s.data, 256, 256, 256, 0, 8, out_load_s.data);
                // Store the result back to l2_out
                out_store_s = dma.copy.async out_load_s.data => l2_out.chunkat(m_tile_s, n_tile_s);
              }
            }
          }
          // Add "after" primitive to chain up DMA operations for the final output
          out_store = dma.copy.async l2_out => output.chunkat(m_tile, n_tile) after out_store_s;
        }
      }
    }
  }
  return output;
}
```

Listing 3.81: Example Code: Matrix Multiplication with Tiling

Memory Tiling with `chunkat`

In the matrix multiplication example, **memory tiling** is achieved using the `chunkat` method, which slices the input matrices `lhs` (left-hand side) and `rhs` (right-hand side) into smaller chunks to be processed in parallel.

Tiling via `chunkat`:

- The `lhs` and `rhs` matrices are tiled using the `chunkat` method. The chunks are defined by `m_tile`, `k_tile`, and `n_tile`, which represent different dimensions of the matrices involved in the multiplication.
- The chunks for both matrices are loaded asynchronously into the (`lhs_load` and `rhs_load`) with the `dma.copy.async` operation. This operation loads a portion of the matrix from a source buffer into a staging buffer, which is a common optimization in large matrix operations to improve data locality.

Bounded Variables:

- The variables `m_tile`, `k_tile`, and `n_tile` are **bounded variables**. They are defined in the context of `parallel-by` and `with-in` constructs, which define the iteration space for the tiling.
- These bounded variables allow the chunks to be computed in parallel, where each thread works on a different slice of the matrix. This parallelization enables efficient computation of matrix multiplications across multiple threads.

3.10.2 Worked Example: rgb2gray

An rgb2gray kernel shows the same composition operator on a different shape.

Example 2: rgb2gray

The second example is the RGB to Grayscale conversion algorithm. The algorithm converts a color (RGB) image to a grayscale image using a weighted average method. This example shows how physical parallelism work together with virtual parallelism.

Example Code

```
__co__ auto rgb2gray(f32 [N, 3, H, W] input) {
  f32 [N, H, W] out;
  parallel q by 6 {
    with index={n, h, w} in [N, H, W]/{#q, 16, 512} {
      foreach n, h, w {
        // _ means no tiling
        input_L1_A = dma.copy input.chunkat(q#n, _, h, w) => local;
        dims : input_L1_A.span[(0), (2), (3)];
        local f32 [dims] out_L1;
        call rgb2gray_kernel_fp32_fp32(input_L1_A.data, out_L1, |out_L1|, 1);
        dma.copy out_L1 => out.chunkat(q#n, h, w);
      }
    }
  }
  return out;
}
```

Listing 3.82: Example Code

3.11 Diagnostics and Methodology

Diagnostics are part of using a language well. This section collects the methodology behind compile-time inspection, verbose builds and runtime checks.

Debug and Verbose: Printing, Assertions, and GDB

The universal debugging discipline is the same everywhere: **systematically narrow the search space**, from cheap checks to expensive ones. Start with static analysis and compile-time assertions (free, catches whole classes of bugs). Move to targeted runtime probes (cheap, localizes the problem). Resort to interactive debuggers only when the cheaper tools have narrowed the suspect list.

Croqtile provides tools at each level: **compile-time shape printing** (`print!`, `println!`) to verify tile dimensions without launching a kernel, **runtime assertions** (`assert`) for invariant checks, **guarded device `println`** for runtime inspection of specific threads, **debug RTTI** for `debugger`, and a **verbose mode** (`-v`) to see what the compiler is actually doing under the hood.

Where to go next

Appendix A collects the syntax in one place. For the compiler design behind the language, see [LancerLab, 2026].

Appendix A

Syntax Quick Reference

A condensed index of the constructs used in this guide, grouped the way you meet them: declarations first, then iteration, tile selection and movement, then statements, coordination and diagnostics. Chapter 3 explains what each construct means and why it is defined that way; this appendix is for looking up a spelling without re-reading an example.

A.1 Declarations and shapes

Construct	Meaning
`__co__ T [shape] f(T [shape] a)`	Croqtile function with shaped parameters.
`__co__ void` / `__co__ auto`	In-place result / inferred return type.
`T [a.span] b;`	Buffer with a copied shape.
`a.span` / `a.span(i)`	Whole shape / one dimension.
`x : [7, 8]`	Named `mdspan` declaration.
`a = 1, 2, 3`	`ituple` declaration.
`tensor.at(i, j)`	Position access.

A.2 Iteration

Construct	Meaning
`parallel p by N` / `parallel p, q by [M, N]`	Independent instances.
`: block` / `: thread` / `: group` / `: group-4`	Annotations on a parallel axis.
`foreach idx in [N]` / `foreach t(1:)`	Ordered / sliced iteration.
`with idx in [a, b]`	Scoped index binding.
`p # m` / `#p`	Compose indices / axis extent.
`cdiv(a, b)`	Ceiling division.

A.3 Tile selection and movement

A.4 Statement modifiers and contraction

Construct	Meaning
`f = dma.copy src => shared`	Tile movement statement.
`f = tma.copy src => shared`	The same statement form on the other engine.
`a.chunkat(t)`	One tile of a regular partition.
`a.subspan(M, K).at(i, j)`	Explicit extents and tile coordinates.
`a.subspan(M, K).step(sM, sK).at(i, j)`	Strided tile selection.
`a.view(M, N).from(r, c)`	Window with explicit element offsets.
`a.span_as([dims])`	Shape reinterpretation without moving data.
`f.data` / `f.span` / `dma.any`	Moved tile / its shape / placeholder handle.
`.async` / `wait f` / `swap` / `rotate`	Async transfer and handle rotation.
`global` / `shared` / `local`	Storage specifiers.

Construct	Meaning
`dma.transp<...>` / `dma.pad<...>`	Transform while moving.
`.zfill` / `.swiz<N>`	Fill and layout modifiers.
`mc = mma.fill 0.0`	Zero an accumulator.
`ma = mma.load src.chunkat(...)`	Load an operand.
`mma.row.row mc, ma, mb`	Contraction with a layout contract.
`mma.store mc, dst`	Store the accumulator.
`.sp` / `.scale` / `mma.commit` / `mma.scale`	Contraction variants and stage fence.

A.5 Coordination, interop and diagnostics

Construct	Meaning
`if (expr)  ...`	Runtime predicate.
`inthreads.async (cond)` / `inthreads (cond)`	Concurrent region / sequential filter.
`shared event e[N]` / `trigger e` / `wait e`	Synchronisation tokens.
`parallel.async ...` / `stream s` / `yield`	Host scheduling and early return.
`__device__ fn()` / `call fn(args)`	Interoperating C++ function and its invocation.
`__cpp__("...")` / `__cpp__(R"(...)")`	Injected source text.
`#define` / `#if` / `#ifdef` / `#error` /`-D`	Preprocessor.
`print!` / `println!` / `print` / `println`	Compile-time and runtime output.
`assert` / `croq_assert`	Device and host assertions.

Where to go next

For the meaning behind a construct, return to the section that introduced it: Chapter 2 for the first reading and Chapter 3 for the design. To see a construct in a complete program, start with the worked kernels in Section 3.4.1, 3.6.1 and 3.8.5.

Bibliography

LancerLab. Croqtile compiler repository. https://github.com/LancerLab/croqtile, 2026.

Croqtile Programming Guide, version 1.0, November 2026.